# Computational Physics Project C: 2D, 2-spin Ising Model Final Report

Brian Hormbrey

30th March 2026

This report details the working principles of a python code designed to simulate the 2D 2-spin ising model using two different Monte-Carlo algorithms, namely the Metropolis and Wolff algorithms. The code is also validated by running temperature sweeps whilst using the same initial conditions and constants used by Lars Onsager in his 1944 paper, where he calculated the critical temperature to be 2.269185K [1]. Additionally validation tests against the analytical values of the critical exponents are also done. These critical exponents are known as $\alpha, \beta$ and $\gamma$. They have values of 0, 1/8 and -7/4 respectively. $\alpha$ was first originally derived by Onsager in 1944, $\beta$ was first calculated by C.N Yang in 1952 [2] and finally $\gamma$ was found in 1973 by E. Barouch [3].

# 1 Introduction

## 1.1 The 2D, 2-spin Ising Model

The 2D, 2-spin Ising model is the simplest mathematical model for ferromagnetism. It is useful for detecting a phase transition between ferromagnetic and paramagnetic phases in materials, since it models a lattice of atomic spins. In the 2D, 2-spin Ising model, there is a square lattice of atomic spins, where the lattice is finite and planar and the lattice occupies the x-y plane. The spins can point in either the positive or negative z directions, corresponding to 'spin-up' and 'spin-down' states. These spins can be flipped from 'spin-down' to 'spin-up' or vice versa depending on random selection and how energetically favourable the energy change from flipping a spin is. In a typical 2D Ising model, the system will always attempt to find the state of lowest energy, and for a lattice of atomic spins, it is most energetically favourable for a spin to have the same energy state as its nearest neighbours on a 2D lattice. This is because as time progresses with an ising model or a computer simulation of one, it will attempt to minimise its Helmholtz function F [4]:

$$F = U - TS = -k_B T ln(Z) \tag{1}$$

Where F is the Helmholtz free energy, U is the internal / thermal energy, T is the temperature and S is the entropy. Z is the partition function given by:

$$Z = \sum_i e^{-\beta J} \tag{2}$$

where $\beta$ is the Boltzmann energy weighting

$$-\frac{E}{k_B T} \tag{3}$$

In an Ising model simulation, U is what the program will attempt to minimise. When a simulation reaches a minimum energy state, the Ising model is said to have reached its minimum energy state, U (and therefore the partition function) is minimised and the simulation is said to have converged.

### 1.2 Uses of Ising Models

Though most commonly used in condensed matter physics, Ising models can also used as a simple model for modelling other systems apart from magnetic materials. These include scenarios related to machine learning [5], large scale astrophysical systems [6] and even social behaviour.

## 2 Theory

### 2.1 Phase Transition and critical temperature

Since the Ising model is fundamentally a model for ferromagnetism, the most common use case for it is modelling a phase transition within a material, between the ferromagnetic and paramagnetic states. A ferromagnetic material has all of its spins aligned in one direction, and is said to be magnetised. A material in a paramagnetic state has no ordering of the spins, and is not magnetised in any way. In the 2D Ising Model, the system temperature dictates the amount of energy that it takes to flip a spin, and hence the temperature directly controls on whether the system will spontaneously magnetise from a disordered state into an ordered state (paramagnetic to ferromagnetic phase transition). In Onsager's analytical 1944 solution to the Ising Model, he showed that for the set of initial conditions with $k_b = 1$, interaction constant J = 1 and no magnetic field, the critical temperature of the 2D Ising Model is exactly $t_{crit}$ = 2.269185K:

$$\frac{K_B T_c}{J} = \frac{2}{ln(1 + \sqrt{2})} \tag{4}$$

Below this temperature the lattice will show ordered magnetisation, at this exact temperature the lattice will begin to show signs of becoming disordered, and above this temperature the lattice will be completely disordered: The reason this occurs is because at higher temperatures, entropy dominates meaning there is more fluctuations in the system, and energy changes can

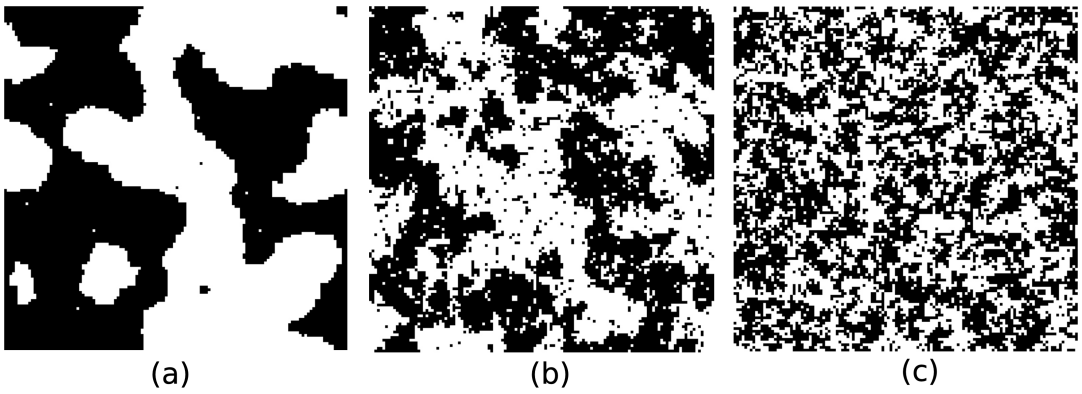


**Figure 1:** Converged computer simulations of Ising model lattices at three different temperatures from Farias 2024 [7]. Figure a) is below the critical temperature (shows an ordered state), figure b) is at around the critical temperature (phase transition from ordered to disordered states) and figure c is above the critical temperature, where it shows a completely disordered state.

only correlate a small group of spins very close to each other. At lower temperatures however, there are far fewer fluctuations, and in the Helmholtz Function (1), the TS term is able to dominate, and since spins cannot align due to the fluctuations there is no net magnetic moment. At lower temperatures however, the there are far fewer fluctuations, and the energy loss by aligning spins dominates, and the system is able to reach an ordered state.

## 2.2 Total energy

The total energy of the 2D, 2-spin Ising model is given by:

$$E = -\frac{J}{2}\sum_{i=1}^{N}\sum_{j=1}^{N}\left(\mathrm{nn}(i,j)\,\mathbf{s}_i\cdot\mathbf{s}_j\right) - \mu_{\mathrm{B}}\sum_{i=1}^{N}(\mathbf{B}\cdot\mathbf{s}_i) \tag{5}$$

Where the first pair of summations make up the interaction term, and the third summation being subracted off the interation term is the magnetic term. J is the exchange interaction constant, (nn(i, j)$s_i \cdot s_j$) represents a nearest neighbour interaction between neighbouring spin sites, $\mu_b$ is the Bohr magneton and B is the magnetic induction field.

## 2.3 Magnetisation

The magnetisation of a 2D Ising lattice is the mean alignment of every spin in the entire lattice. If a lattice is magnetised (which almost always happens at low temperatures below t_*crit*, all the spins will point in roughly the same direction, yielding an average alignment (and therefore magnetisation) of around 1 or -1 depending if the lattice has a positive or negative net magnetic moment. If the lattice is disordered and highly random, there is roughly an equal amount of positively and negatively aligned spins. Therefore the net magnetisation is close to 0 for

disordered systems at high temperatures. The equation for magnetisation is given by:

$$\mathbf{M} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{s}_i \tag{6}$$

This equation computes the sum of the spin's alignment over every spin in the lattice, and then divides the sum by the total number of spins in the lattice, yielding average spin alignment and therefore magnetisation. A special note on magnetisation is that since *absolute* magnetisation is around 1 for ordered systems, but close to 0 for disordered systems, absolute magnetisation must experience a rapid drop at the critical temperature. This can be used to calculate critical temperature over a large range of magnetisation samples by calculating the consecutive drop between magnetisations at each temperature and figuring out which two temperatures yield the most negative drop in magnetisation. The midpoint between the two temperatures between which there is the most negative magnetisation drop should be the critical temperature.

## 2.4 Magnetic Susceptibility

Magnetic susceptibility is a measure of how much a system responds to an external magnetic field. If the magnetic susceptibility is high, the system is easily magnetised by even a small magnetic field, whereas if the magnetic susceptibility is low, then it means the system is very hard to magnetise. The sign of the magnetic susceptibility determines whether a spin will align in the same direction as the field (positive susceptibility) or align in the opposite direction to the field (negative susceptibility). In the context of the Ising model however, magnetic susceptibility peaks at the critical temperature, because at the critical temperature, neither thermal fluctuations or spontaneous magnetisation dominates. This means large clusters of spins form at different length scales, and the system is therefore incredibly sensitive to even a small perturbation from a magnetic induction field [8]. The magnetic susceptibility is given by:

$$\chi = \lim_{H\to 0}\frac{\partial}{\partial H}\langle M\rangle = \frac{1}{k_{\mathrm{B}}T}\left(\langle M^2\rangle - \langle M\rangle^2\right) \tag{7}$$

The magnetic susceptibility is directly proportional to the variance, and inversely proportional to the temperature, but despite this, due to the behaviour of clusters of spins at around the critical temperature the magnetic susceptibility does not peak at the lowest temperature. However, this particular equation gives the susceptibility in terms of the entire system, rather than per spin. Comparing susceptibility per spin means that it is easier to compare systems of differing sizes, since susceptibility will be per spin.

$$\chi_{perspin} = \frac{1}{N}\lim_{H\to 0}\frac{\partial}{\partial H}\langle M\rangle = \frac{1}{N}\frac{1}{k_{\mathrm{B}}T}\left(\langle M^2\rangle - \langle M\rangle^2\right) \tag{8}$$

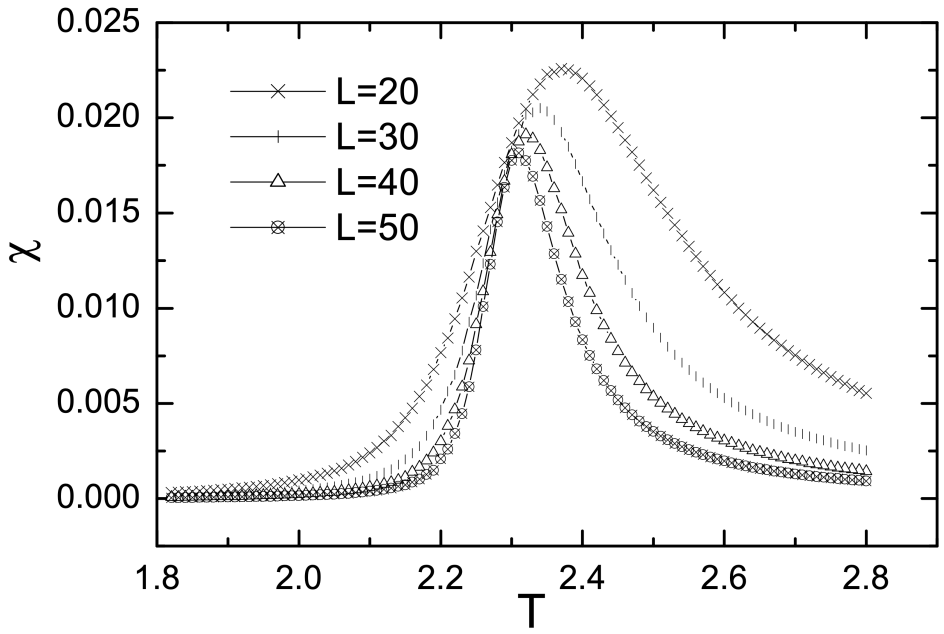


**Figure 2:** Figure from Min 2016 [9] showing how magnetic susceptibility peaks at around Onsager's 1944 value of 2.269185K for $T_{crit}$

## 2.5 Heat Capacity

The heat capacity of the Ising model is the amount of energy that can be absorbed by the system per kelvin. The specific heat capacity of the Ising model is given by:

$$C = \frac{\partial}{\partial T}\langle E\rangle = \frac{1}{k_{\mathrm{B}}T^2}\left(\langle E^2\rangle - \langle E\rangle^2\right) \tag{9}$$

The behaviour of the heat capacity is directly proportional to the variance of the energy rather than the magnetisation, and rather than inversely proportional to the temperature it is inversely proportional to the square of the temperature. This particular equation gives the heat capacity in terms of the entire system, and in order to more easily compare the clustering behaviour of spins within systems of different sizes, it is better to compute the heat capacity per individual spin rather than for the entire system. This is because a larger system contains more spins, and therefore has a higher heat capacity. If you compare heat capacity per spin, you can see how the heat capacity is affected by the behaviour of lattices rather than their size alone. Heat capacity per spin is given by

$$C_{perspin} = \frac{1}{N}\frac{\partial}{\partial T}\langle E\rangle = \frac{1}{N}\frac{1}{k_{\mathrm{B}}T^2}\left(\langle E^2\rangle - \langle E\rangle^2\right) \tag{10}$$

Like magnetic susceptibility, the specific heat capacity also peaks at the critical temperature, and the reason why this occurs is because as shown in [9], the system is on the knife edge between the ordered and the disordered phases, and therefore enormous clusters form at every length scales. Due to this, for a system at the critical temperature, there are huge changes in the amount of energy with time, and therefore the variance of the system peaks at critical temperature.

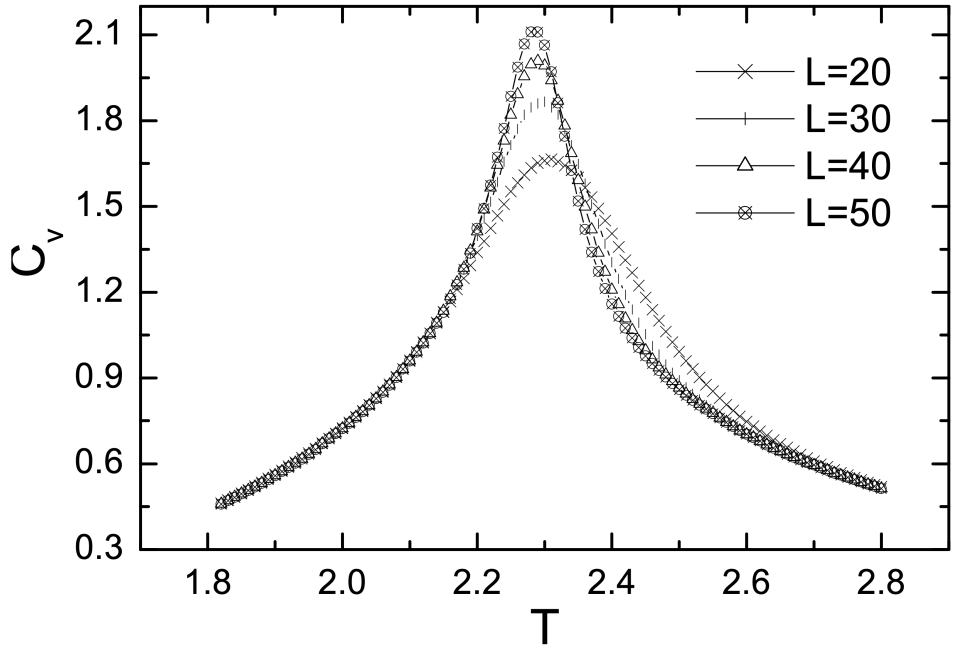


**Figure 3:** Another figure from Min 2016 showing how heat capacity varies with temperature, showing the peak in heat capacity caused by the energy variance also peaking at the critical temperature, since equation 9 shows how the heat capacity is directly proportional to the variance

## 2.6 Internal magnetic field strength

A 2D Ising model lattice also possesses its own magnetic field intensity, known as H. This field arises from the difference between the net magnetisation of the spins and the external magnetic induction field B.

$$H = \frac{1}{\mu_0} B - M \tag{11}$$

Where H is the internal magnetic field of the lattice, B is the external magnetic induction field and M is the lattice's magnetisation. This equation originates from classical electromagnetism since it is a rearranged version of

$$B = \mu_0(H + M) = \mu_0(1 + \chi)H \tag{12}$$

from [10] which gives magnetic induction field strength B from the magnetic field intensity and magnetisation. This is also equal to the intensity multiplied by vacuum permeability and $1+\chi$, where chi is the magnetic susceptibility. According to [10], any material that obeys equation (12) is a linear material, and since the code detailed further on in this report calculates H using equation (11) which is a rearranged form of equation (12), all Ising models simulated here are of a magnetically linear material.

## 2.7 Monte Carlo Algorithms

The code uses two different algorithms to simulate the Ising model. The first algorithm is the Metropolis algorithm, and it was first discovered in by Nicholas Metropolis in his 1953

paper [11]. The specific version of the metropolis algorithm used for this code is as follows:

1. Pick a random spin in the lattice.

2. Calculate the energy of the lattice in its current state using the total energy of the lattice based on the spin's nearest neighbours in its current state using (5). This value for the energy is the 'old' energy of the lattice $E_1$.

3. Flip the spin selected in step 1.

4. Recalculate the lattice's energy using (5). This recalculated energy value is the 'new' energy of the lattice $E_2$.

5. Calculate the change in energy caused by the spin flip:

$$\Delta E = E_1 - E_2 \tag{13}$$

6. If the energy change is negative, automatically accept the new spin flip, because the Ising model always tries to minimise the Helmholtz function, and the term in the Helmholtz function that can be minimised here is the internal energy, U.

7. If the energy change is positive, then a probabilistic comparison is made between a random probability, and the probability of that positive energy change occurring made based on the Maxwell Boltzmann Energy Distribution

$$P(E) = \exp(\frac{-\Delta E}{k_b T}) \tag{14}$$

   The random probability used in this code is the PCG64DXSM random number generator from the NumPy package in python, and the generator itself is described in [12] and was first published in a 2014 report by Melissa O'Neil [13].

   - If the probability of the positive energy change (a random probability generated using PCG64DXSM) is lower than the Maxwell Boltzmann Distribution probability, then the spin flip is undone (revert the lattice to its original state in step 2).
   - On the other hand if the random PCG64DXSM probability of the positive energy spin flip occurring is higher than the Maxwell Boltzmann distribution, then the spin flip is kept.

The second algorithm used to simulate the Ising model is the Wolff algorithm. It was first discovered by Ulli Wolff in 1989 [14]. It differs significantly from the much older metropolis algorithm in that rather than selecting individual spins, it grows a cluster of spins based on a cluster addition probability, defined as $P_{add}$. Whenever it runs out of spins to add to the cluster,

it will then flip the entire cluster. A very good description of the algorithm is provided in a 1997 paper by G. T. Barkema and M. E. J. Newman 1997 [15].

The Wolff algorithm is much faster at getting an Ising model lattice to reach equilibrium since it flips clusters rather than individual spins, but at the cost of Wolff's original 1989 algorithm not being able to account for an external magnetic induction field. A detailed description of the specific Wolff algorithm implementation in the code is as follows:

1. The probability of adding a spin to the cluster is defined as

$$P_{add} = 1 - \exp(-2\beta J) \tag{15}$$

   where beta is the Boltzmann energy weighting:

$$\beta = \frac{1}{k_b T} \tag{16}$$

2. The second stage of the Wolff algorithm is selecting the 'seed spin' to begin the growth of the cluster. This is done using the exact same PCG64DXSM random number generator as used in the metropolis algorithm.

3. Spins are added to the cluster based on if they are pointing in the same direction (up or down) as the seed spin, and if the probability $P_{add}$ is greater than a random probability from the random number generator. Spins added to the cluster also have to be nearest neighbours. This step has the chance to previously add spins in the cluster that have previously been rejected as explained by [15]. This can happen since they were either not nearest neighbours, and if they were nearest neighbours they were rejected due to random probability selection.

4. When the algorithm runs into the scenario that there are no more spins to add to the cluster (no nearest neighbours pointing in the same direction) then the entire cluster is flipped and the algorithm will start growing a new cluster of spins.

## 2.8 Onsager Critical Exponents

The Onsager critical exponents are power law relations that are valid near the critical temperature. Each exponent is related to a specific quantity of interest.

| Exponent | Value | Ising Model Quantity of Interest |
|---|---|---|
| $\alpha$ | 0 | Heat Capacity |
| $\beta$ | $\frac{1}{8}$ | Magnetisation |
| $\gamma$ | $\frac{-7}{4}$ | Magnetic Susceptibility |

**Table 1:** Table of Onsager's critical exponents and the quantities each one is related to.

The Onsager critical exponents are related to the specific heat capacity, magnetisation and the magnetic susceptibility by the following equations:

Exponent $\alpha$:

$$C \propto |T - T_c|^{\alpha} \tag{17}$$

Exponent $\beta$:

$$M \propto |T - T_c|^{\beta} \tag{18}$$

Exponent $\gamma$:

$$\chi \propto |T - T_c|^{\gamma} \tag{19}$$

## 2.9 Onsager Amplitude

The Onsager critical amplitude is 0.4945, and the expression that yields this first was derived by Onsager in 1944 [1]. This has also been verified both computationally (using the metropolis algorithm) and experimentally in [16]. The critical amplitude is a measure of how much the heat capacity per spin scales as the lattice side length $l$ increases in size by a factor of $e$. Therefore, the relation

$$C = Aln(l) \tag{20}$$

Will mean that the gradient of a graph of heat capacity against lattice side length $l$ will have an expected value of 0.4945. However, due to the laws of logarithms involving power laws, if the graph is plotted against lattice size D instead of lattice side length $l$, and letting $l^2 = D$ for a square lattice the equation becomes

$$C = C_0 ln(D) = C_0 ln(l^2) = 2C_0 ln(l) \tag{21}$$

Where $A$ is the Onsager critical amplitude, $D$ is the lattice size and $C_0$ is the scaling relation between logarithmic increases in lattice size rather than lattice side length. Therefore, from equations (20) and (21) it is easy to see that A is equal to $2C_0$ and therefore the expected value of $C_0$ is $C_0 = \frac{0.4945}{2} = 0.24725$

# 3 Code Description

The python simulation code for the Ising Model is a collection of functions in a Jupyter Notebook, with each function fulfilling a singular purpose in the code, save the plotting functions which also return errors as well as the plots. There are also four .py files used for multiprocessing with python's Pool module.

## 3.1 Main Jupyter Notebook

- **Imports**: The notebook imports everything required apart from the .py multiprocessing files to begin with. The core packages and modules it relies upon are:

    1. NumPy: For all array calculations [17].
    2. SciPy, specifically scipy.constants for getting the permeability of free space $\mu_0$, as well as performing linear regression fits when calculating quantities of interest [18].
    3. Matplotlib for all plotting [19]
    4. os (which is part of the python language itself [20] for reading in files and extracting the amount of CPU cores.
    5. tqdm.auto, which is part of the tqdm python package [21]
    6. Pool, which is also part of python's built in packages just like os. Pool is used for multiprocessing, which is used to spawn multiple instances of a process. In this code, multiprocessing is used to spawn multiple ising model simulations at once, each simulation with either a different temperature (multiple temperatures are sampled in order to detect the critical temperature) or multiple lattice sizes at the critical temperature to investigate the correlation between the natural logarithm of lattice size and heat capacity.

- **Sanity checks**:

    1. CPU core count and os package. First uses the os package to get the number of CPU cores present on the computer the notebook is running on. If the number returned is anything but an integer, the notebook throws a ValueError because if the number of cores isn't an integer, then the os package is not working correctly.
    2. If the core count check passes, then os is used again to verify that all four of the required multiprocessing files are in the same directory as the notebook. If any one of the files is not present, then the code will throw a ValueError and tell the user to put all the files in the same directory before attempting to run the notebook.

- **Functions**:

1. **File Reading Functions:** There are three file reading functions in this Ising Model Code. These file reading functions all read in .txt files. One function is for reading in a parameter file for a singular Ising Model simulation with a final lattice pixel plot, and the other two file reading functions read in parameter files for a sweep of multiple simulations for different temperatures whilst holding all other parameters constant, or lattice size whilst holding all other parameters constant.
2. **2D lattice initialisation** Initialises a random 2D N×$N$ lattice of spins as a NumPy array, with spin up represented by 1 and spin down represented by -1.
3. **MMC Spin flip energy change** Calculates the the energy of the entire 2D lattice using the Ising Model nearest neighbours hamiltonian.
4. **Metropolis Monte Carlo Step Function (MMC step()))** Performs a metropolis step using the MMC Spin flip energy change function and the metropolis algorithm.
5. **Wolff Monte Carlo Step Function (WMC step())** Performs a Wolff Monte Carlo Cluster Sweep using the Wolff algorithm. Is entirely self contained and does not require a function call to another function.
6. **Calculate total lattice energy** Calculates the energy of the entire lattice per spin using the Ising model hamiltonian. Differs from **MMC Spin flip energy change** by calculating lattice energy per spin rather than for the entire lattice, which enables lattices of different sizes to be compared. Uses np.roll to enforce periodic boundary conditions on the lattice (turns the lattice into a mathematical torus, where even edge spins have nearest neighbours on the other side of the lattice).
7. **Calculate total lattice magnetisation** Calculates the total lattice magnetisation per spin using equation (6).
8. **Calculate total lattice magnetic field** Extracts the lattice magnetisation using **Calculate total lattice magnetisation** and then uses equation (11) to calculate the intensity of the lattice's internal magnetic field.
9. **Calculate Linear Regression Fit** Uses scipy.stats to perform a linear regression fit of two arrays and calculate the best-fit linear gradient. This function is incredibly useful for performing linear fits of log-log plots, which is exactly what is used to calculate the critical exponents, or for C and ln(D) when calculating $C_0$ and validating it against Onsager's value of 0.24725.
10. **Straight line fit** Returns y = mx + c, used for plotting a straight line through fitted data.
11. **Function to run a single metropolis algorithm simulation with end system visualisation)**: Uses the file reading function for reading in the correct parameter file for a single Ising model simulation, and then using a for loop, runs **Metropolis**

**Monte Carlo Step Function (MMC step())** $n_{steps}$ times. As the simulation runs, the functions to calculate energy per spin, magnetisation and internal magnetic field are called with every timestep. These values are appended to lists and later converted into arrays for making plots of how those quantities vary with time. The heat capacity and susceptibility however rely on variance, and therefore the running variance needs to be computed. The method used to compute the running variance in this code is the Welford online algorithm [22]. The Welford accumulation algorithm only begins after the Ising model has passed an equilibration threshold, which in this code, is defined as any timestep past 60% of the total amount of steps. After this threshold is passed, the Welford algorithm begins:

(a) Two floats each for the running mean magnetisation and running mean energy are defined (running mean magnetisations 1 & 2, and running mean energies 1 & 2).

(b) The number of equilibrated samples is computed.

(c) For both magnetisation and energy, a two stage approach is used to compute the running variance: The first stage is computing the difference between the quantity per spin (calculated with every equilibrated step) and the running mean quantity. Then the running mean quantity per spin is re-calculated by dividing the calculated difference by the number of equilibrated steps. The second stage involves re-computing the difference calculation but using the re-assigned magnetisation difference, and then the second mean running quantity is assigned the product of the first and second differences.

(d) Then the overall running quantity variance is computed by dividing the second running mean value by the number of equilibrated samples.

Once the running variances of the magnetisation and energy per timestep have been computed using the Welford algorithm, they are then used to compute the magnetic susceptibility per spin and heat capacity per spin using equations (8) and (10) respectively. After the Ising model simulation finishes running, all lists of energy per spin, magnetisation per spin, internal magnetic field per spin, magnetic susceptibility per spin and heat capacity per spin are converted into arrays. Plots of all the quantities of interest with respect to timestep are made and saved with the parameter file in their png filenames, alongside a pixel plot of the lattice's final configuration.

12. **Function to run a single Wolff algorithm simulation with end system visualisation)**. Exactly the same as **Function to run a single metropolis algorithm simulation with end system visualisation)** but uses the Wolff algorithm instead of the Metropolis algorithm.

13. **Parallelised critical temperature and exponents investigation using the metropo-**

**lis algorithm.** Uses the metropolis algorithm to and multiprocessing (and therefore one of the four multiprocessing files; parallelised temperature investigation functions MMC.py) to run multiple simulations at the same time (it is much faster to use all available CPU cores to run multiple simulations at once rather than one simulation at a time) to perform a parallelised temperature sweep. This temperature sweep goes over the critical temperature, and with every simulation run, the equilibrated heat magnetisations, magnetic susceptibilities and heat capacities are calculated, and plotted with respect to temperature. The data from these simulations is used to calculate the critical temperature using four different methods:

(a) Between the two temperatures where the largest drop in absolute magnetisation occurs (phase transition between ordered and disordered states).

(b) Temperature where there is the maximum value of magnetic susceptibility (as seen in Min 2016).

(c) Temperature where there is the maximum value of heat capacity (as seen in Min 2016).

(d) Average of all three methods for highest statistical accuracy.

Plots of magnetisation, magnetic susceptibility and heat capacity against reduced temperature are made, with the three calculated critical temperatures overplotted as vertical dashed lines, as well as Onsager's analytical critical temperature of 2.269185K. Reduced temperature is used because it is dimensionless. This function has to read in a defined temperature step to increase the temperature by with each simulation, as well as the starting temperature and number of runs from a paramter file, so it uses a different parameter file and file reading function to **Function to run a single Metropolis algorithm simulation with end system visualisation)** and **Function to run a single Wolff algorithm simulation with end system visualisation)**. It also uses python's Pool() to spawn multiple processes for running multiple simulations at once. Also plots the Onsager critical exponents out after computing the required logarithms to make log-log fitted plots. Critical exponents are extracted as gradients of these plots, and since the function also stores Onsager's analytical values of the critical exponents the errors of the calculated values from this function are also printed out for easy access. Additionally, since alpha is 0, rather than attmepting to compute it via a gradient fit, which would leave residuals behind, it is instead verified by plotting the natural logarithm of reduced temperature against heat capacity. If the gradient is the negative of the Onsager critical amplitude, then alpha is 0 and is behaving as expected. The gradient should be the negative of the Onsager critical amplitude because as the difference from the critical temperature increases, the heat capacity reduces and since the natural logarithm is being taken, heat capacity

should reduce at a rate of -0.4945 since it is being compared to the natural logarithm of temperature.

$$C = -Aln(\frac{T - T_c}{T_c}) \tag{22}$$

Furthermore this function only calculates exponents where they are valid, and never plots directly over $T_{crit}$, since the reduced temperature would become 0, causing the logarithm to diverge to infinity. Alpha is valid both above and below $T_{crit}$, Beta is only valid below $T_{crit}$, and Gamma is valid both above and below $T_{crit}$.

14. **Parallelised critical temperature and exponents investigation using the Wolff algorithm.** Exactly the same as **Parallelised critical temperature and exponents investigation using the metropolis algorithm.** but uses the wolff algorithm instead of the metropolis algorithm. Since the Wolff algorithm requires far fewer steps than the metropolis algorithm, this function relies on a different parameter file, but the same file reading function can still be used. Uses the second of the paraellelisation .py files: parallelised temperature investigation functions WMC.py.
15. **Parallelised lattice side length change investigation function using the metropolis algorithm** Uses the third parallelisation .py file (parallelised lattice side length change investigation functions MMC.py). Calculates $C_0$ by running multiple simulations at different lattice sizes and recording their equilibrated heat capacities and computing the gradient between the logarithm of lattice size D and heat capacity.
16. **Parallelised lattice side length change investigation function using the wolff algorithm** Exactly the same as **Parallelised lattice side length change investigation function using the metropolis algorithm** but uses the Wolff algorithm, and the fourth parallelisation file, (parallelised lattice side length change investigation functions WMC.py).

## 3.2 Parallelisation files

Each one of the four parallelisation files contains an importable worker function and all of the secondary functions it required for calculating quantities of interest. There are four of these because there are parallelised investigations for both the metropolis and Wolff algorithms that involve changing either the lattice size or the temperature.

# 4 Figures

These figures were generated by running the Jupyter notebook with all of the parallelisation and parameter files in the same directory.

## 4.1 Critical Temperature Investigation using the Metropolis Algorithm for a $32 \times 32$ lattice

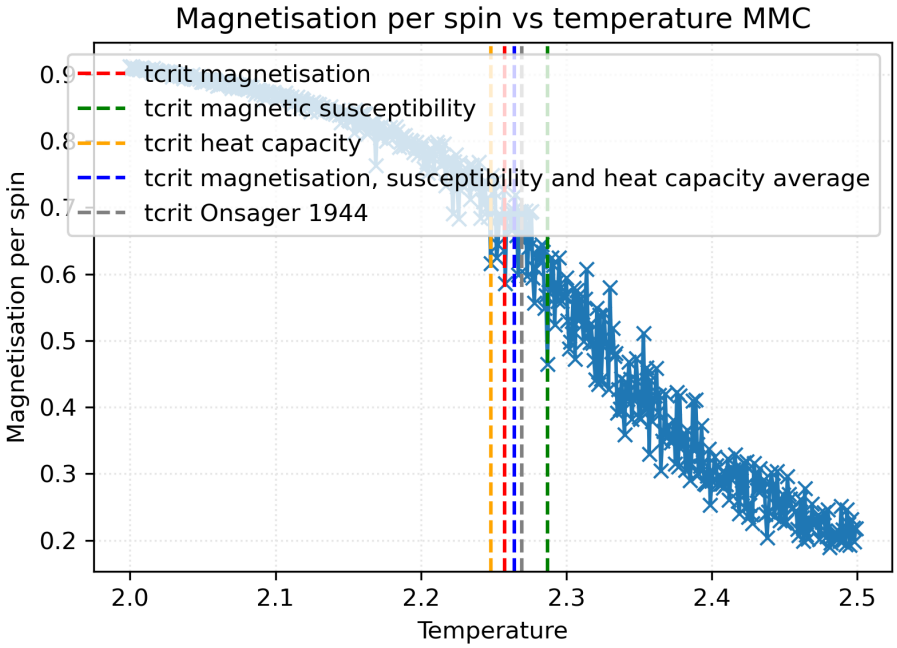


**Figure 4:** Temperature vs magnetisation (Critical Temperature Investigation (Metropolis, $32 \times 32$ Lattice))

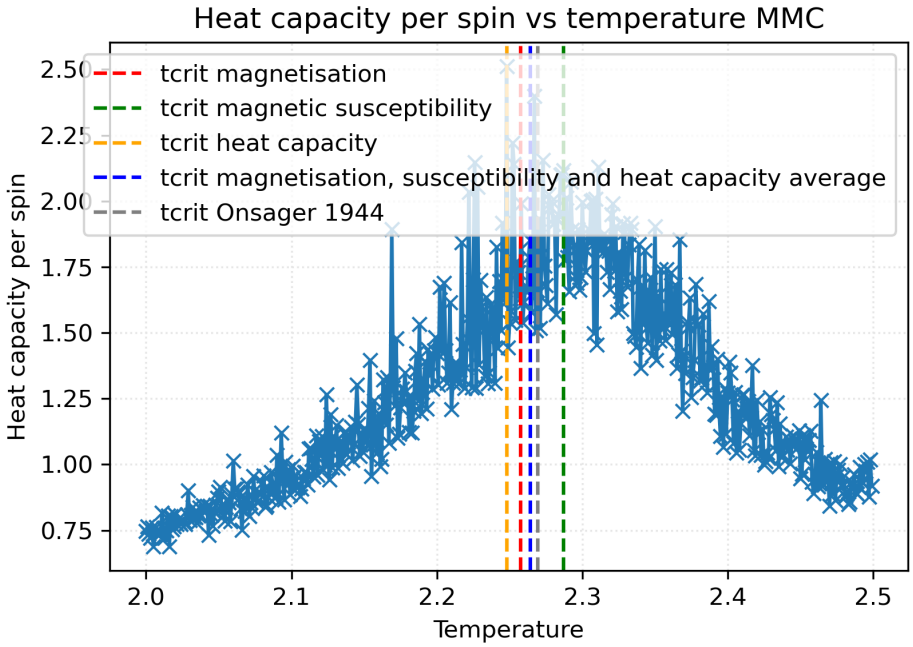


**Figure 5:** Temperature vs heat capacity (Critical Temperature Investigation (Metropolis, $32 \times 32$ Lattice))

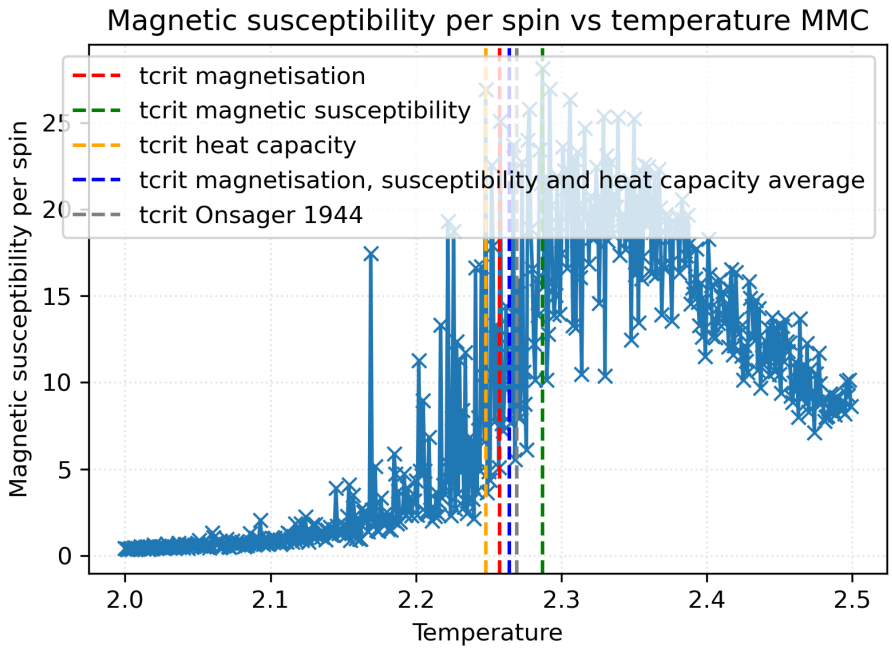


**Figure 6:** Temperature vs magnetic susceptibility (Critical Temperature Investigation (Metropolis, $32 \times 32$ Lattice))

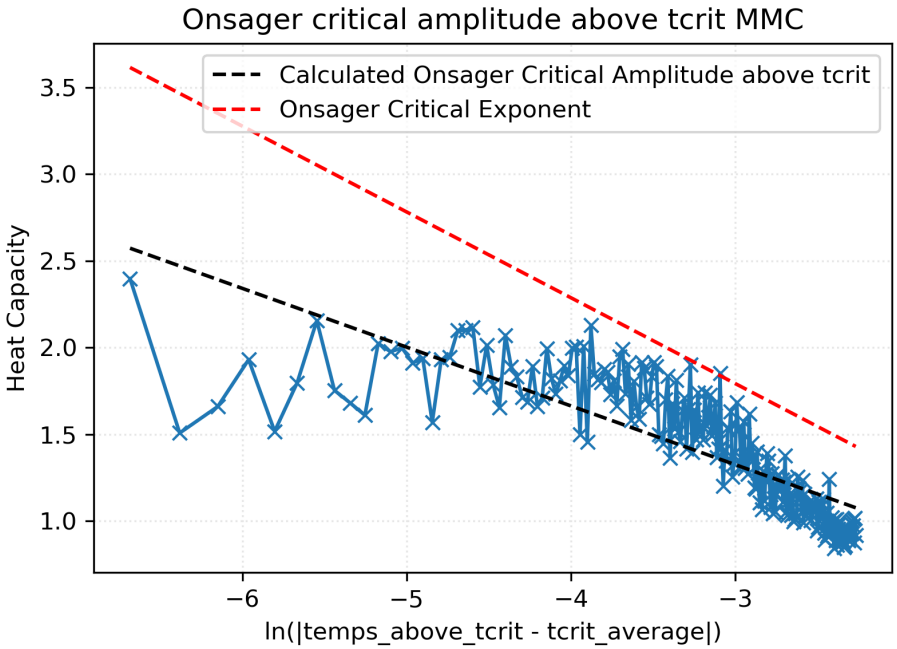


**Figure 7:** Onsager critical exponent above tcrit (Critical Temperature Investigation (Metropolis, $32 \times 32$ Lattice))

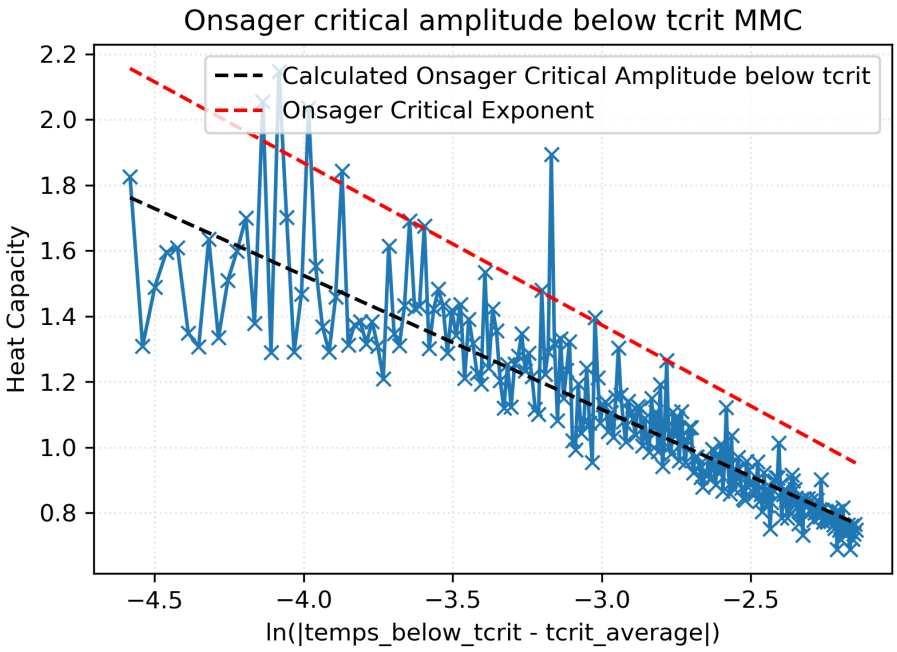


**Figure 8:** Onsager critical exponent below tcrit (Critical Temperature Investigation (Metropolis, $32 \times 32$ Lattice))

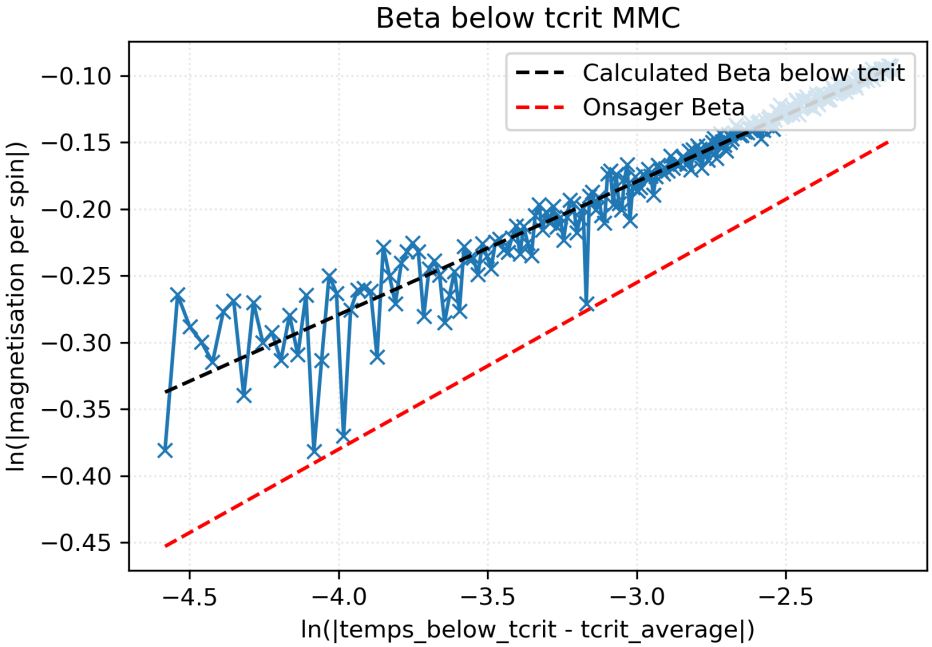


**Figure 9:** Beta below tcrit (Critical Temperature Investigation (Metropolis, $32 \times 32$ Lattice))

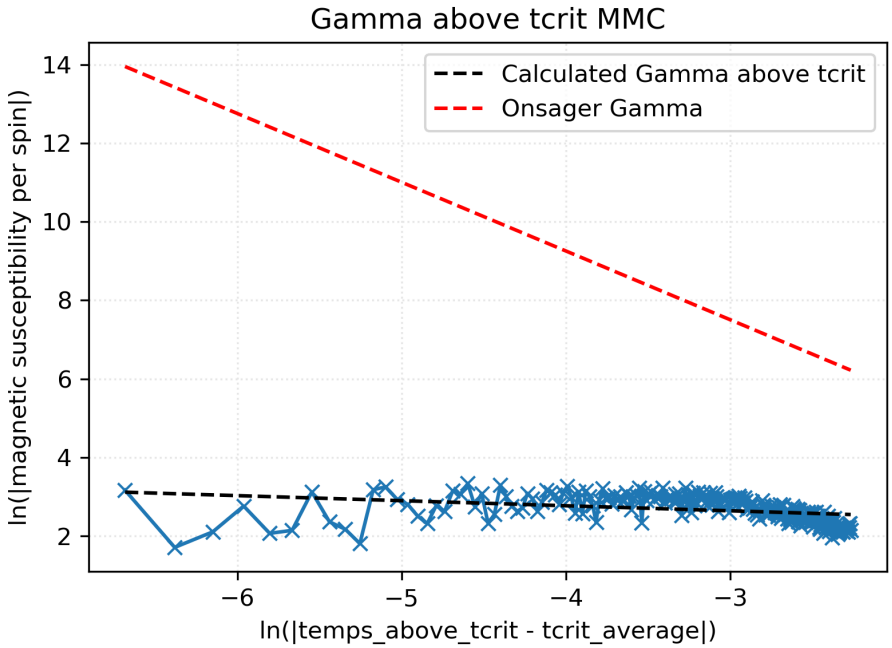


**Figure 10:** Gamma above tcrit (Critical Temperature Investigation (Metropolis, $32 \times 32$ Lattice))

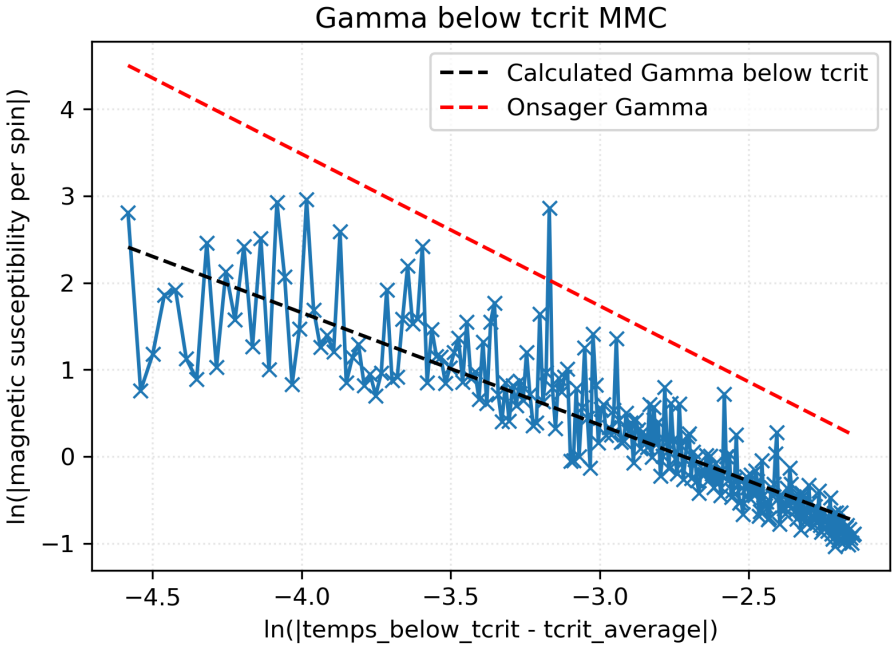


**Figure 11:** Gamma below tcrit (Critical Temperature Investigation (Metropolis, $32 \times 32$ Lattice))

## 4.2 Heat Capacity vs ln(D) using the Metropolis algorithm

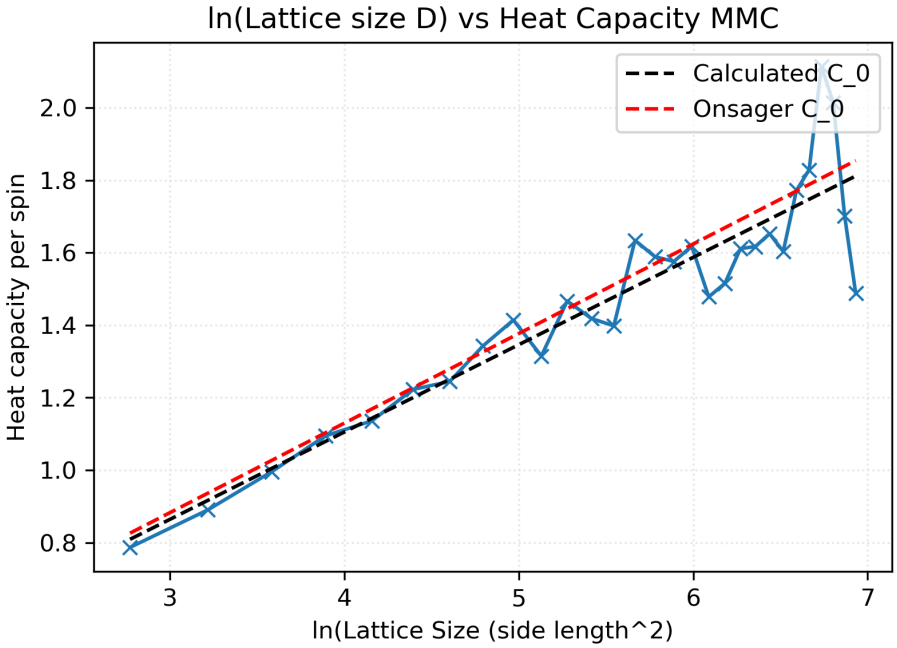


**Figure 12:** ln(Lattice size D) vs heat capacity per spin (Lattice Size vs $C_0$ (Metropolis))

## 4.3 Critical Temperature Investigation using the Wolff Algorithm for a $64 \times 64$ Lattice)

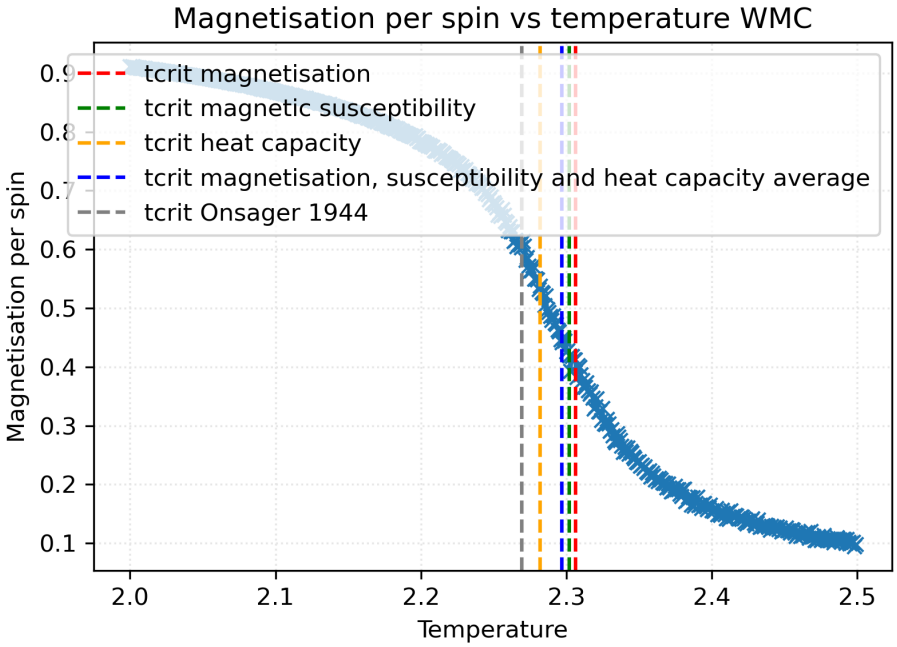


**Figure 13:** Temperature vs magnetisation (Critical Temperature Investigation (Wolff, $64 \times 64$ Lattice))

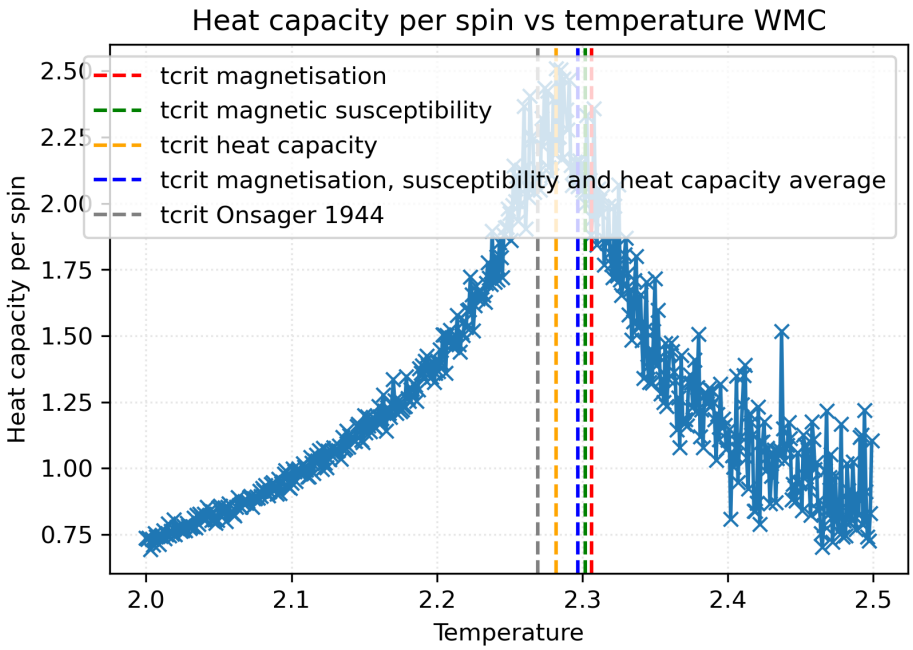


**Figure 14:** Temperature vs heat capacity (Critical Temperature Investigation (Wolff, $64 \times 64$ Lattice))

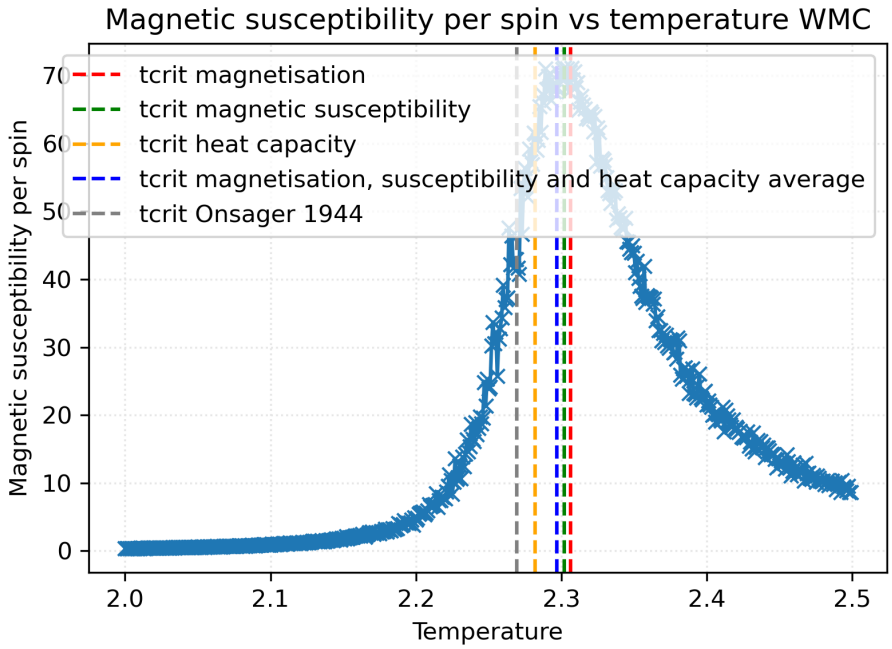


**Figure 15:** Temperature vs magnetic susceptibility (Critical Temperature Investigation (Wolff, $64 \times 64$ Lattice))

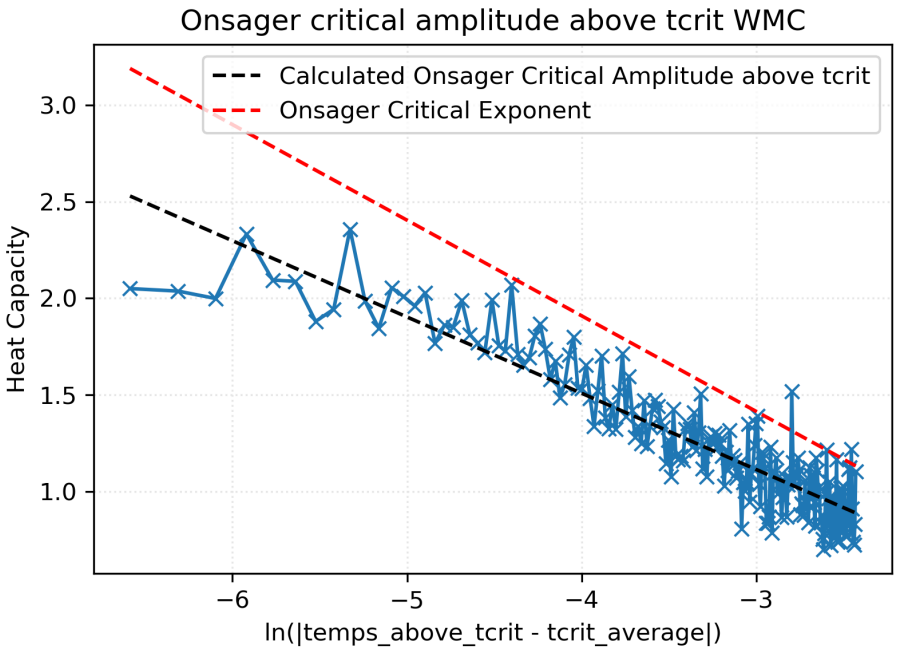


**Figure 16:** Onsager critical exponent above tcrit (Critical Temperature Investigation (Wolff, 64×64 Lattice))

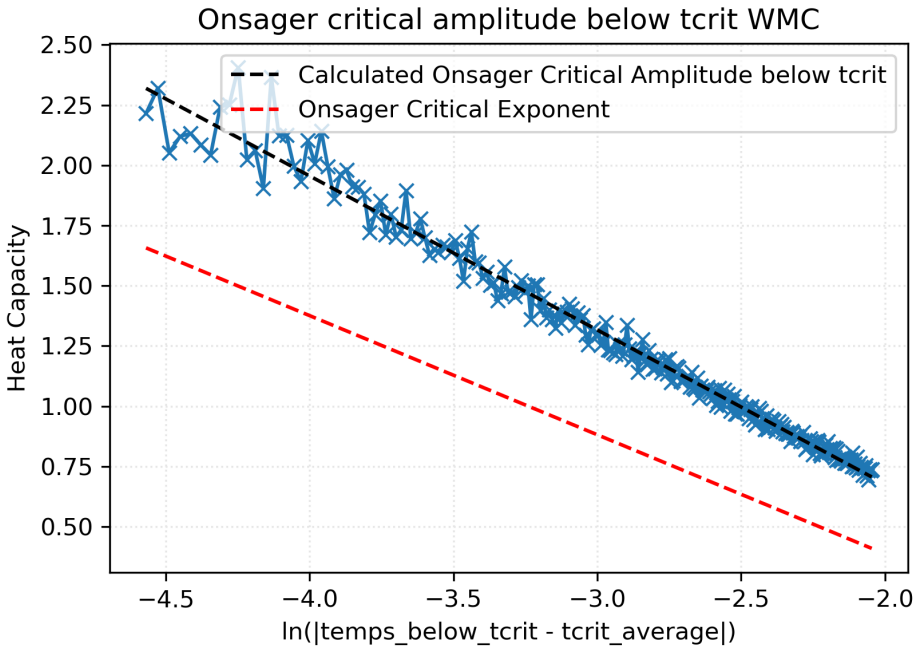


**Figure 17:** Onsager critical exponent below tcrit (Critical Temperature Investigation (Wolff, 64×64 Lattice))

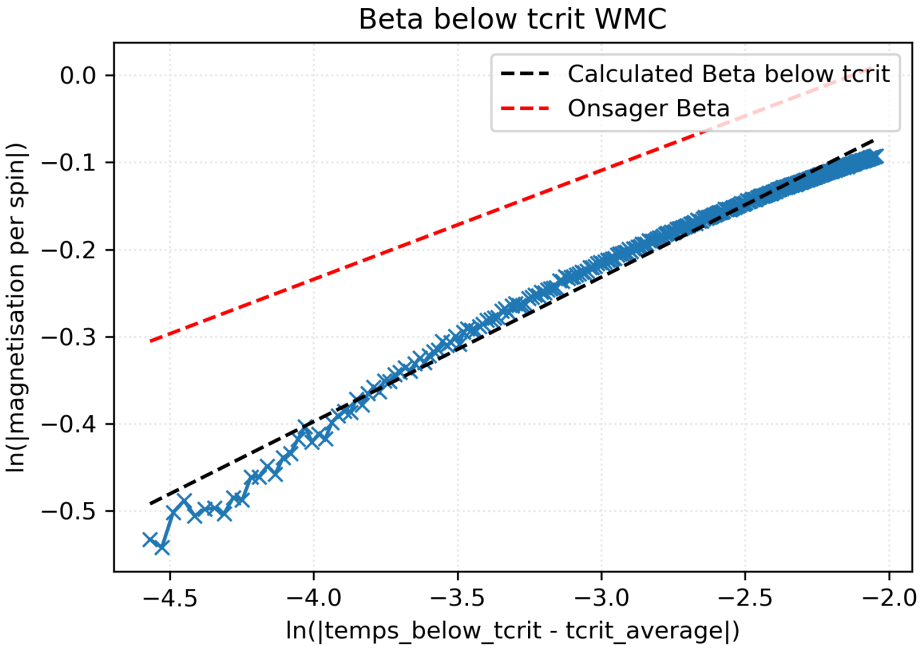


**Figure 18:** Beta below tcrit (Critical Temperature Investigation (Wolff, $64 \times 64$ Lattice))

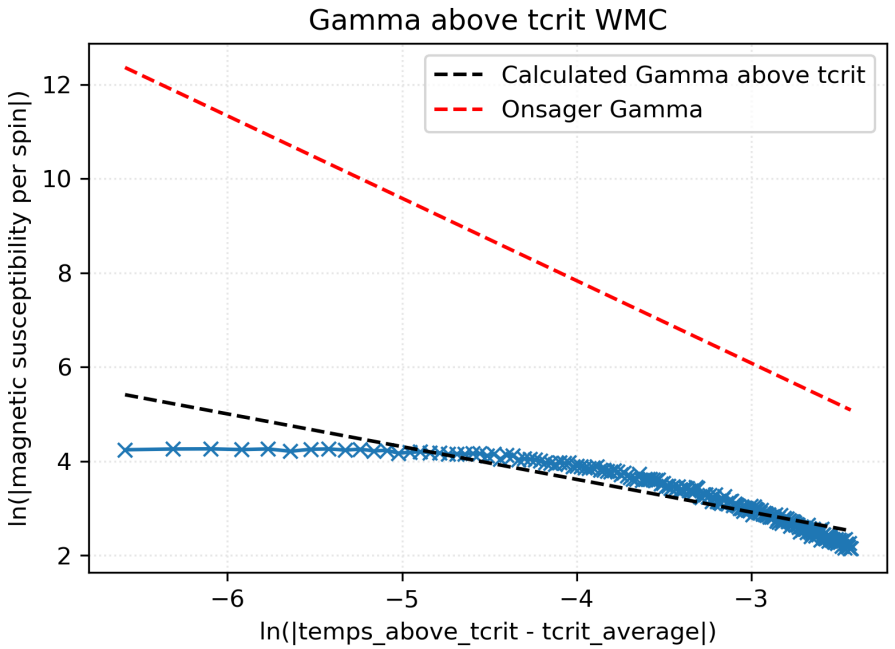


**Figure 19:** Gamma above tcrit (Critical Temperature Investigation (Wolff, $64 \times 64$ Lattice))

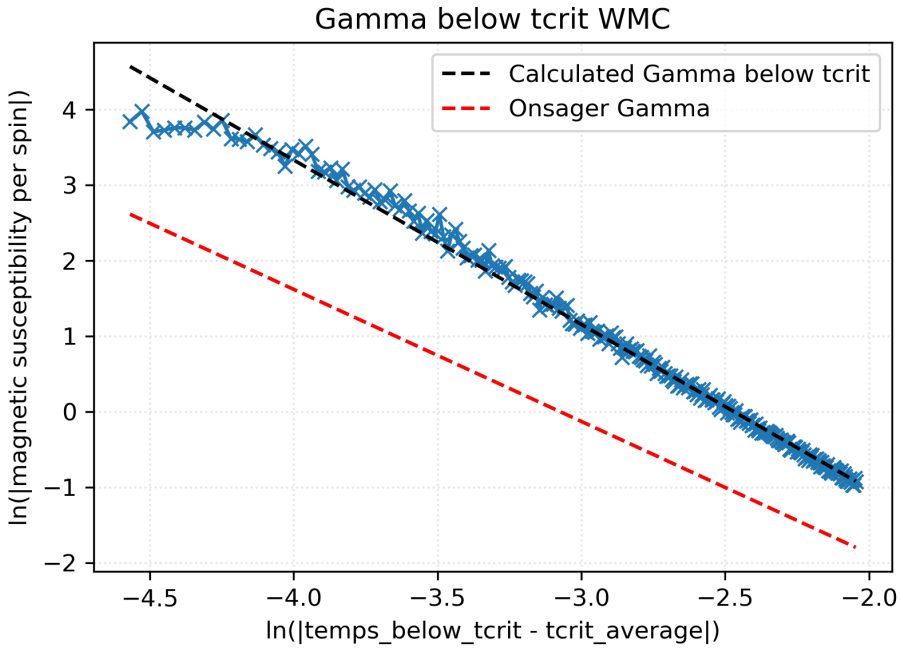


**Figure 20:** Gamma below tcrit (Critical Temperature Investigation (Wolff, $64 \times 64$ Lattice))

## 4.4 Heat Capacity vs ln(D) using the Wolff algorithm

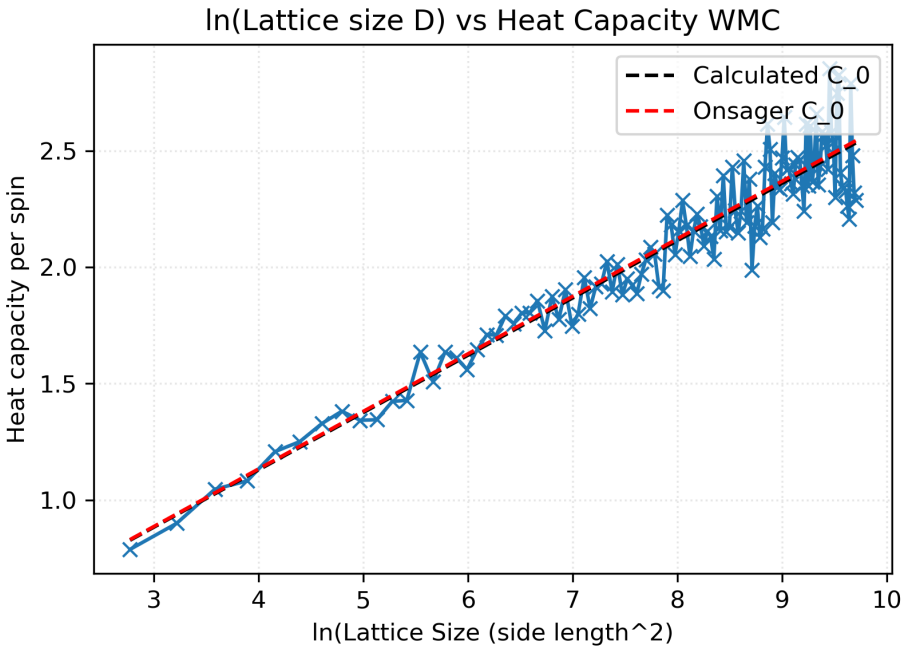


**Figure 21:** ln(Lattice size D) vs heat capacity per spin (Lattice Size vs $C_0$ (Wolff))

# 5 Metropolis Algorithm: Increasing Lattice Size at $T_{crit}$

## 5.1 $16 \times 16$ Lattice

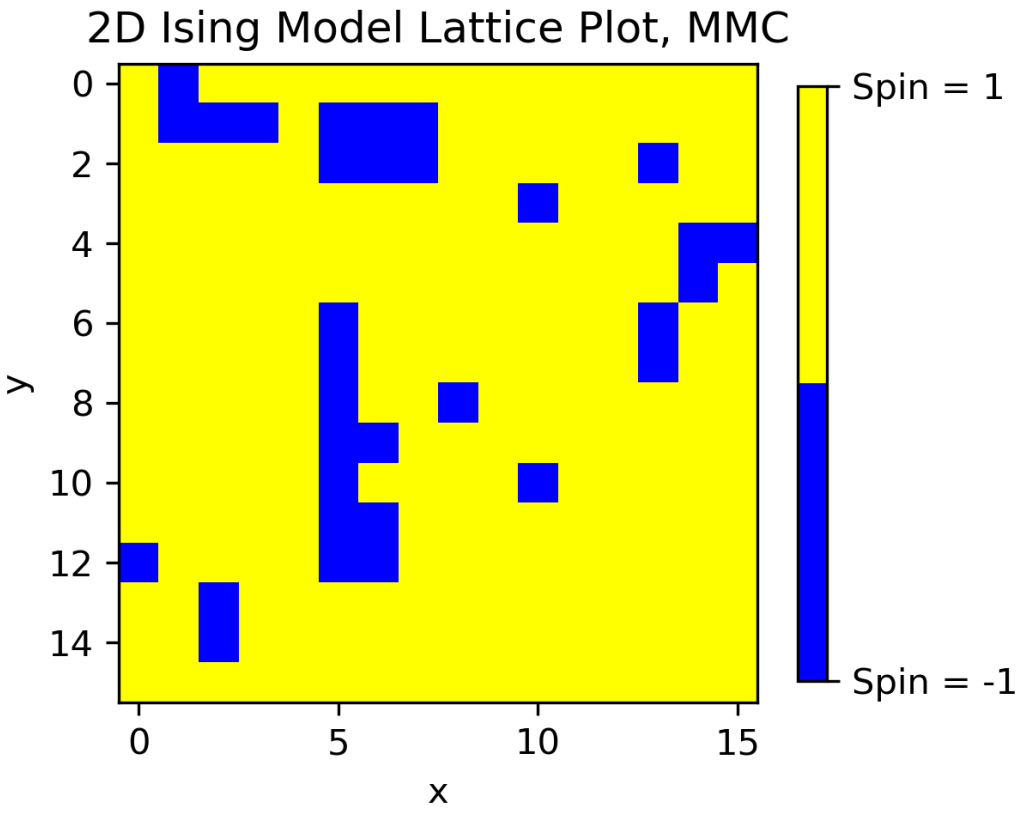


**Figure 22:** End lattice configuration ($16 \times 16$ Lattice)

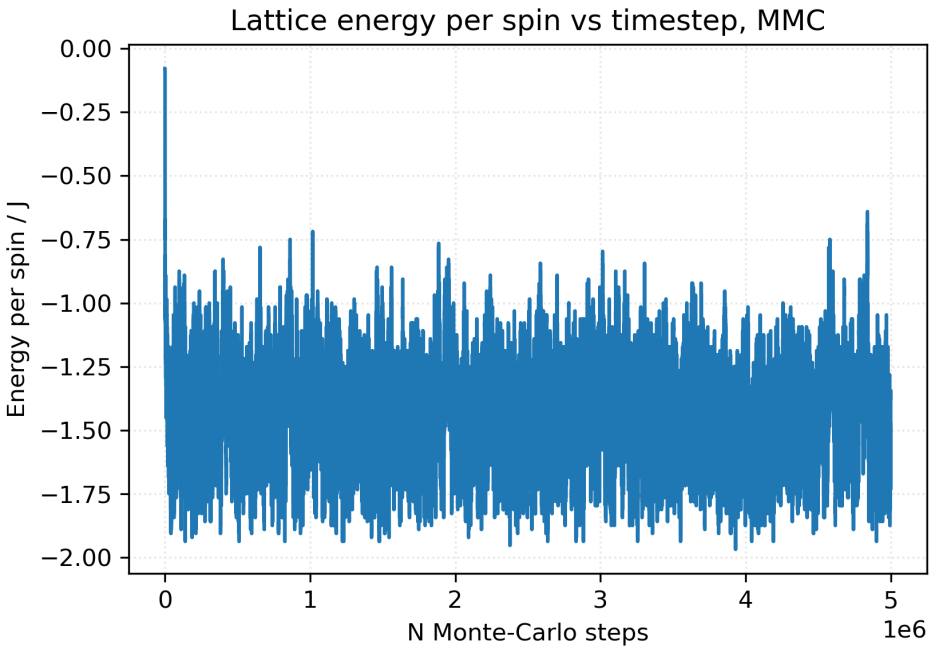


**Figure 23:** Lattice energy per spin vs timestep ($16 \times 16$ Lattice)

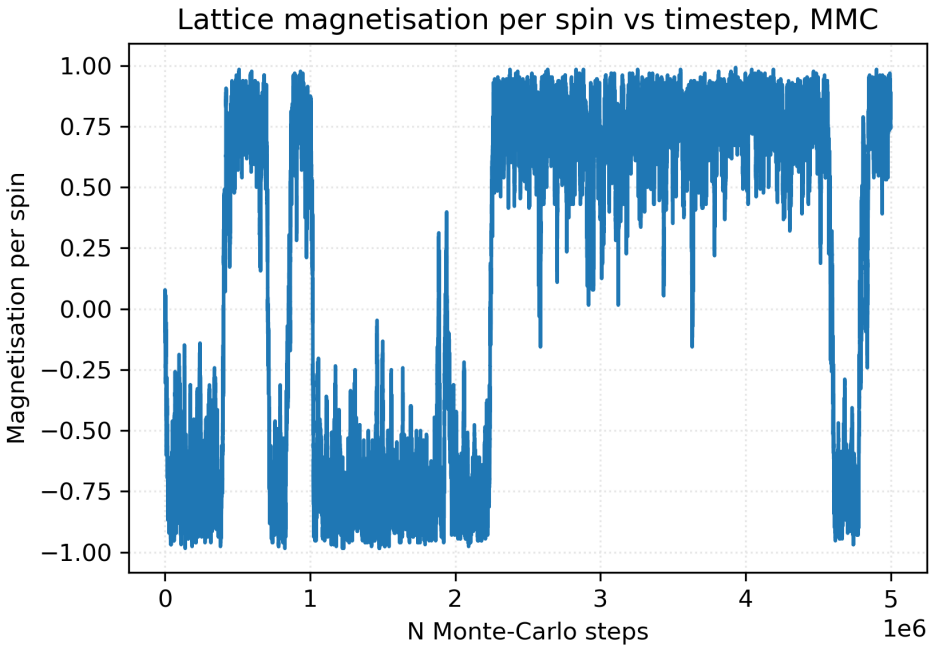


**Figure 24:** Lattice magnetisation per spin vs timestep ($16 \times 16$ Lattice)

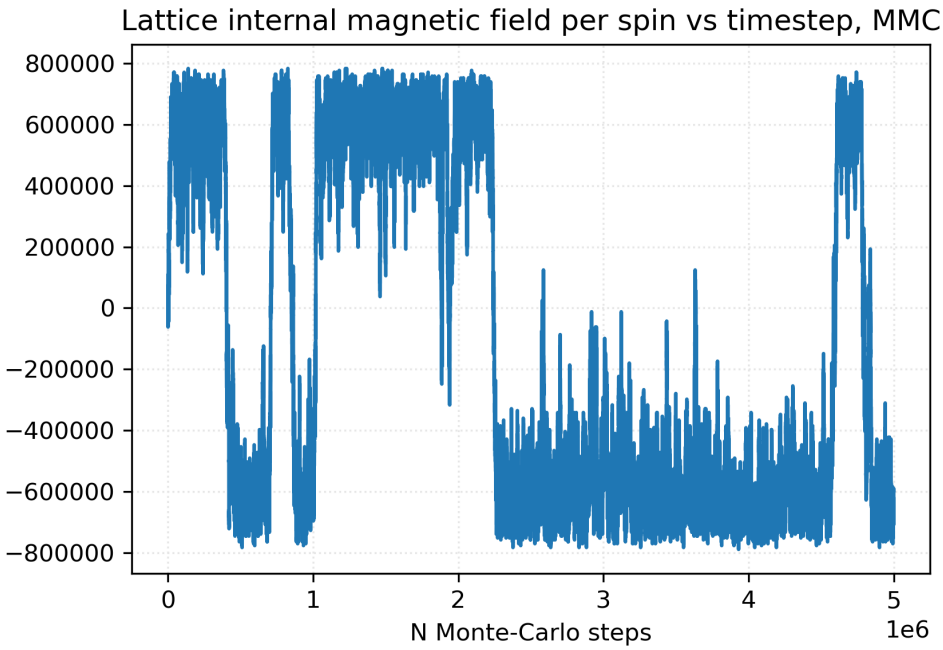


**Figure 25:** Internal magnetic field per spin vs timestep ($16 \times 16$ Lattice)

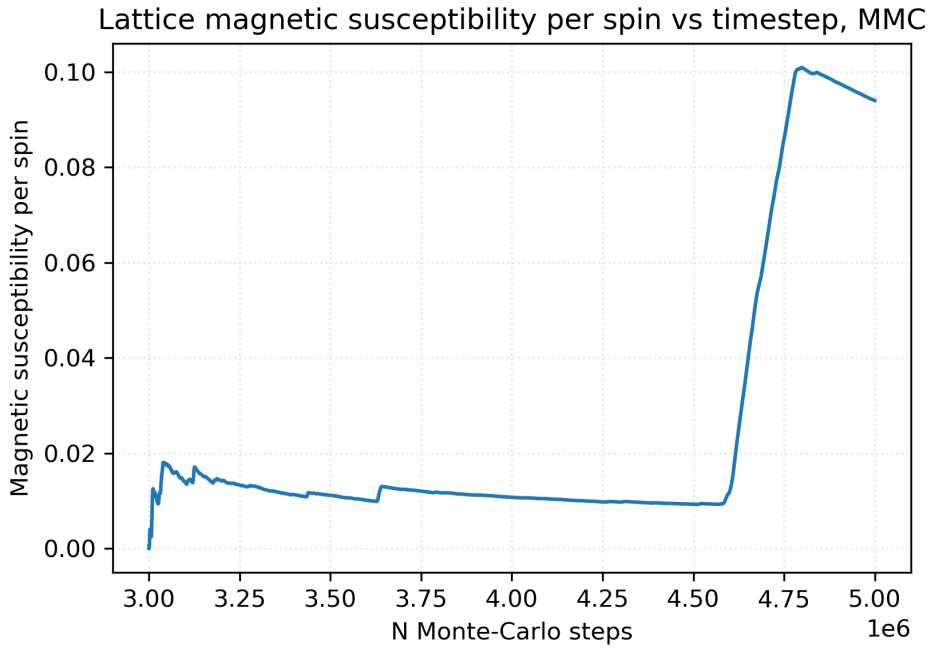


**Figure 26:** Magnetic susceptibility per spin vs timestep ($16 \times 16$ Lattice)

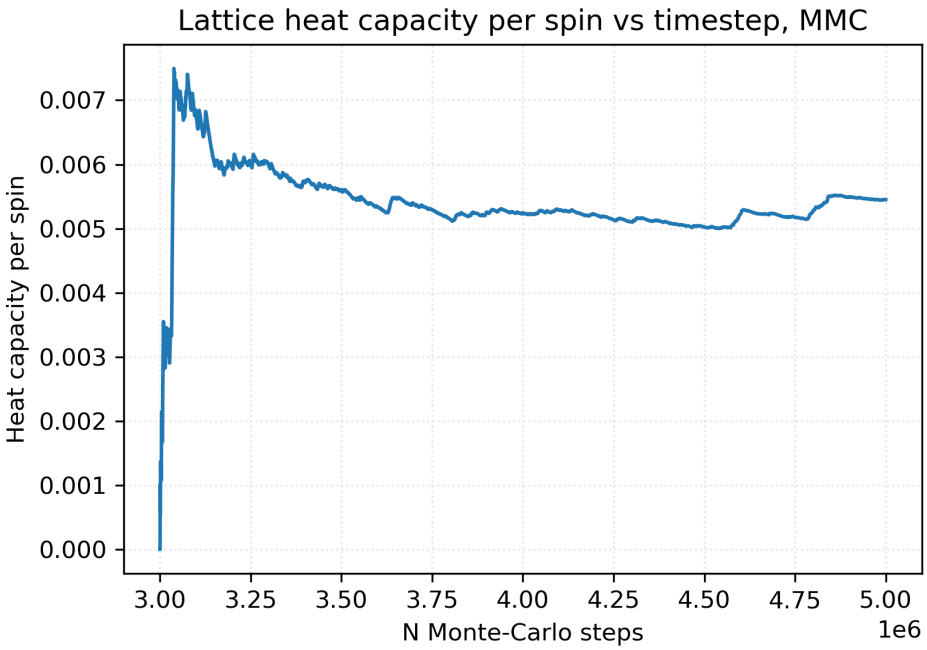


**Figure 27:** Lattice heat capacity per spin vs timestep ($16 \times 16$ Lattice)

## 5.2 $32 \times 32$ Lattice

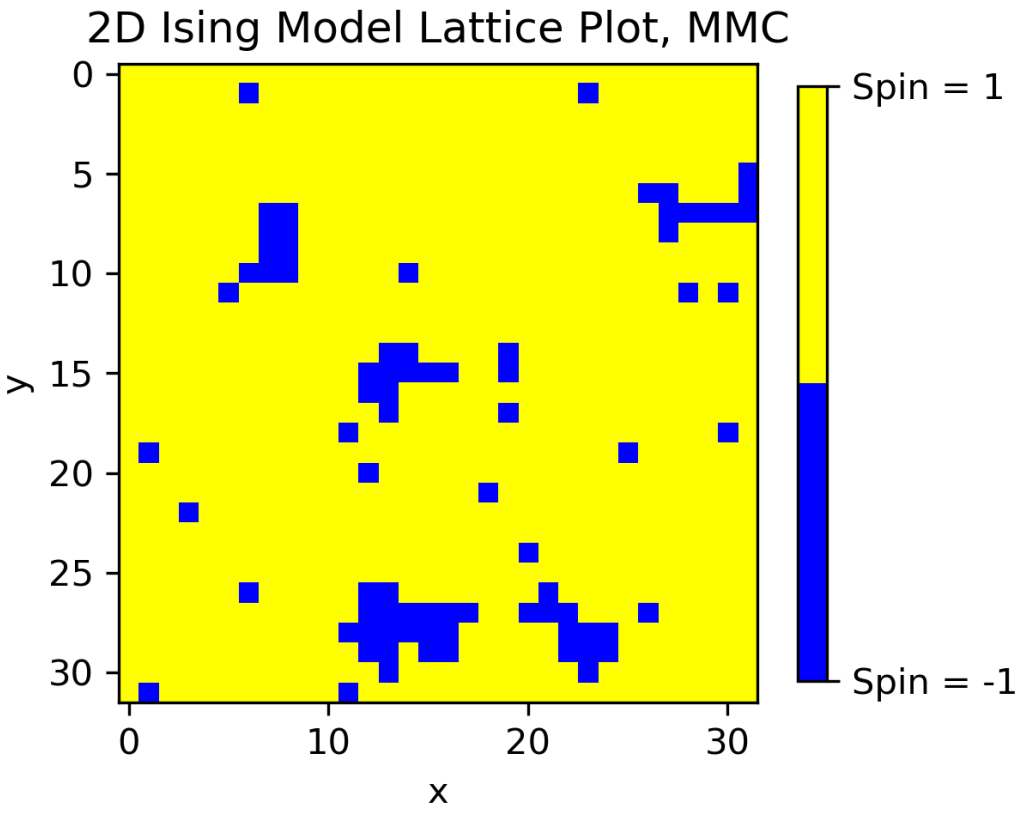


**Figure 28:** End lattice configuration ($32 \times 32$ Lattice)

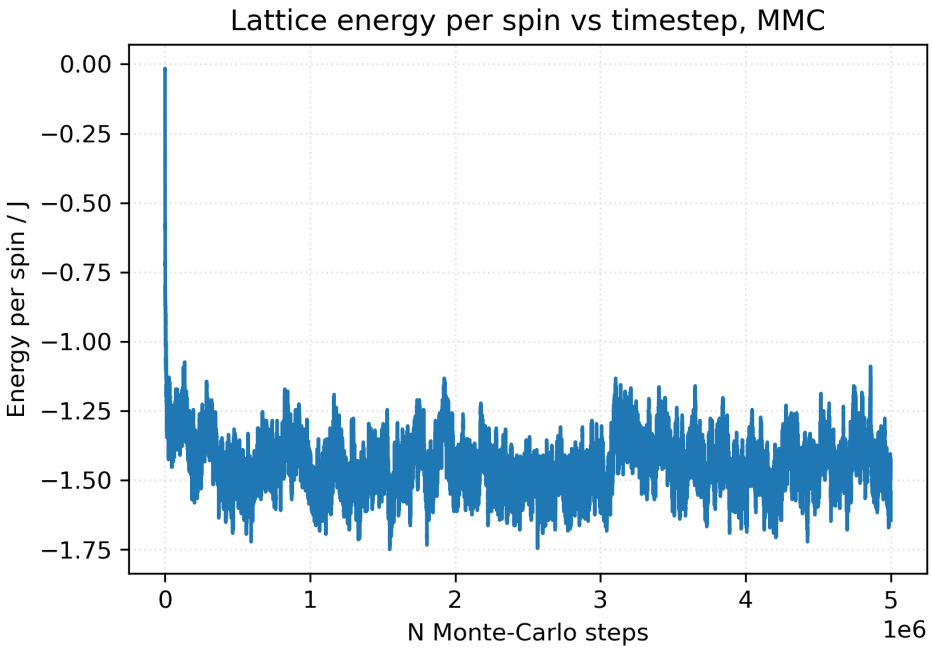


**Figure 29:** Lattice energy per spin vs timestep ($32 \times 32$ Lattice)

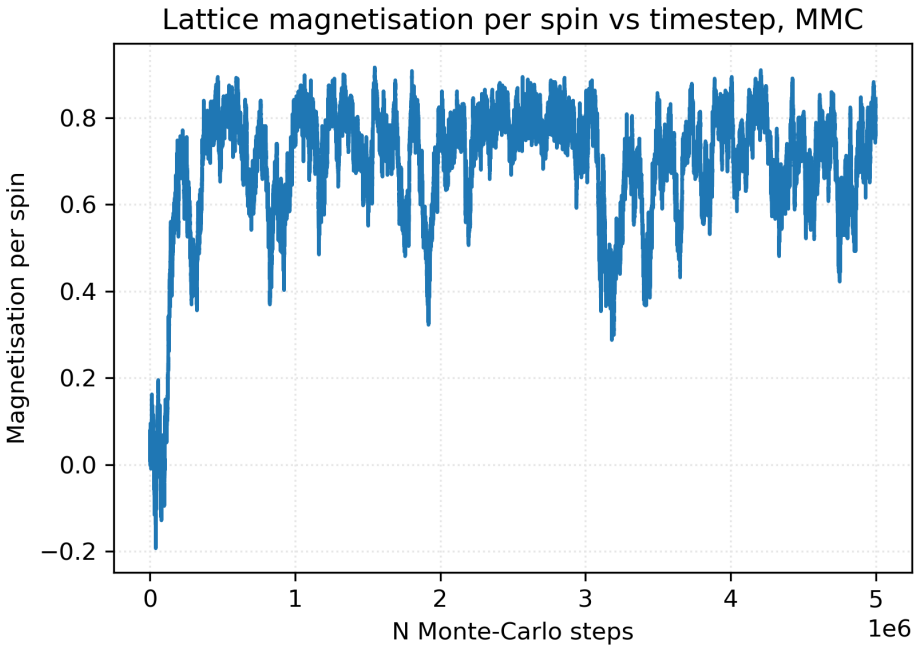


**Figure 30:** Lattice magnetisation per spin vs timestep ($32 \times 32$ Lattice)

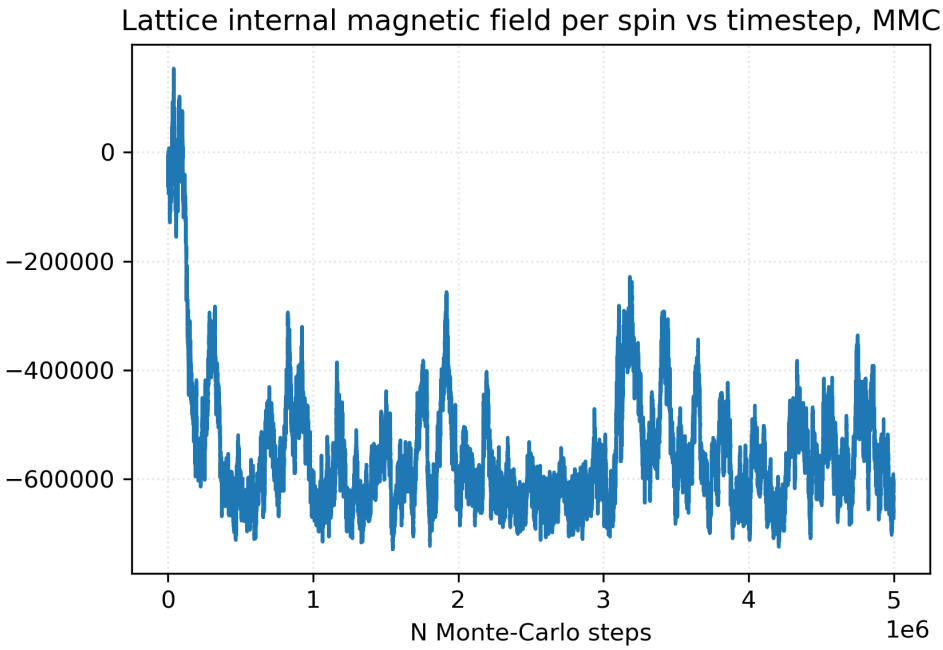


**Figure 31:** Internal magnetic field per spin vs timestep ($32 \times 32$ Lattice)

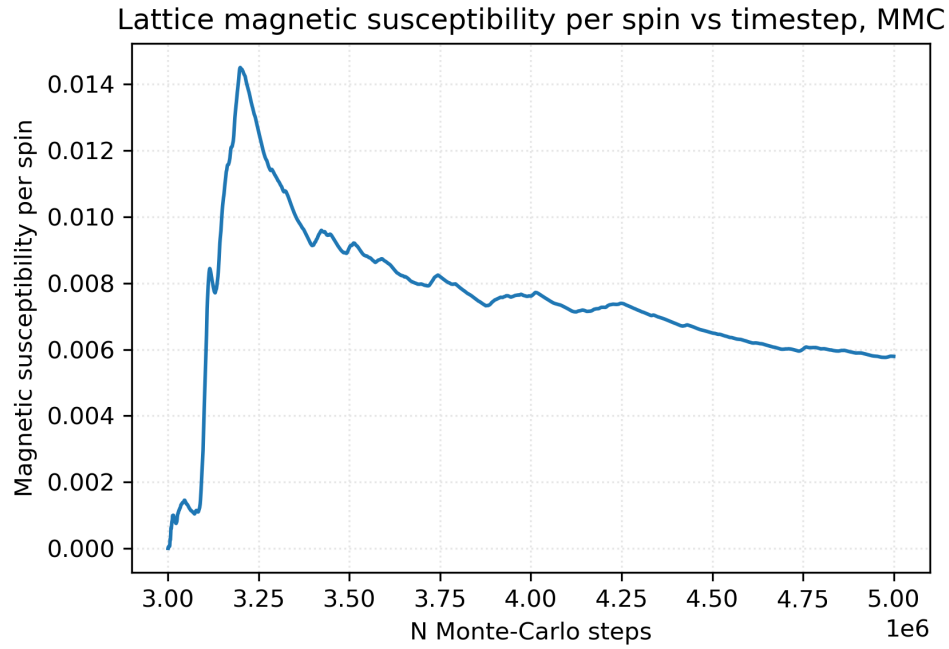


**Figure 32:** Magnetic susceptibility per spin vs timestep ($32 \times 32$ Lattice)

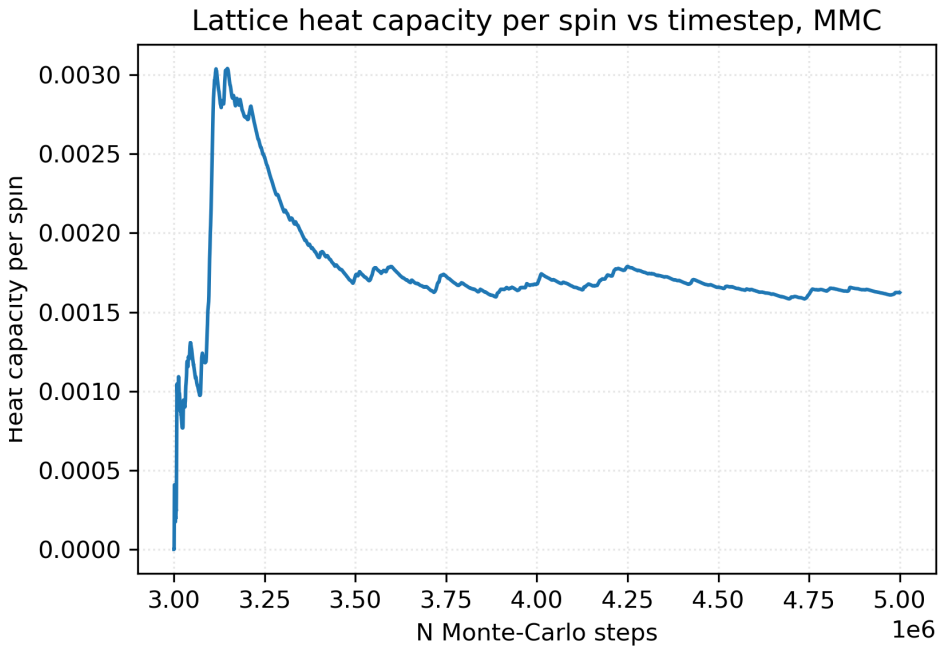


**Figure 33:** Lattice heat capacity per spin vs timestep ($32 \times 32$ Lattice)

## 5.3 $64 \times 64$ Lattice

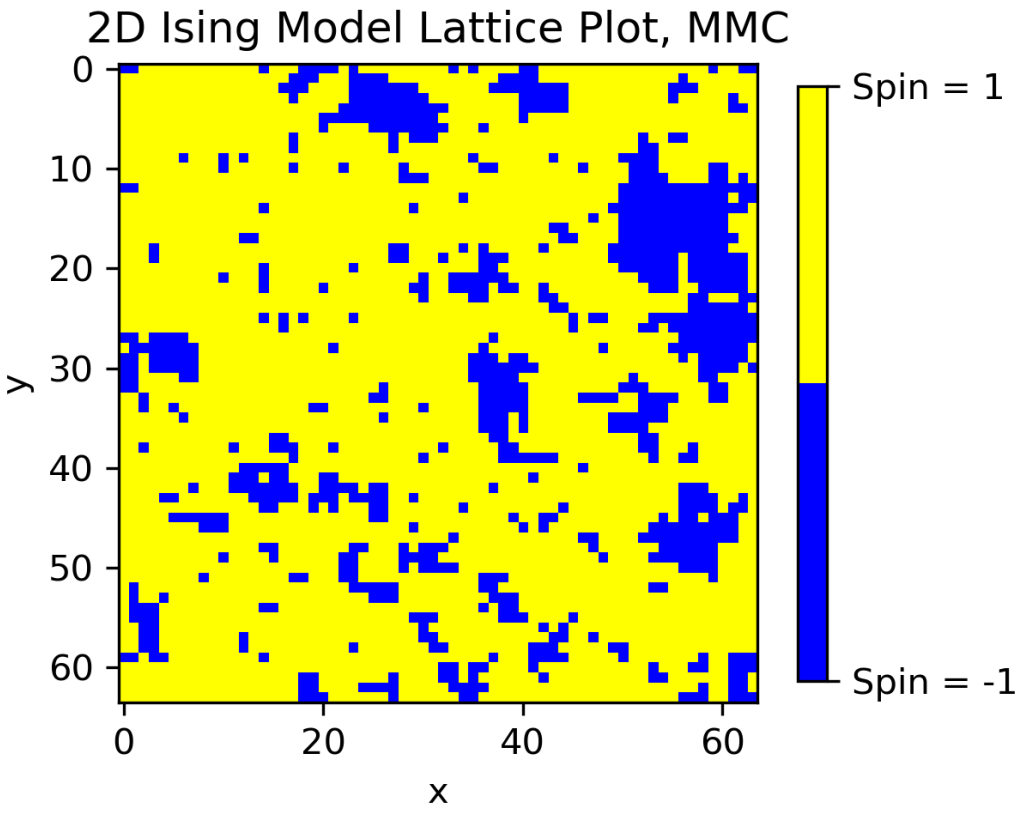


**Figure 34:** End lattice configuration ($64 \times 64$ Lattice)

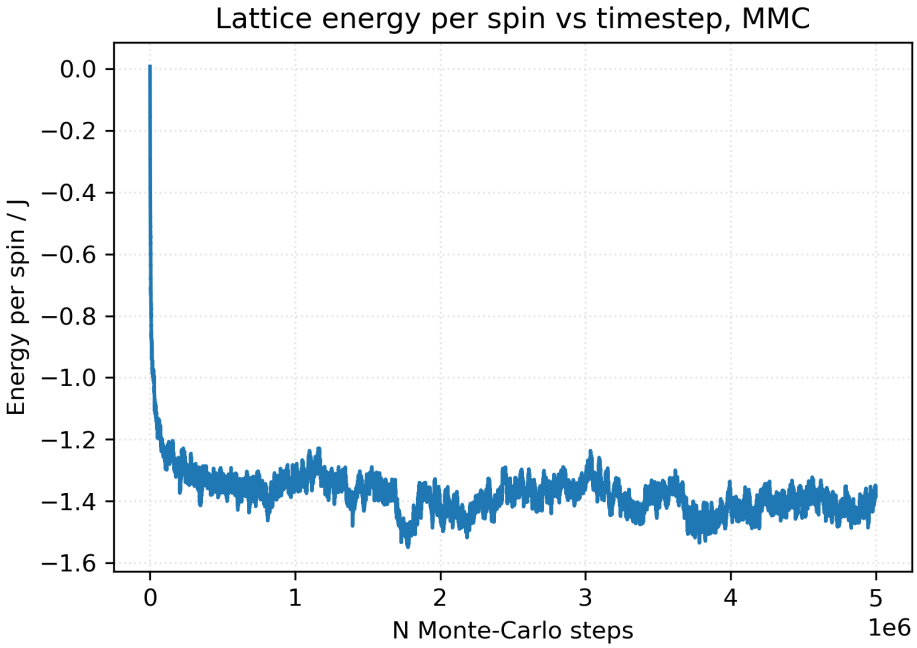


**Figure 35:** Lattice energy per spin vs timestep ($64 \times 64$ Lattice)

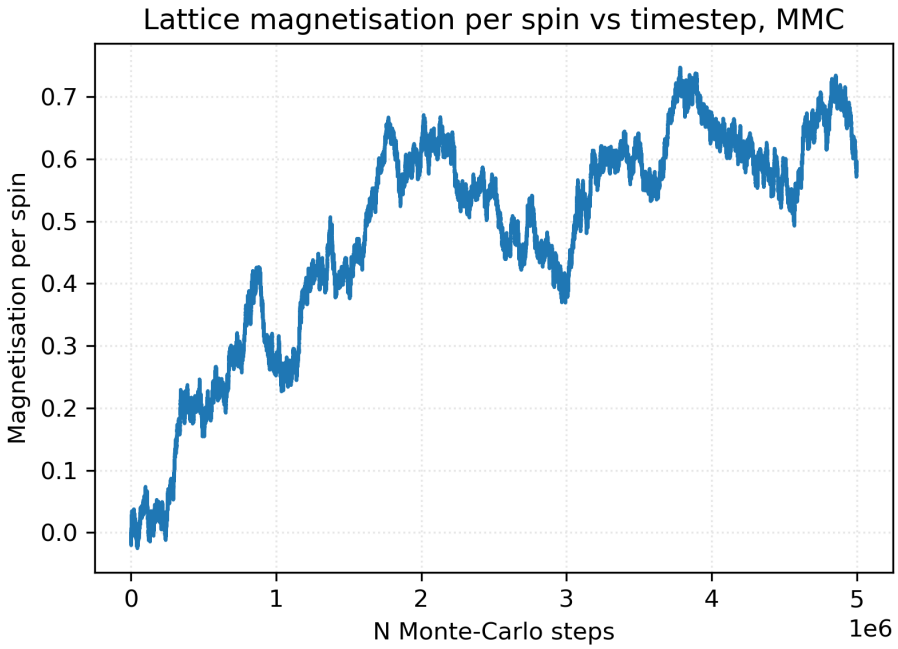


**Figure 36:** Lattice magnetisation per spin vs timestep ($64 \times 64$ Lattice)

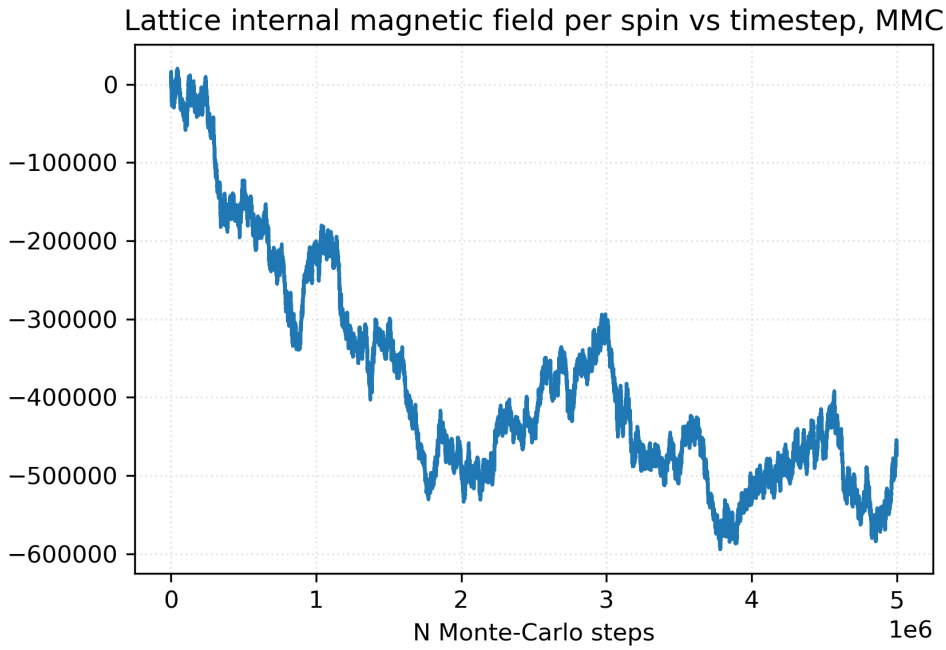


**Figure 37:** Internal magnetic field per spin vs timestep ($64 \times 64$ Lattice)

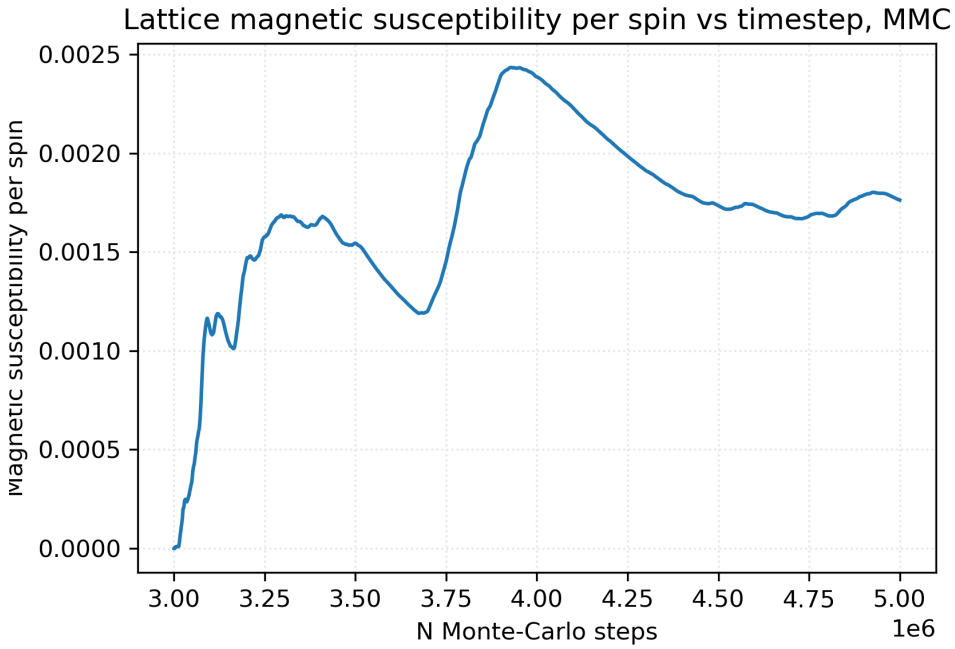


**Figure 38:** Magnetic susceptibility per spin vs timestep ($64 \times 64$ Lattice)

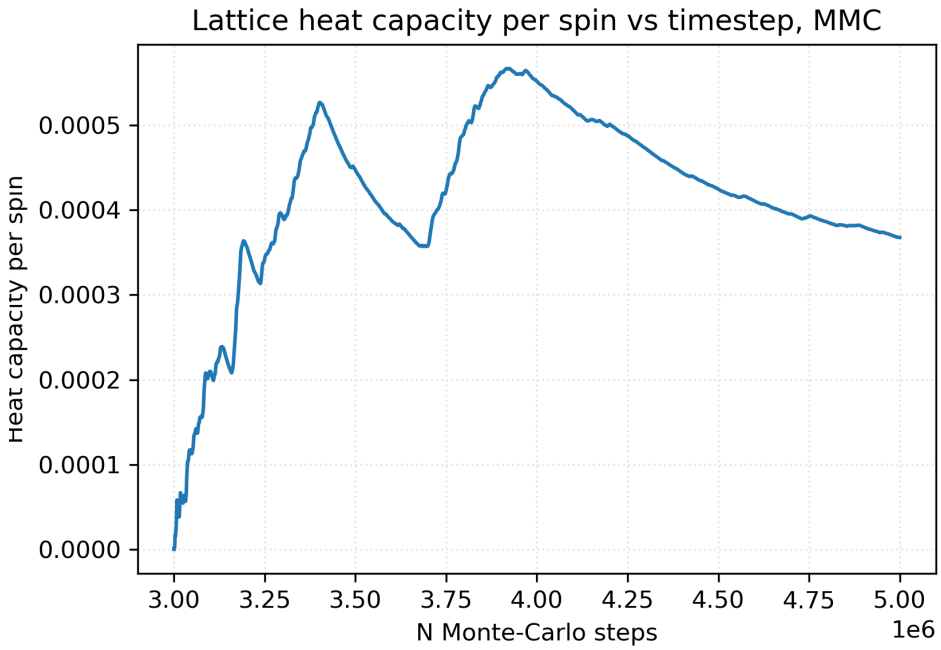


**Figure 39:** Lattice heat capacity per spin vs timestep ($64 \times 64$ Lattice)

## 5.4 $128 \times 128$ Lattice

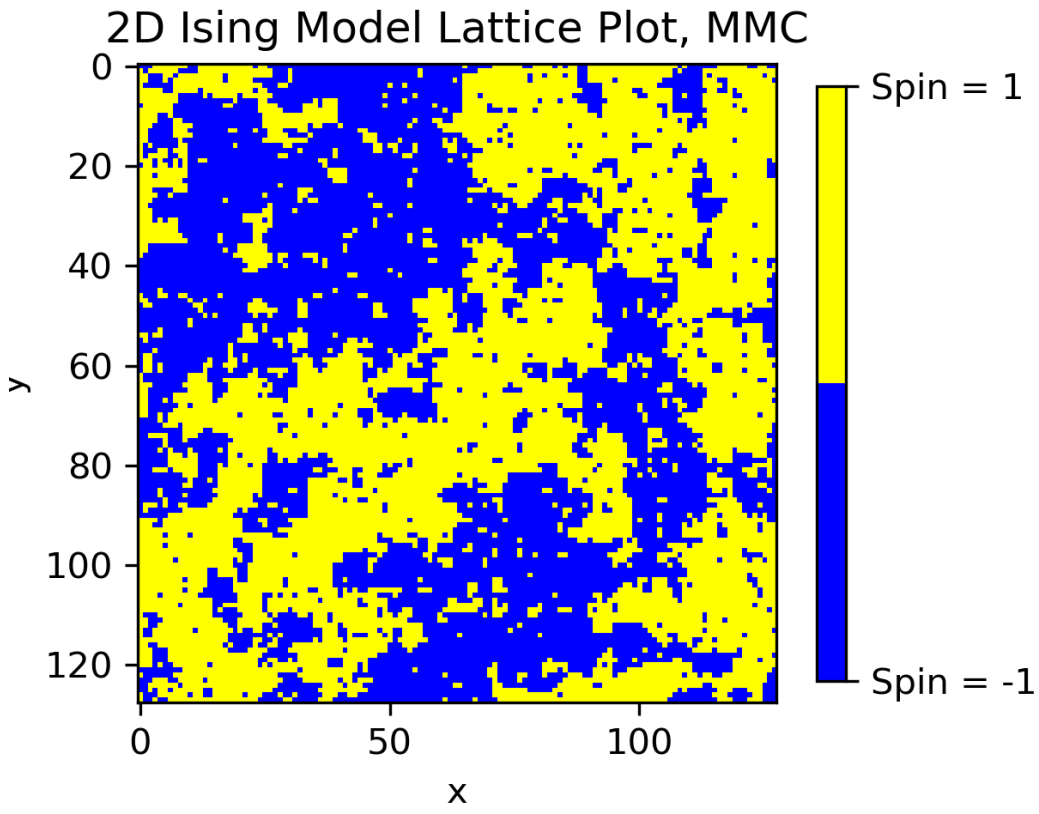


**Figure 40:** End lattice configuration ($128 \times 128$ Lattice)

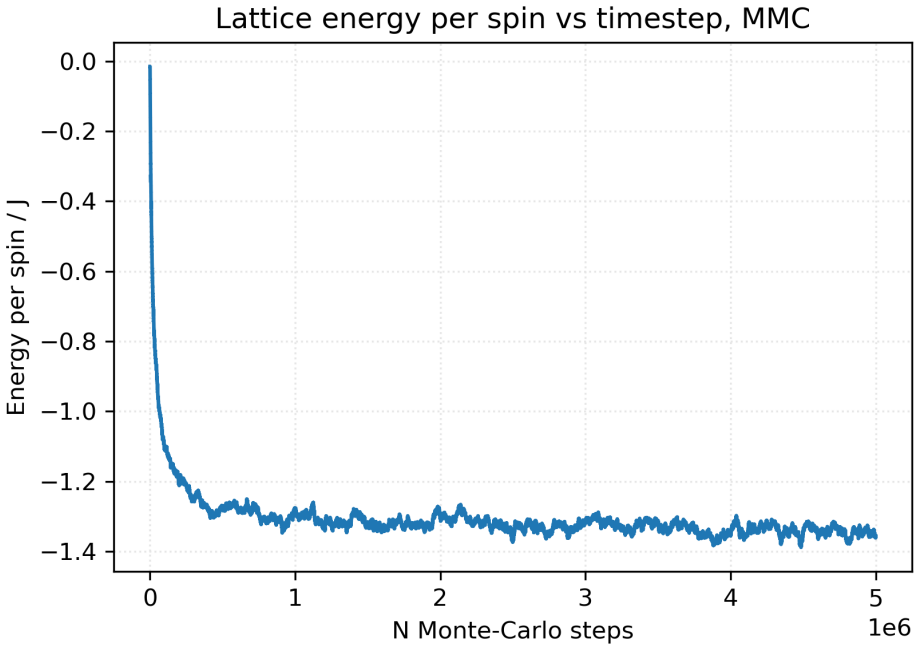


**Figure 41:** Lattice energy per spin vs timestep (128 × 128 Lattice)

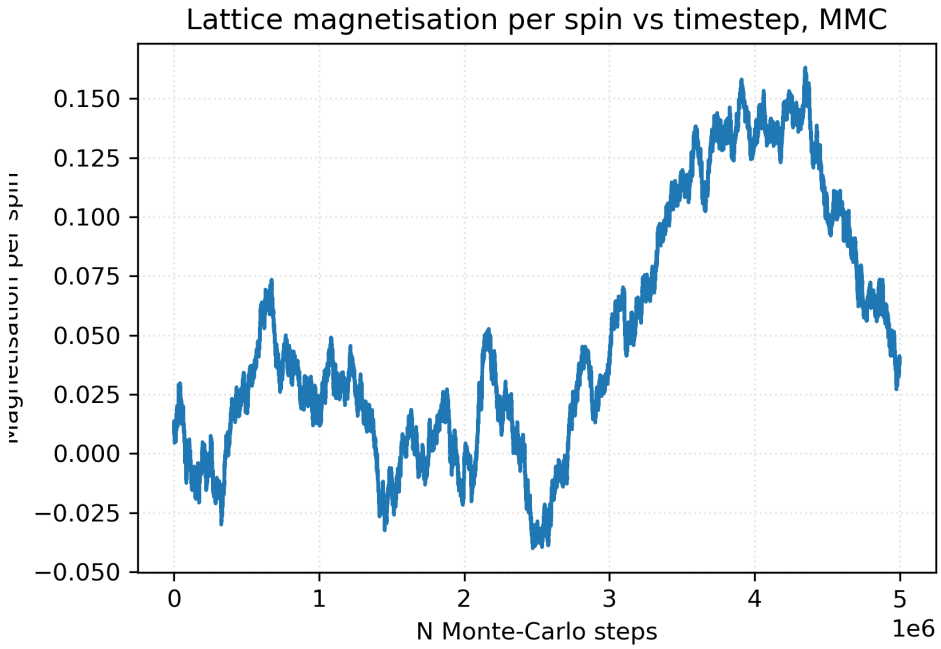


**Figure 42:** Lattice magnetisation per spin vs timestep (128 × 128 Lattice)

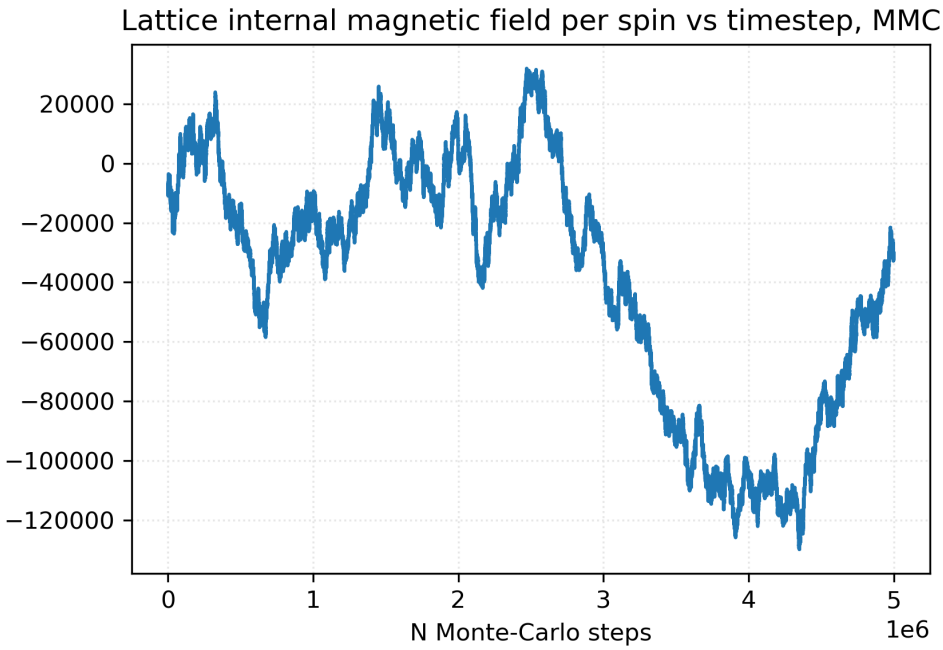


**Figure 43:** Internal magnetic field per spin vs timestep (128 × 128 Lattice)

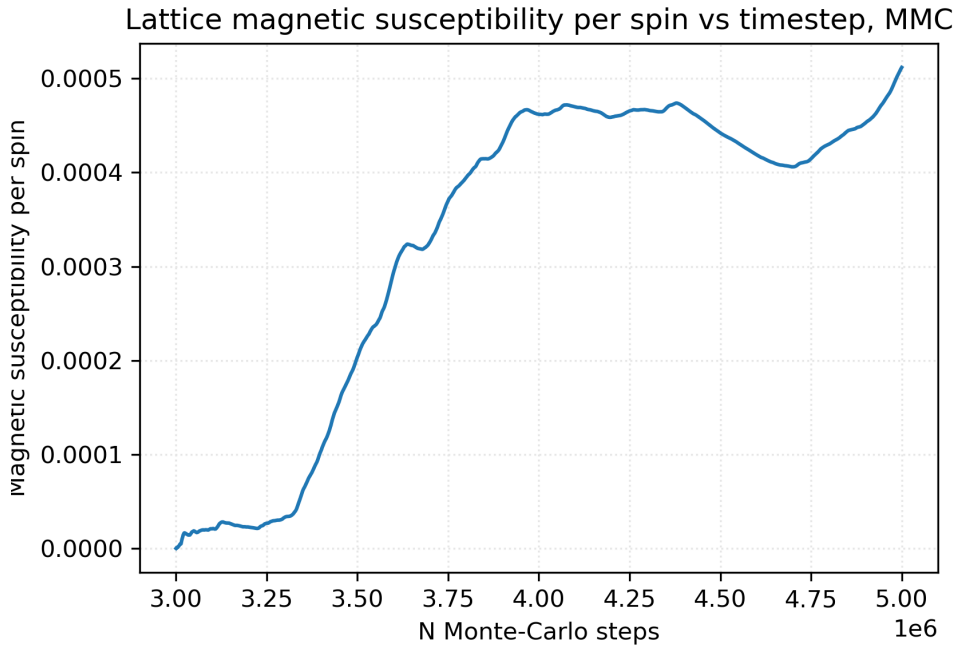


**Figure 44:** Magnetic susceptibility per spin vs timestep (128 × 128 Lattice)

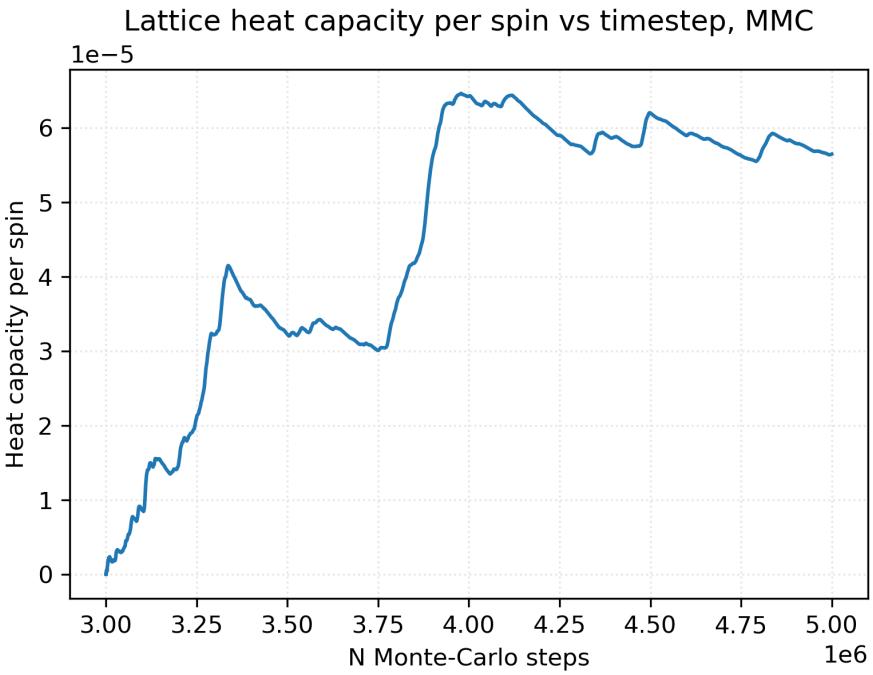


**Figure 45:** Lattice heat capacity per spin vs timestep ($128 \times 128$ Lattice)

# 6 Metropolis Algorithm: Temperatures near absolute zero, below, at and above $T_{crit}$

## 6.1 $T = 0.001$ K

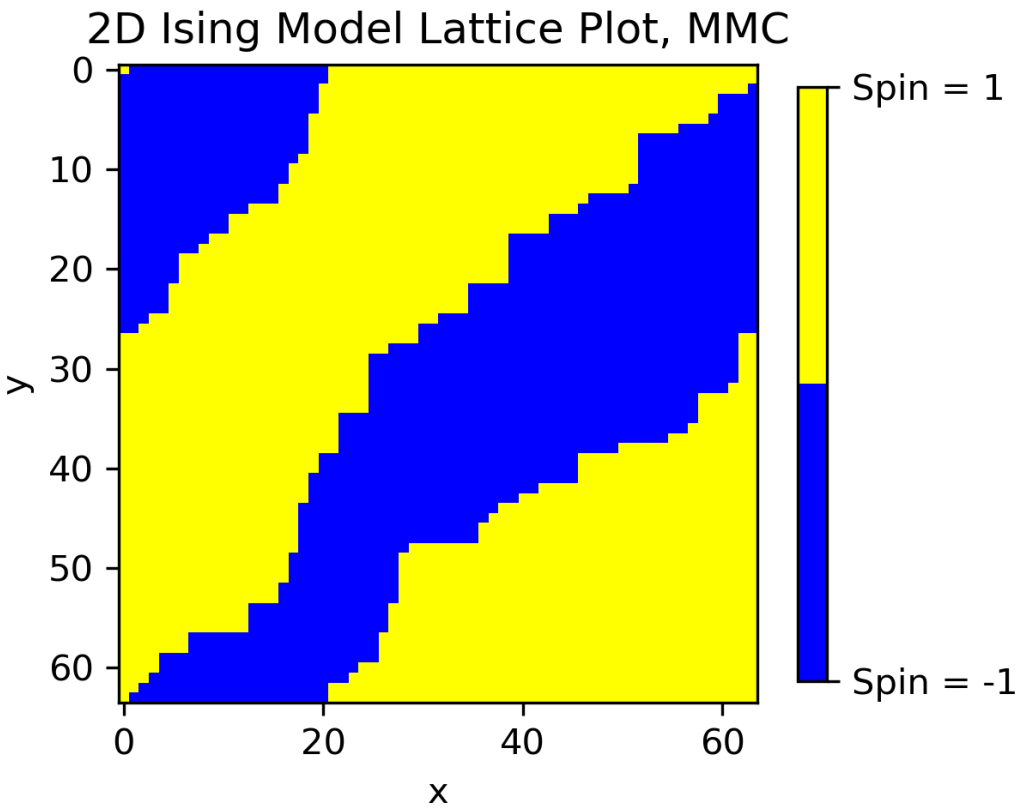


**Figure 46:** End lattice configuration ($T = 0.001$ K)

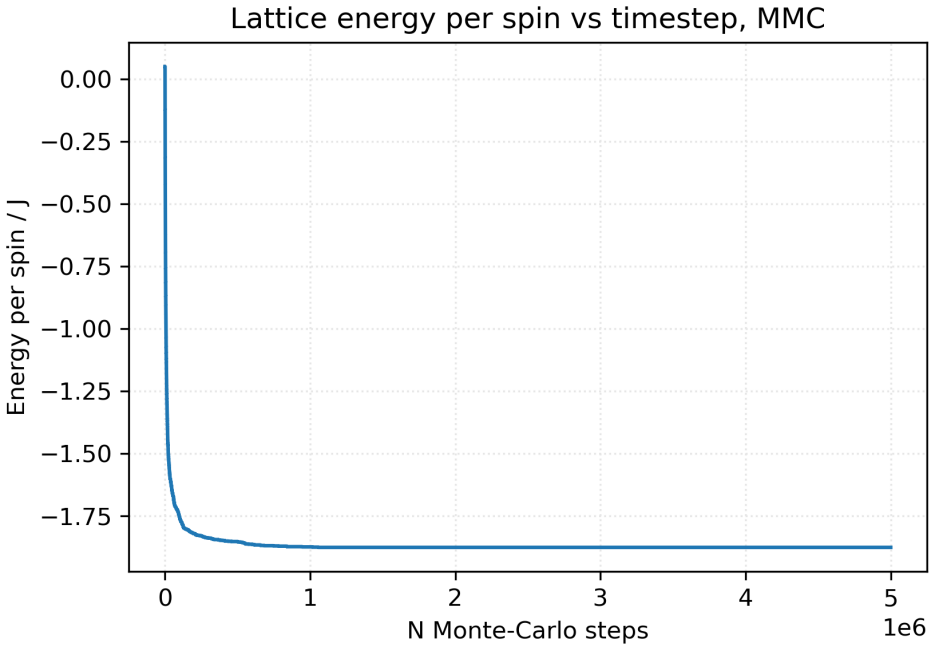


**Figure 47:** Lattice energy per spin vs timestep ($T = 0.001$ K)

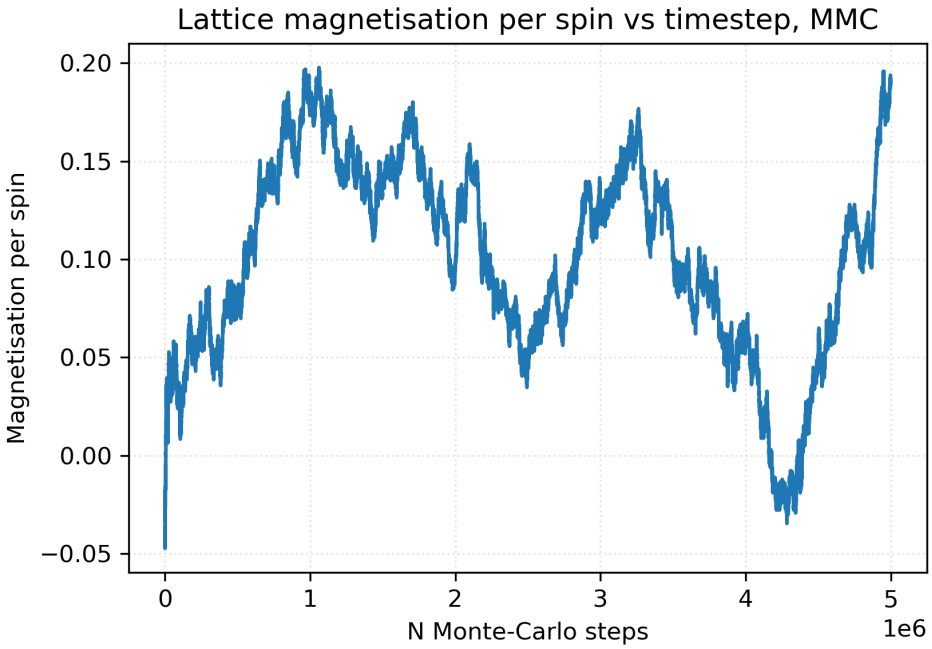


**Figure 48:** Lattice magnetisation per spin vs timestep ($T = 0.001$ K)

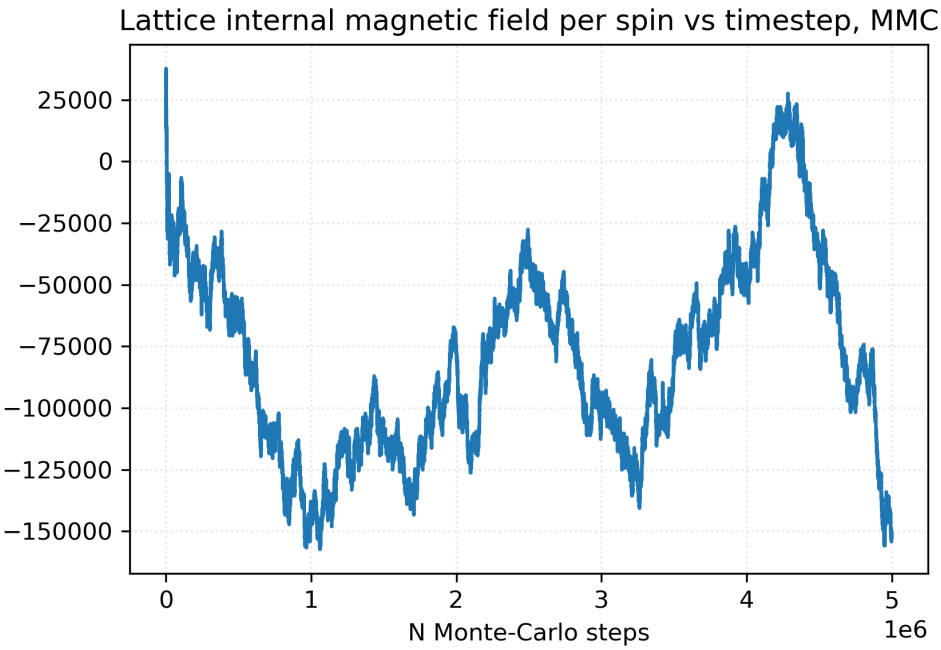


**Figure 49:** Internal magnetic field per spin vs timestep ($T = 0.001$ K)

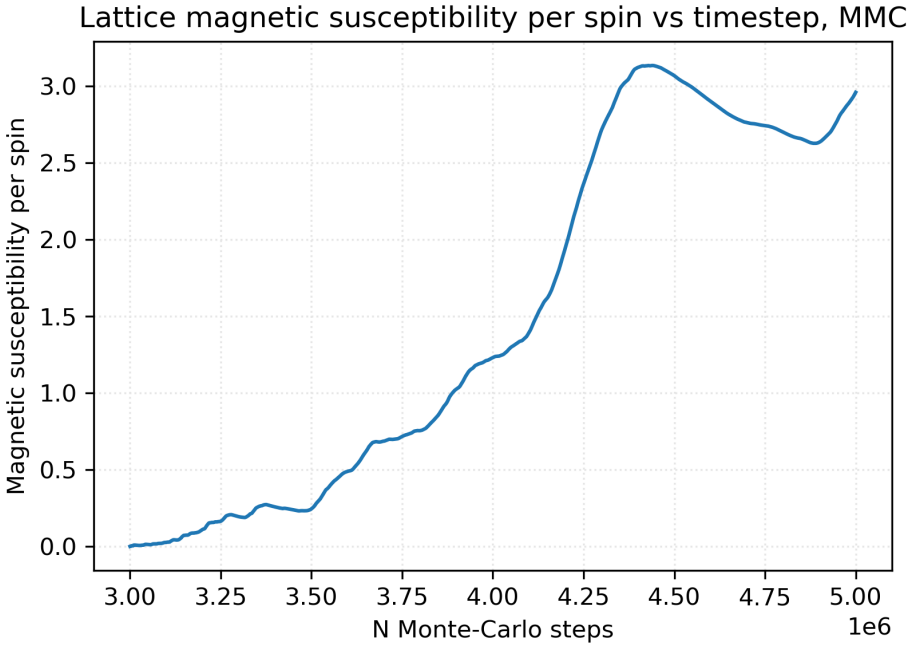


**Figure 50:** Magnetic susceptibility per spin vs timestep ($T = 0.001$ K)

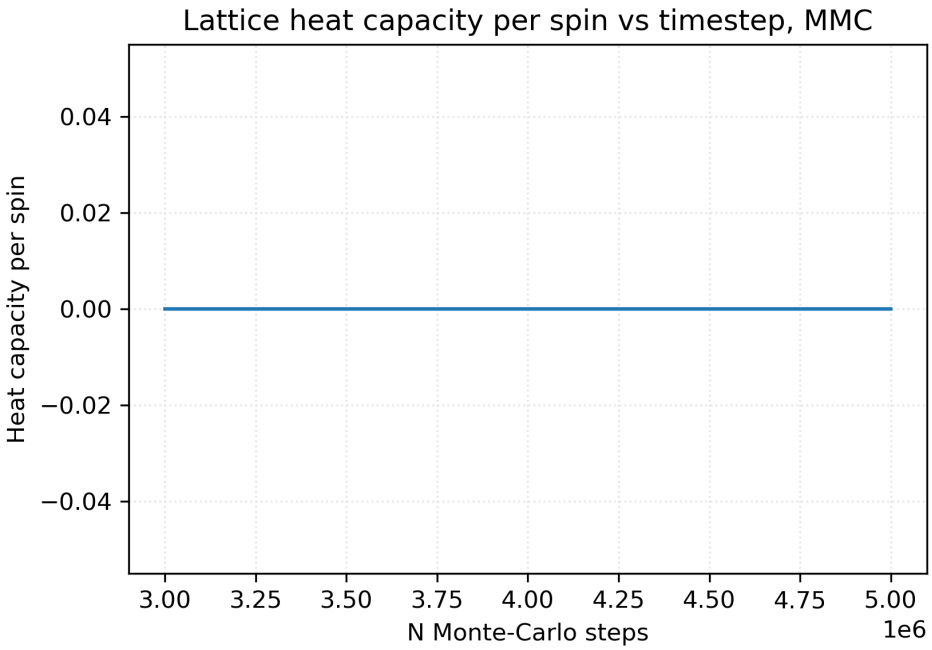


**Figure 51:** Lattice heat capacity per spin vs timestep ($T = 0.001$ K)

## 6.2 $T = 1$ K

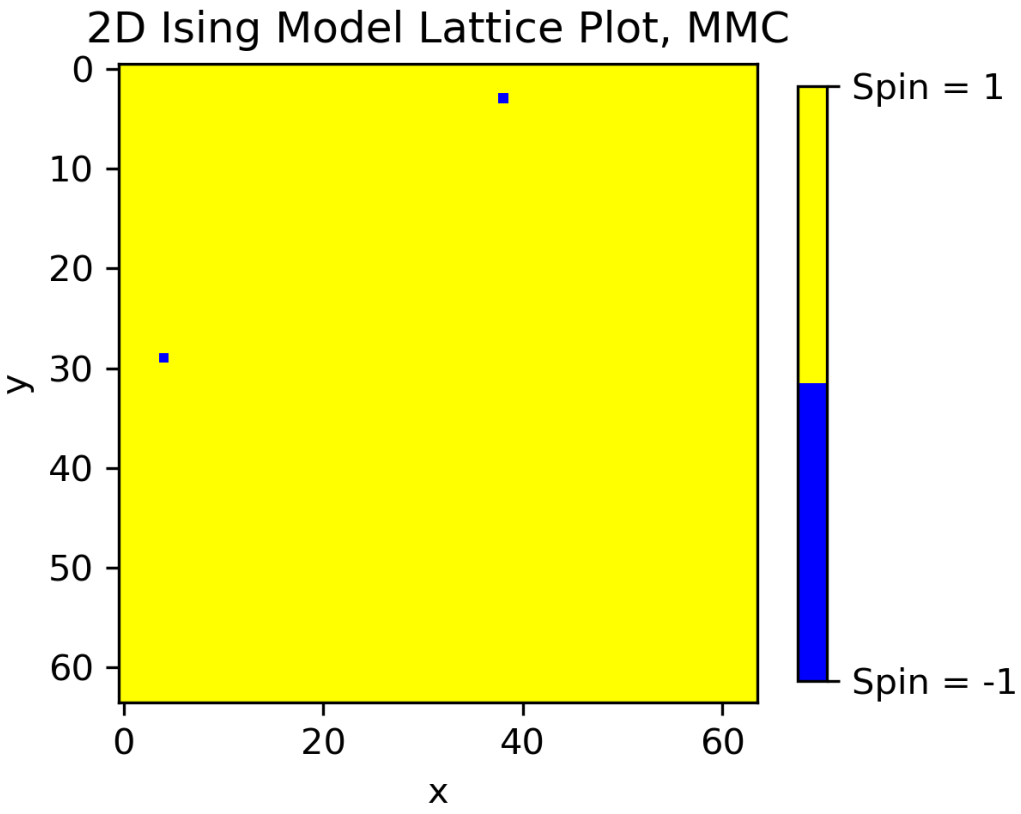


**Figure 52:** End lattice configuration ($T = 1$ K)

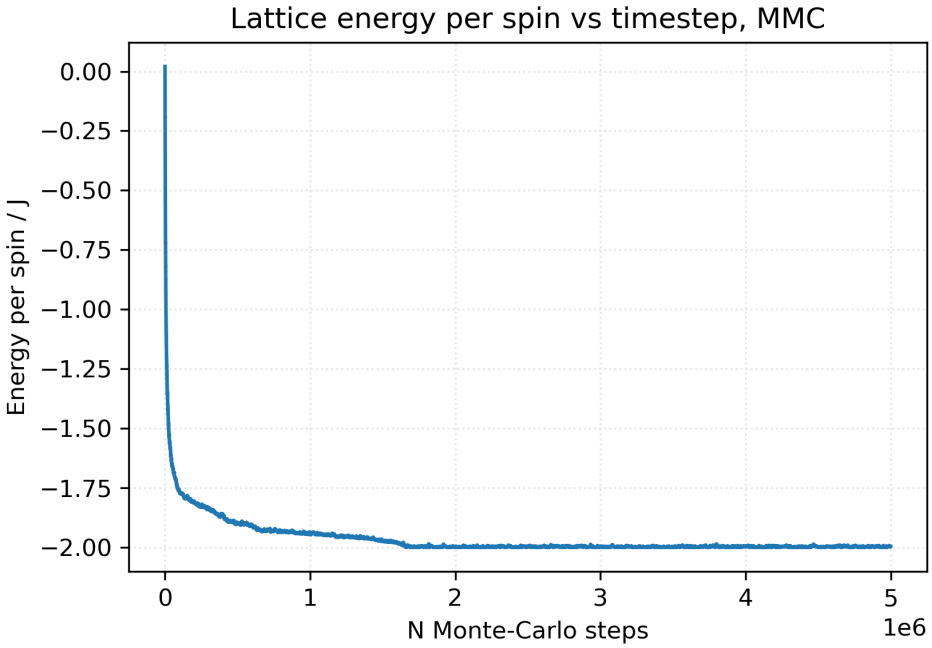


**Figure 53:** Lattice energy per spin vs timestep ($T$ = 1 K)

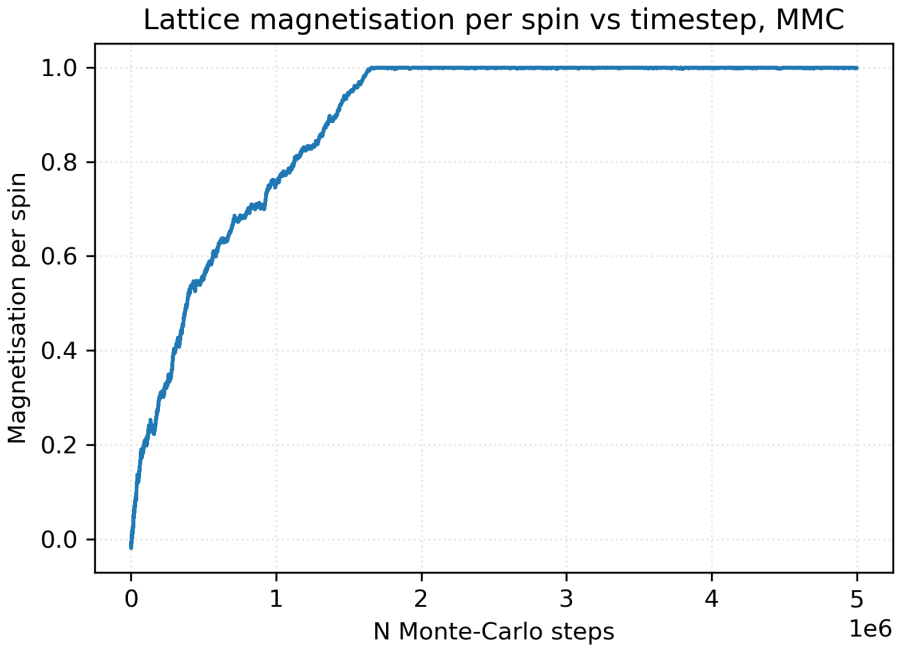


**Figure 54:** Lattice magnetisation per spin vs timestep ($T$ = 1 K)

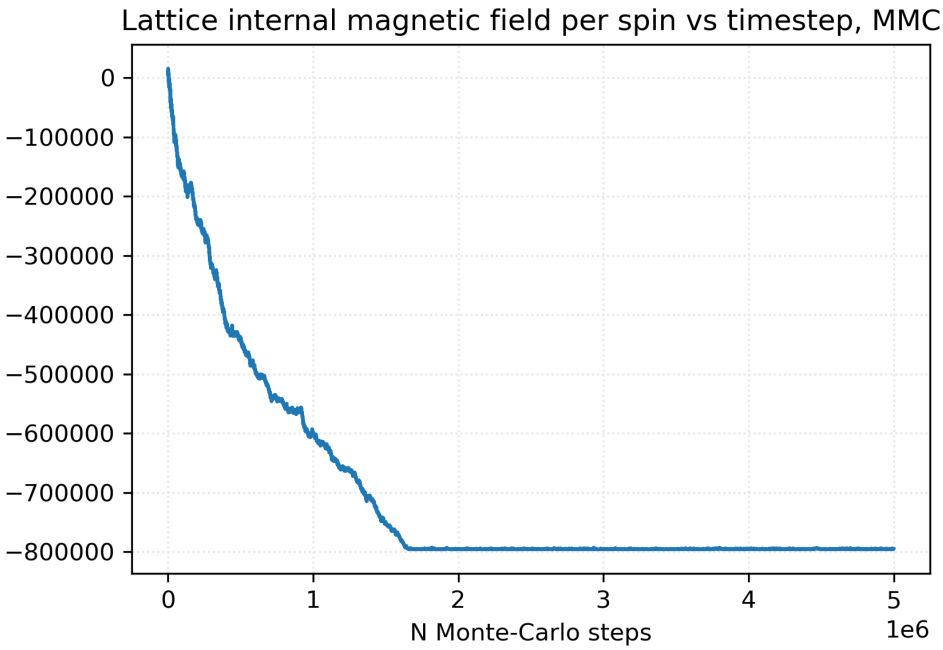


**Figure 55:** Internal magnetic field per spin vs timestep ($T = 1$ K)

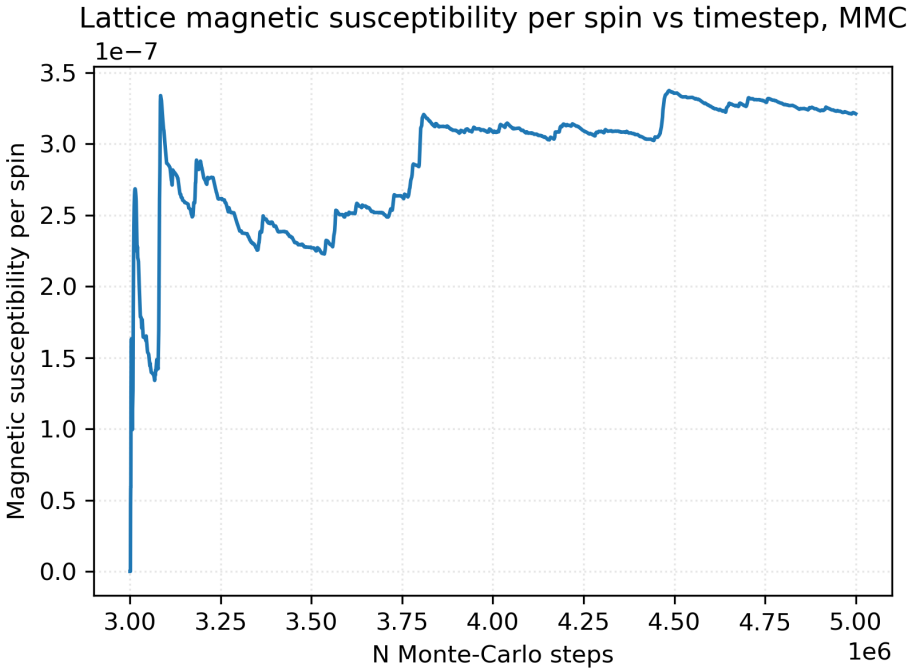


**Figure 56:** Magnetic susceptibility per spin vs timestep ($T = 1$ K)

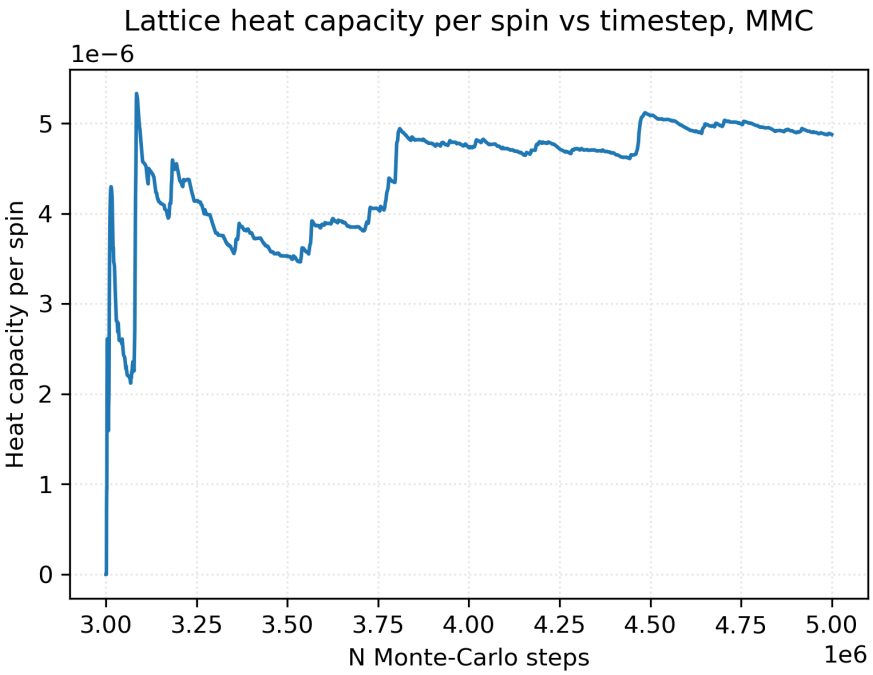


**Figure 57:** Lattice heat capacity per spin vs timestep ($T$ = 1 K)

## 6.3 $T = T_c$

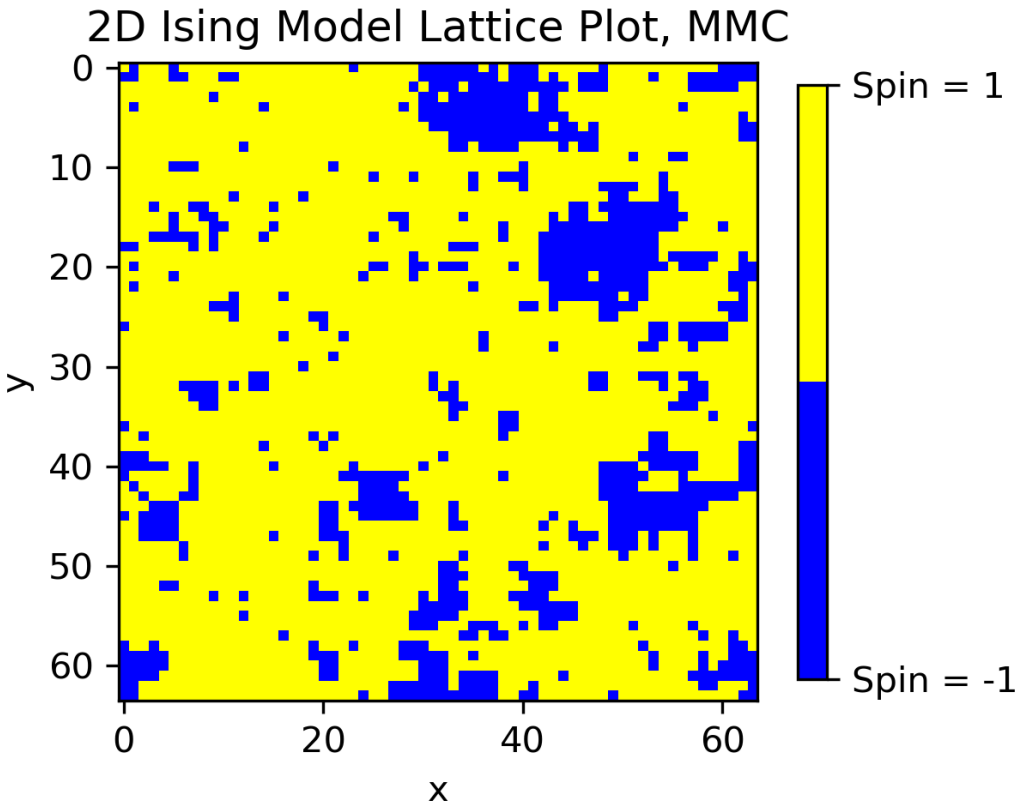


**Figure 58:** End lattice configuration ($T = T_c$)

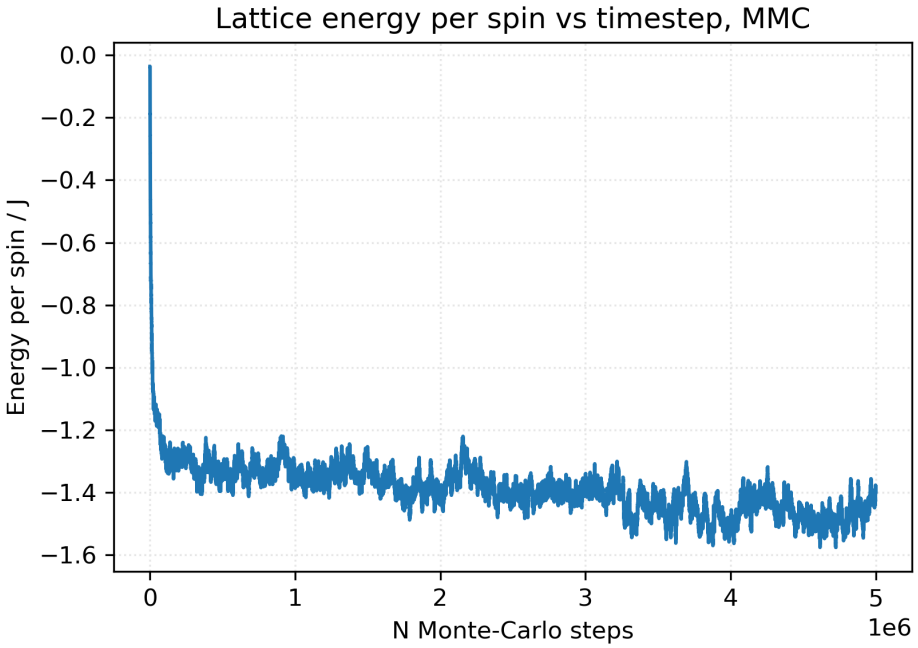


**Figure 59:** Lattice energy per spin vs timestep ($T = T_c$)

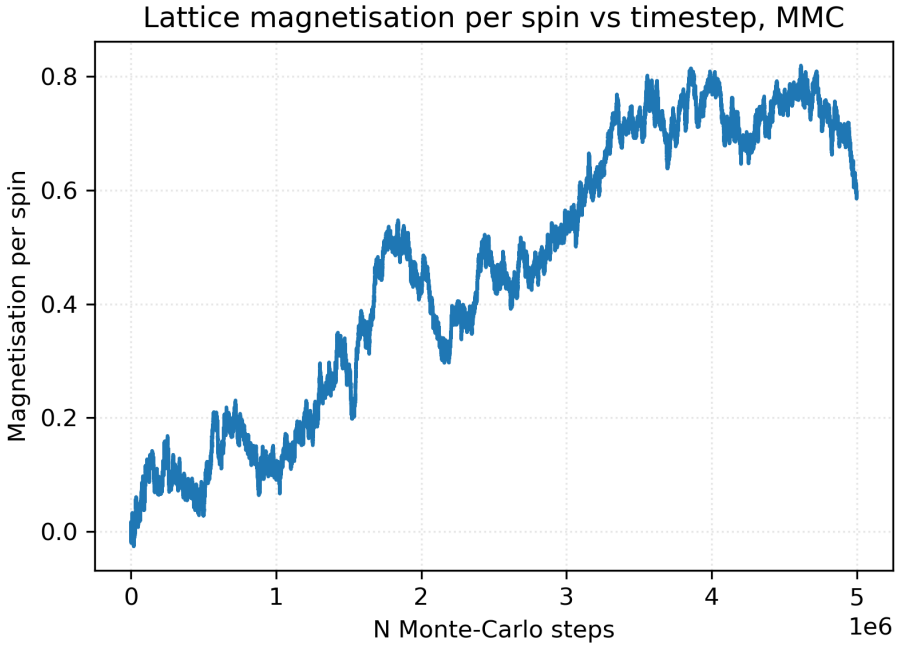


**Figure 60:** Lattice magnetisation per spin vs timestep ($T = T_c$)

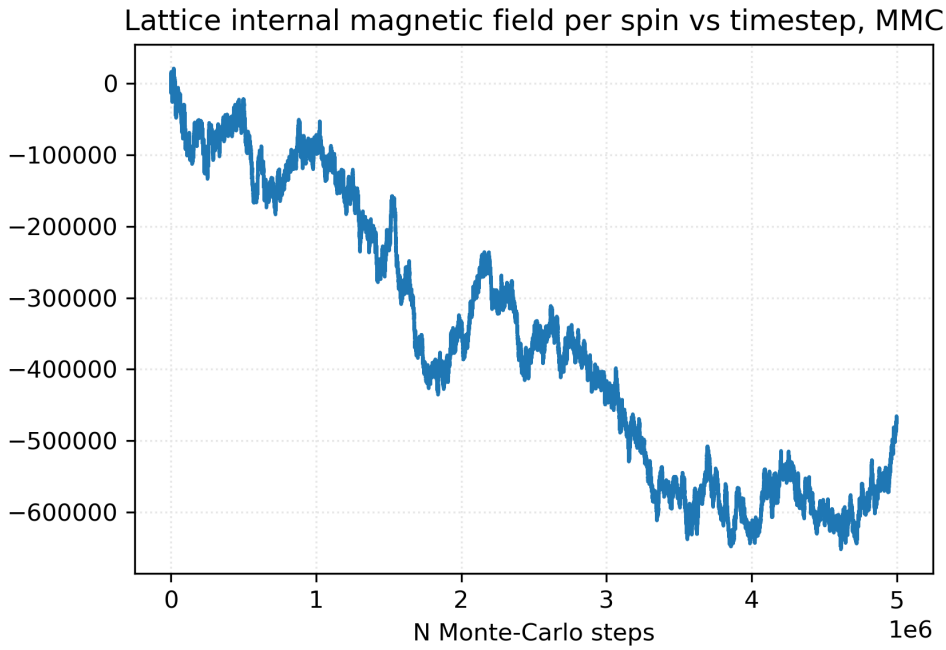


**Figure 61:** Internal magnetic field per spin vs timestep ($T = T_c$)

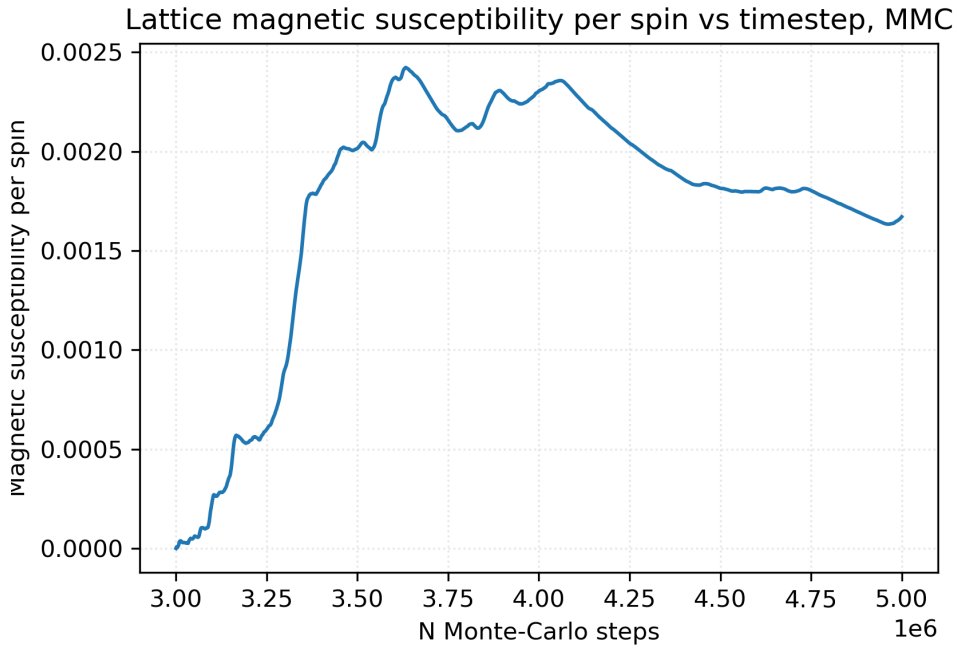


**Figure 62:** Magnetic susceptibility per spin vs timestep ($T = T_c$)

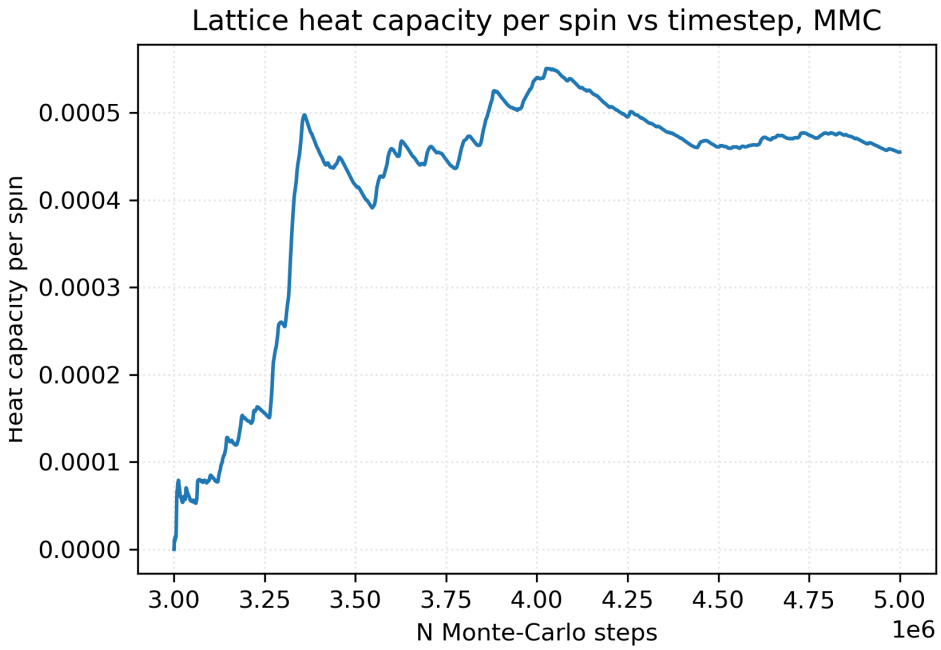


**Figure 63:** Lattice heat capacity per spin vs timestep ($T = T_c$)

### 6.4 $T = 3$ K

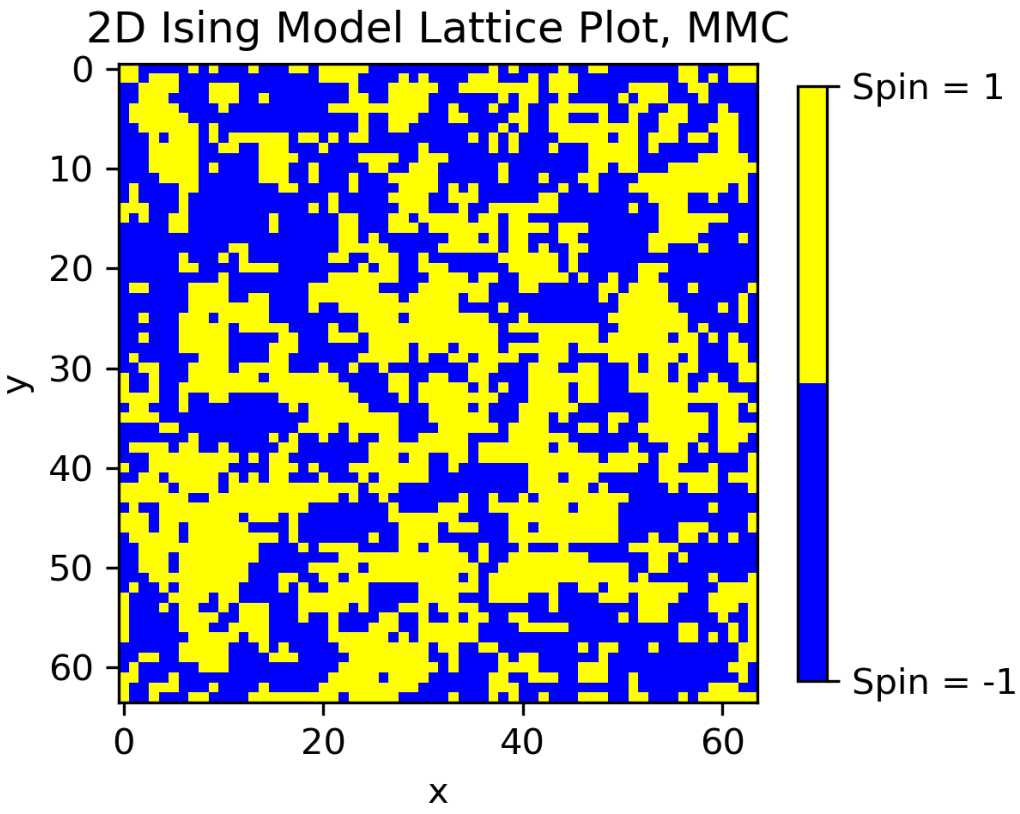


**Figure 64:** End lattice configuration ($T = 3$ K)

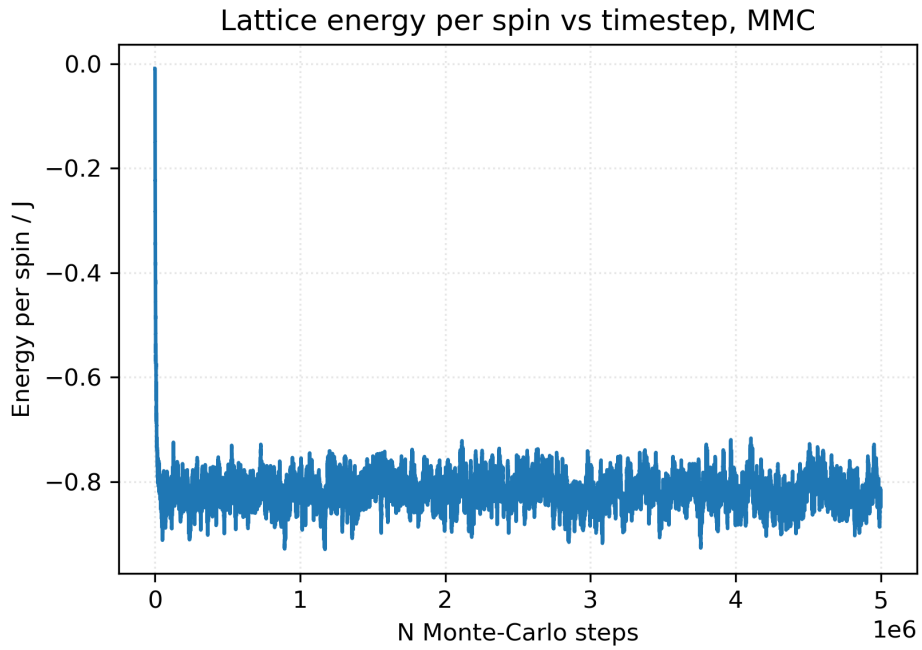


**Figure 65:** Lattice energy per spin vs timestep ($T = 3$ K)

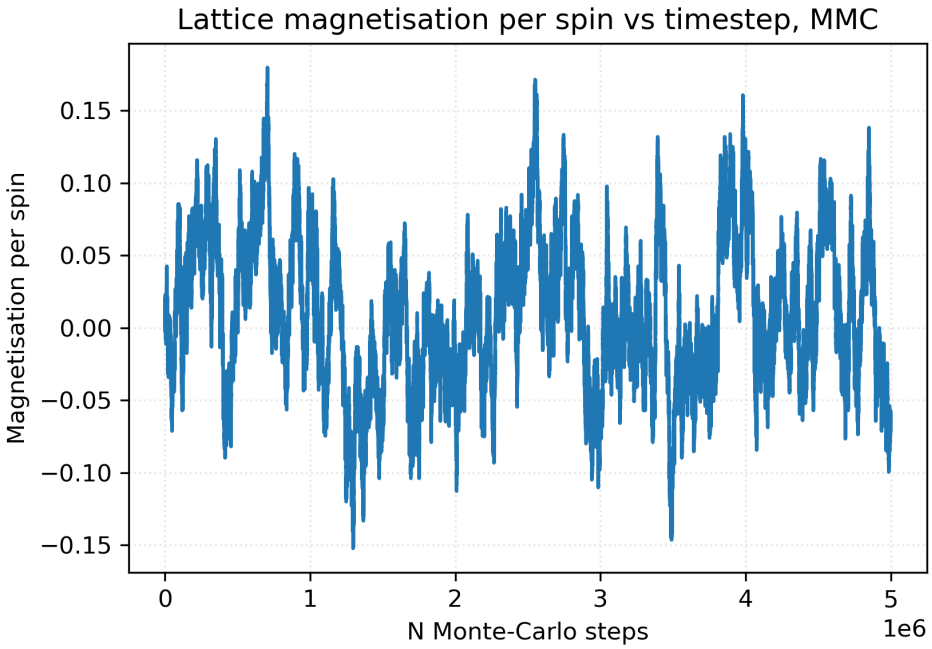


**Figure 66:** Lattice magnetisation per spin vs timestep ($T = 3$ K)

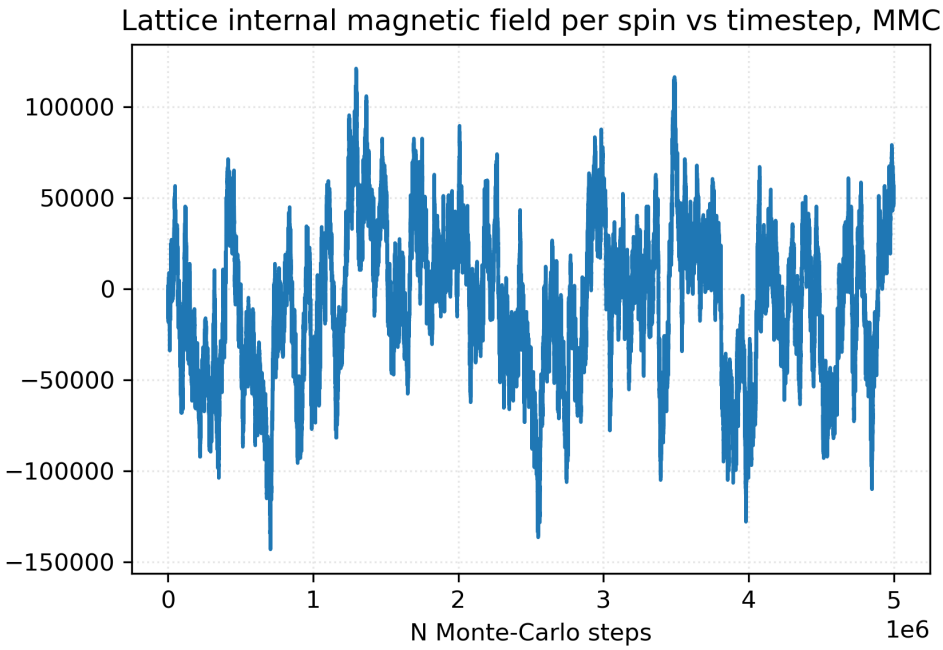


**Figure 67:** Internal magnetic field per spin vs timestep ($T = 3$ K)

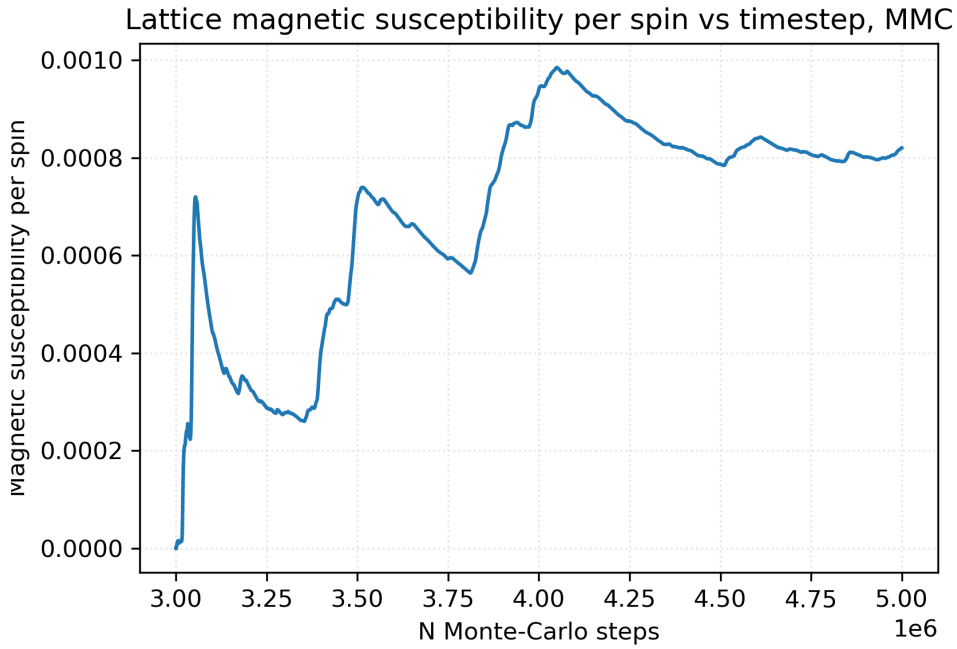


**Figure 68:** Magnetic susceptibility per spin vs timestep ($T = 3$ K)

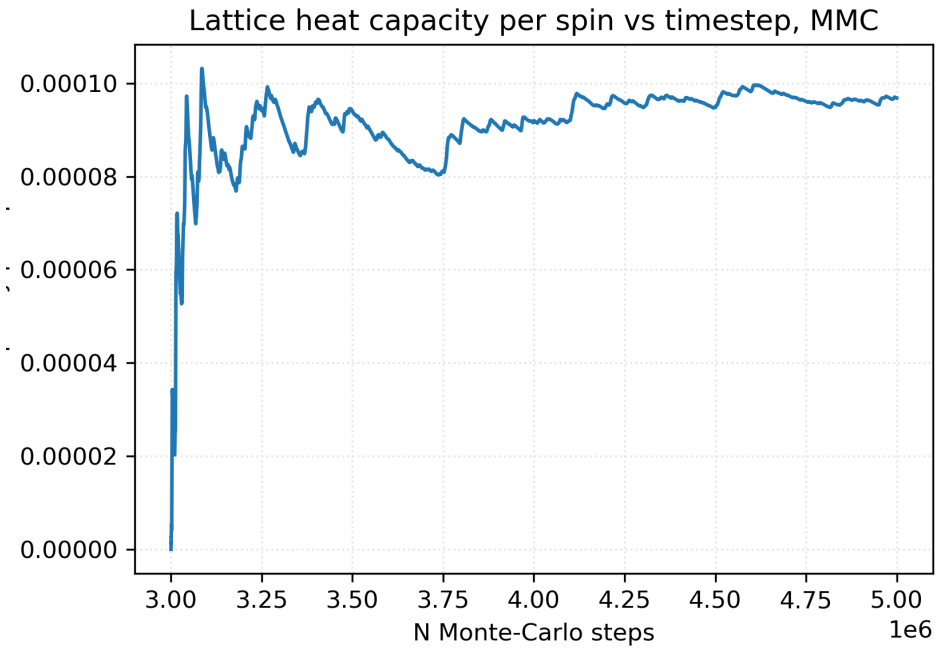


**Figure 69:** Lattice heat capacity per spin vs timestep ($T$ = 3 K)

# 7 Metropolis Algorithm: Changing the Exchange Constant $J$ at $T_{crit}$

## 7.1 $J = 0.01$

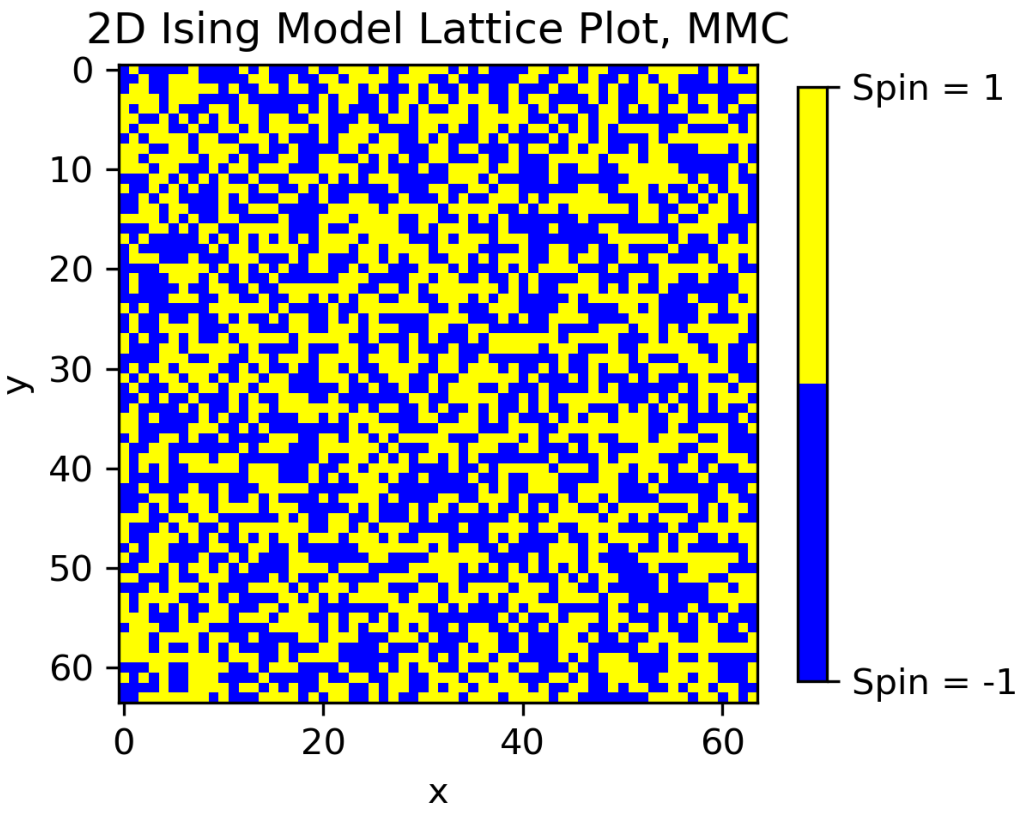


**Figure 70:** End lattice configuration ($J = 0.01$)

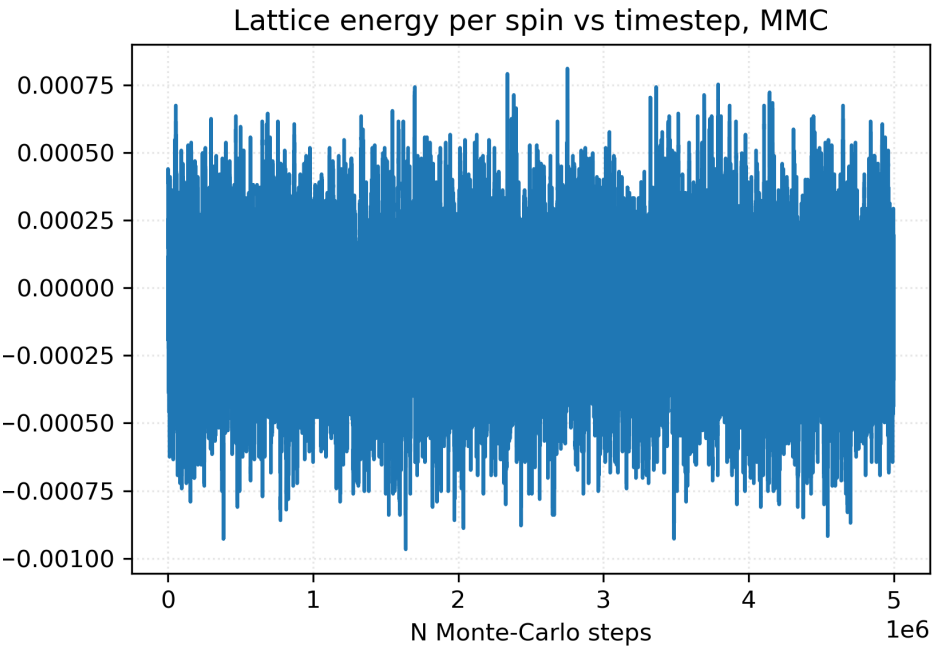


**Figure 71:** Lattice energy per spin vs timestep ($J = 0.01$)

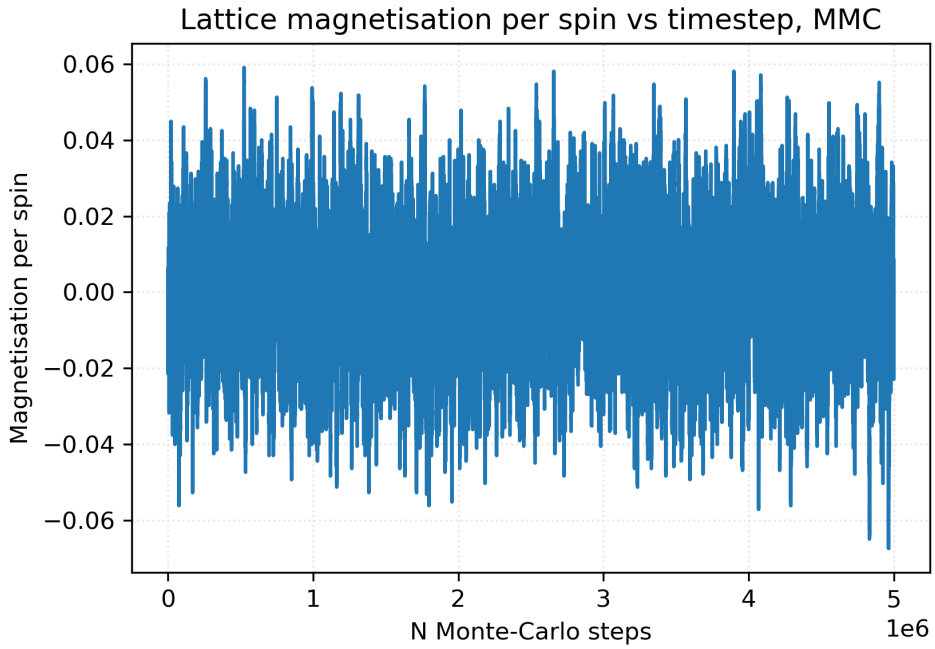


**Figure 72:** Lattice magnetisation per spin vs timestep ($J = 0.01$)

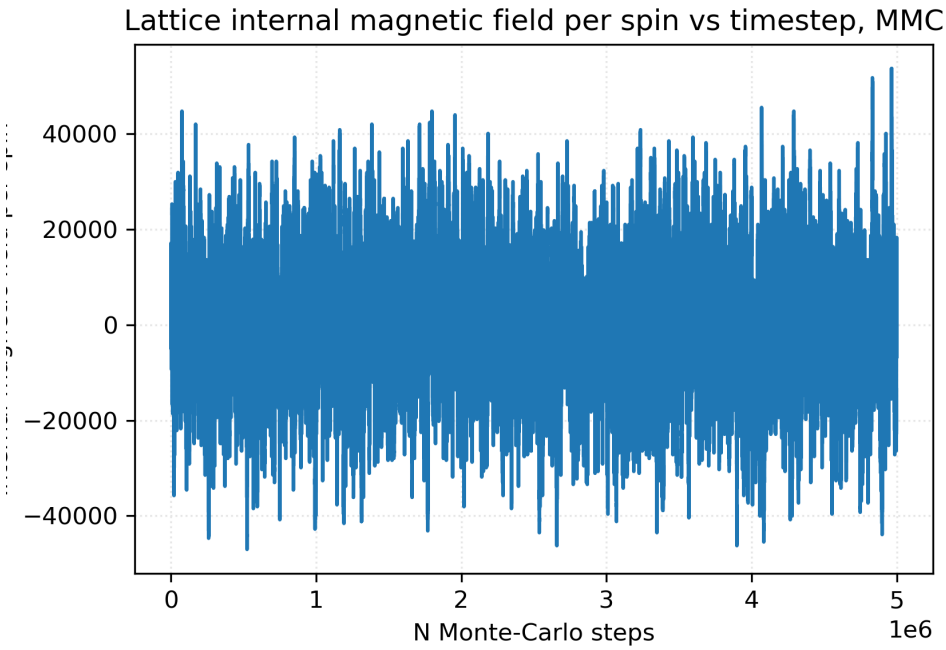


**Figure 73:** Internal magnetic field per spin vs timestep ($J = 0.01$)

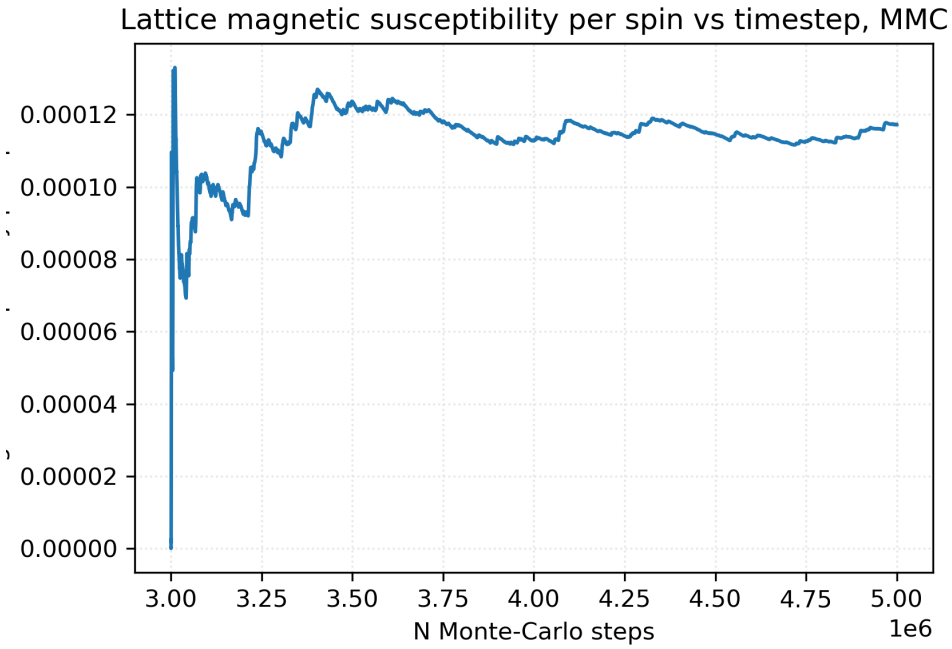


**Figure 74:** Magnetic susceptibility per spin vs timestep ($J = 0.01$)

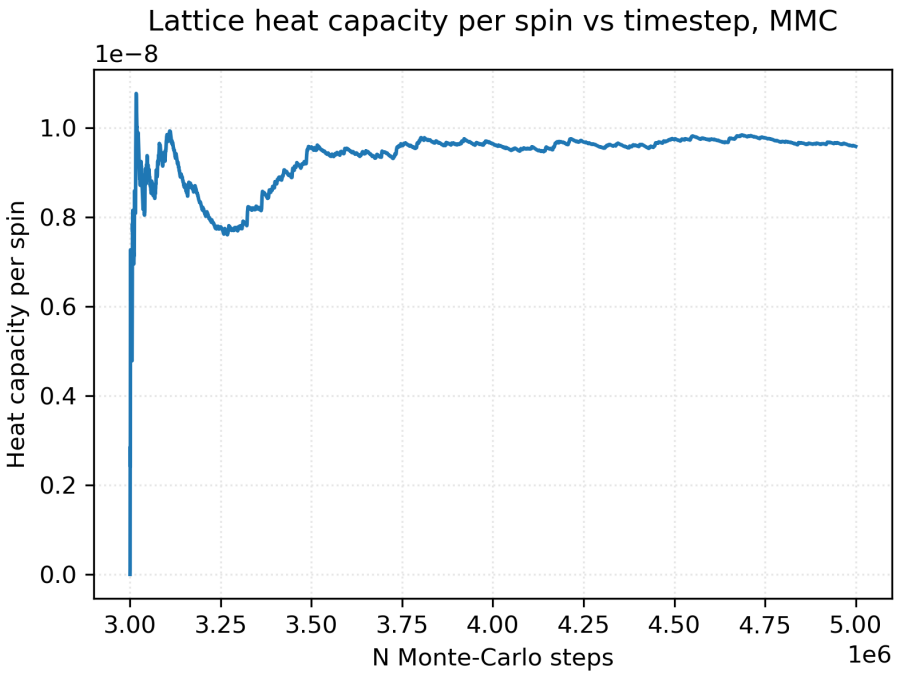


**Figure 75:** Lattice heat capacity per spin vs timestep ($J = 0.01$)

## 7.2 $J = 0.1$

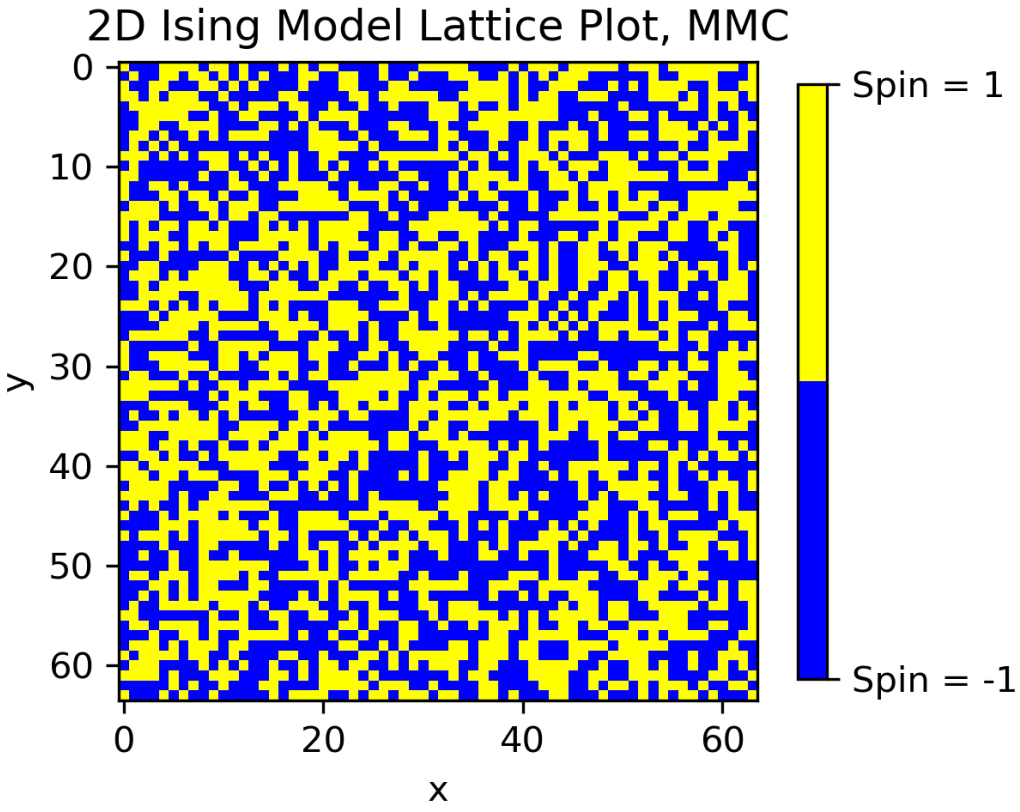


**Figure 76:** End lattice configuration ($J = 0.1$)

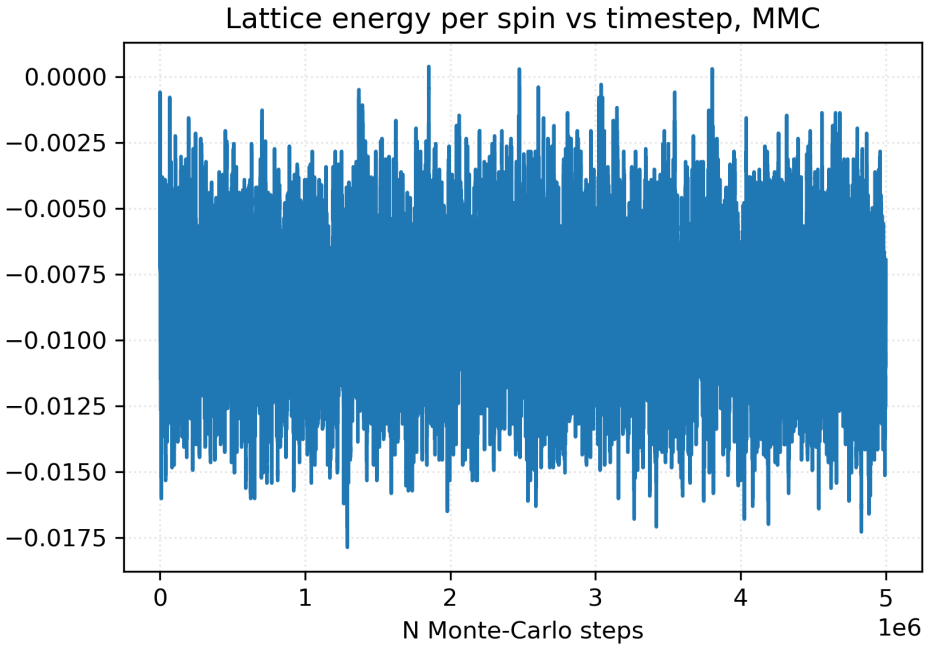


**Figure 77:** Lattice energy per spin vs timestep ($J = 0.1$)

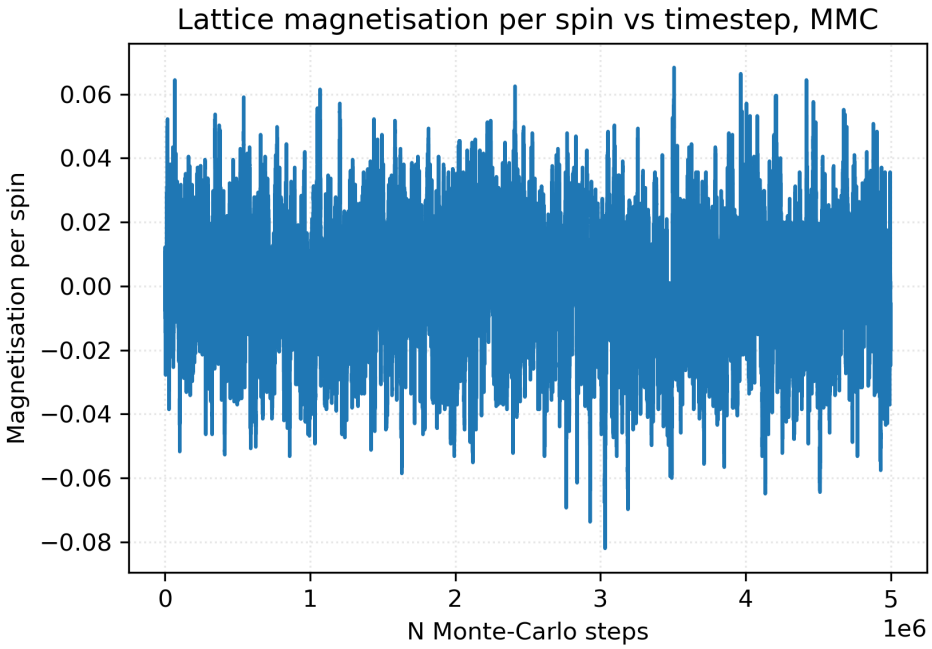


**Figure 78:** Lattice magnetisation per spin vs timestep ($J = 0.1$)

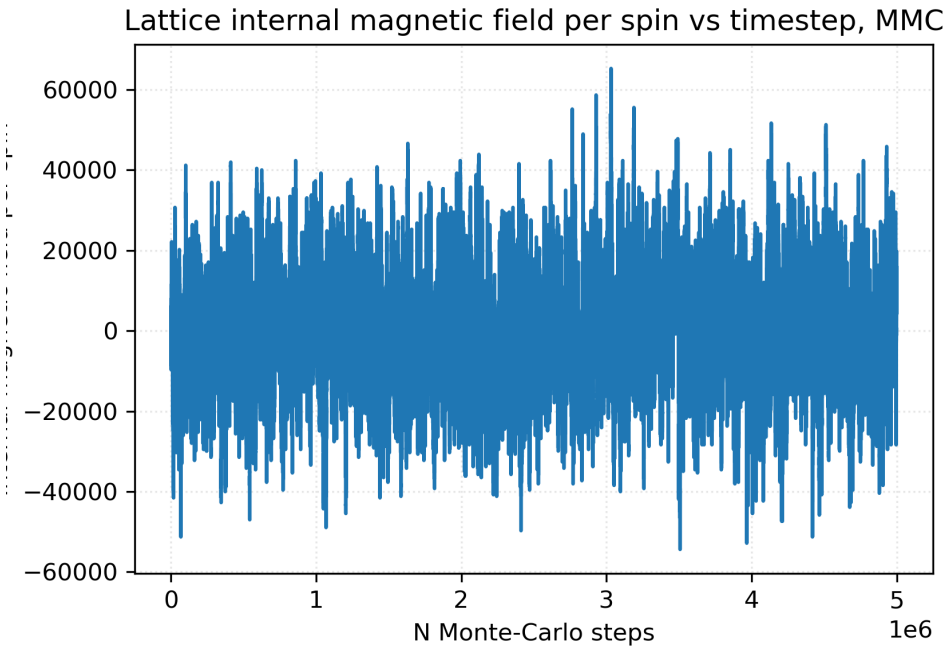


**Figure 79:** Internal magnetic field per spin vs timestep ($J = 0.1$)

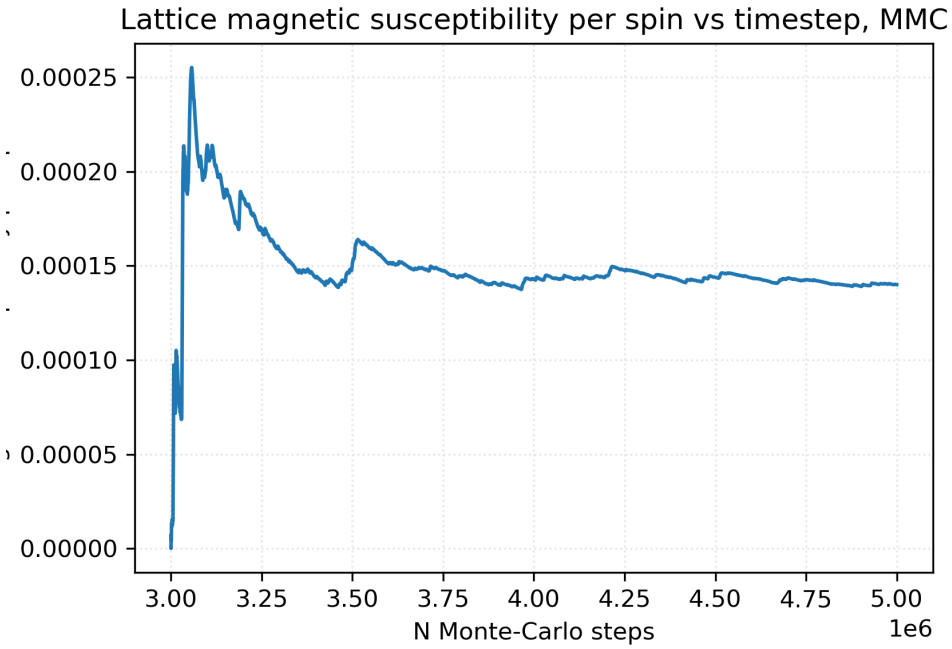


**Figure 80:** Magnetic susceptibility per spin vs timestep ($J = 0.1$)

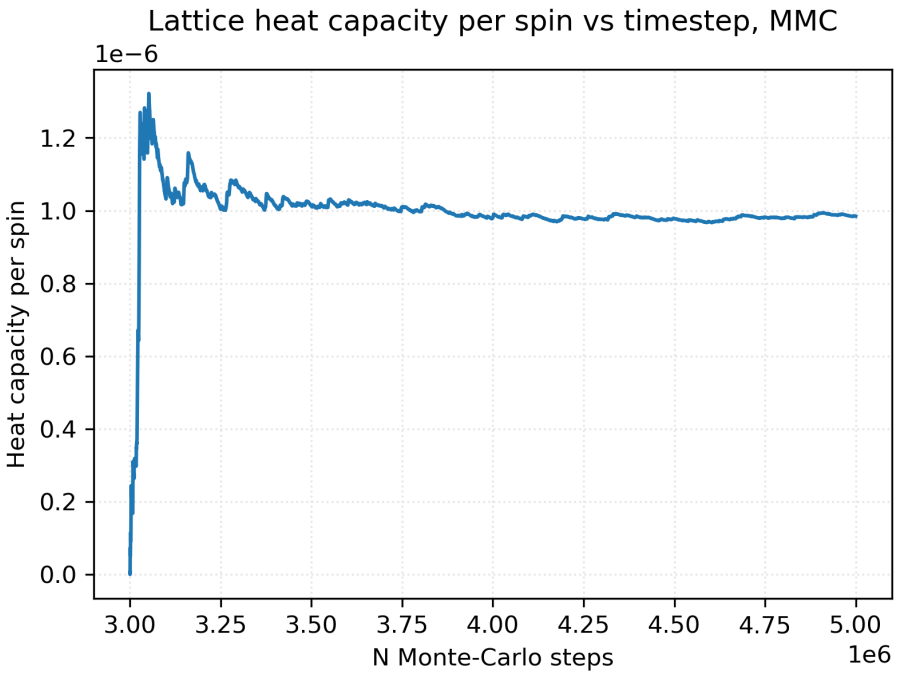


**Figure 81:** Lattice heat capacity per spin vs timestep ($J = 0.1$)

### 7.3 $J = 10$

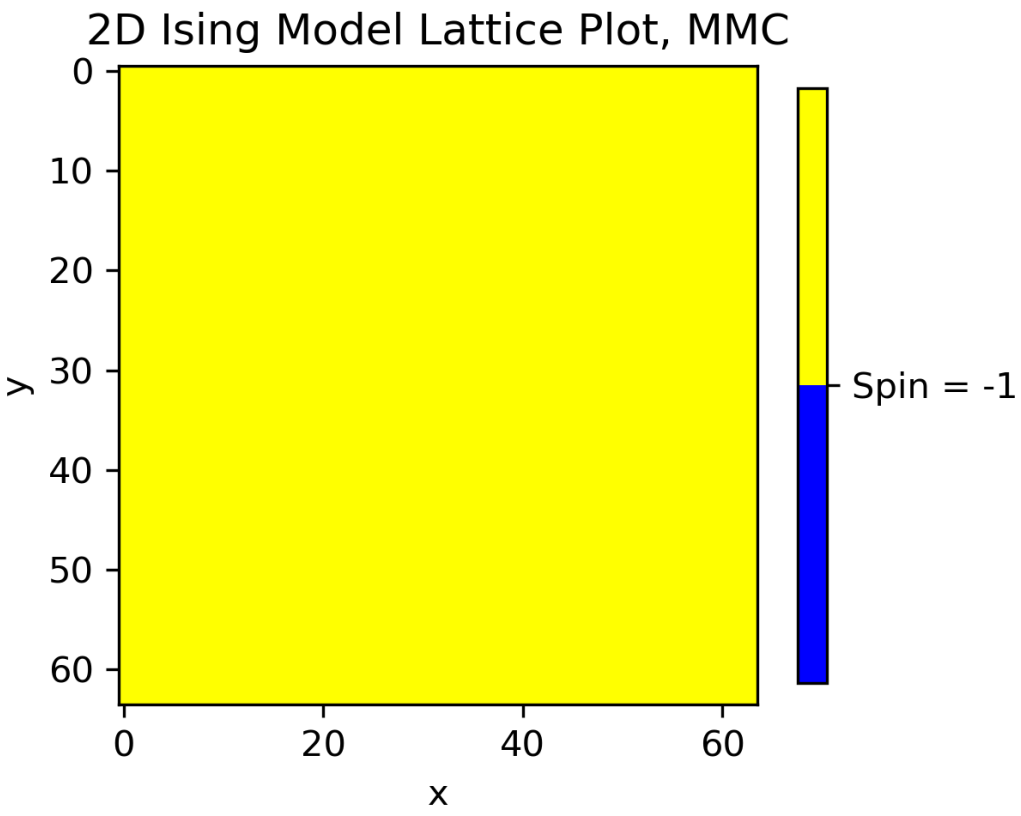


**Figure 82:** End lattice configuration ($J = 10$)

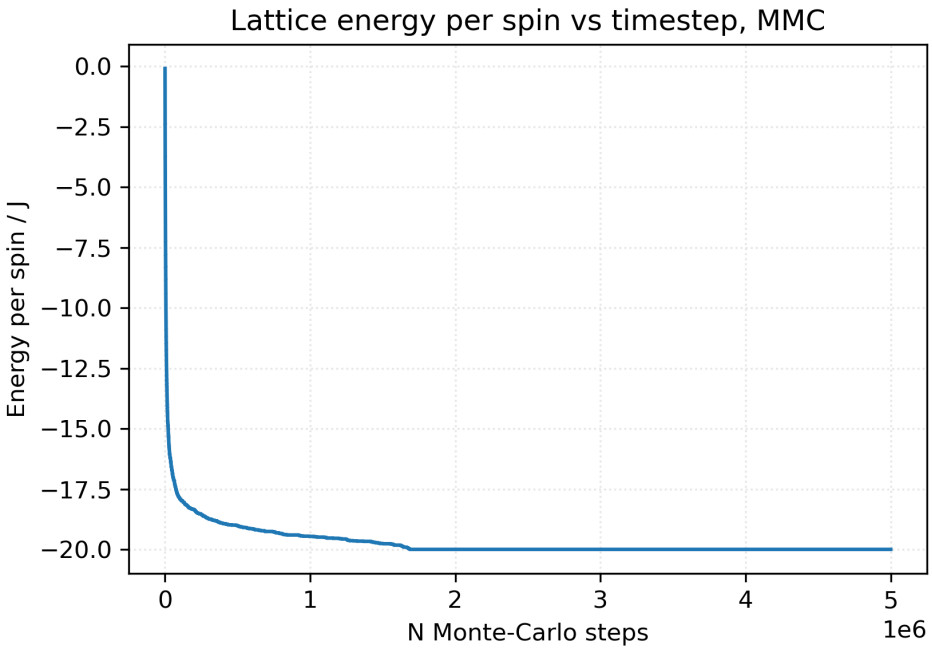


**Figure 83:** Lattice energy per spin vs timestep ($J = 10$)

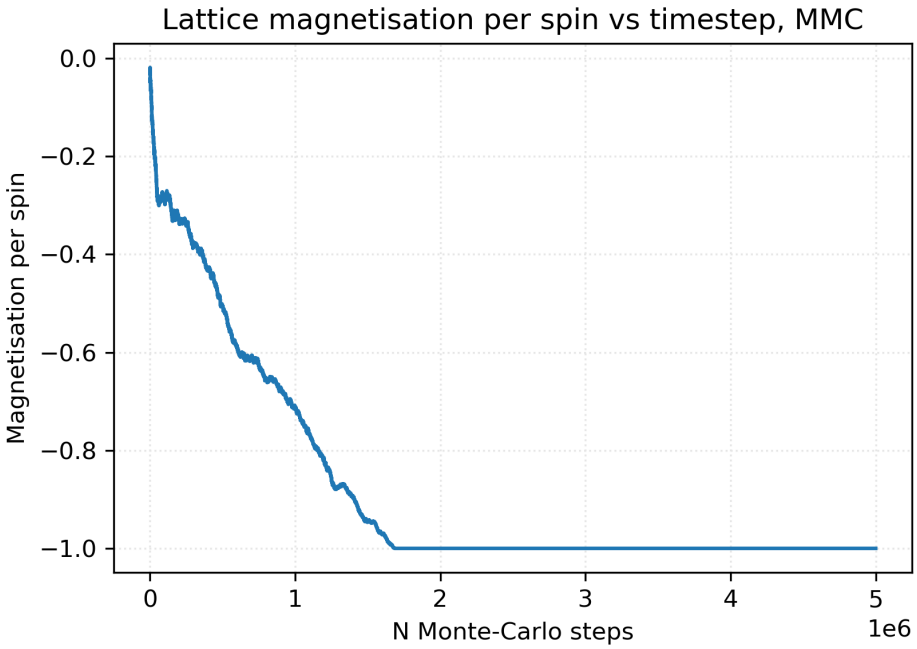


**Figure 84:** Lattice magnetisation per spin vs timestep ($J = 10$)

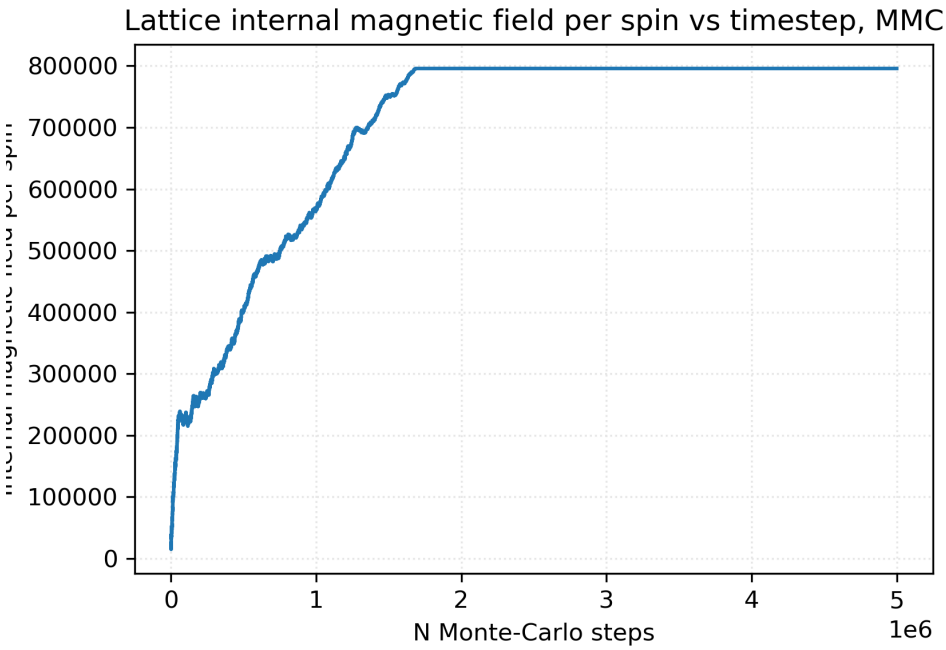


**Figure 85:** Internal magnetic field per spin vs timestep ($J = 10$)

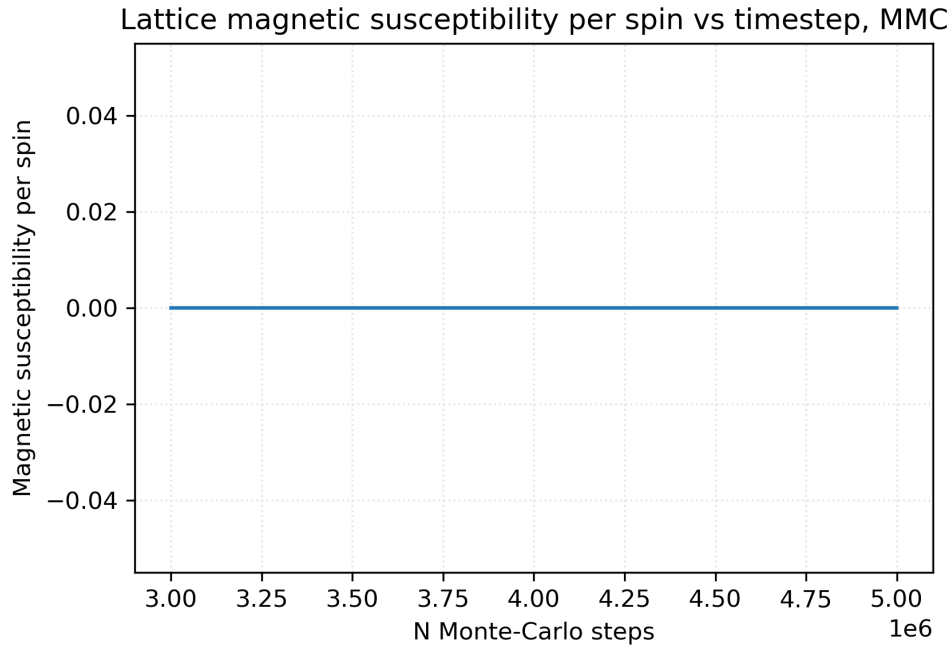


**Figure 86:** Magnetic susceptibility per spin vs timestep ($J = 10$)

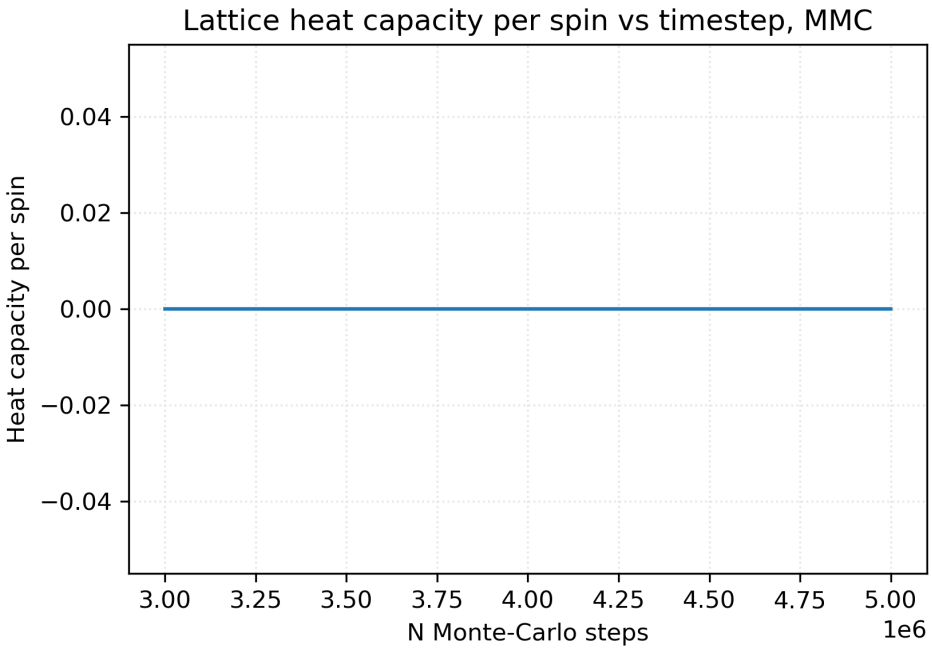


**Figure 87:** Lattice heat capacity per spin vs timestep ($J = 10$)

### 7.4 $J = 100$

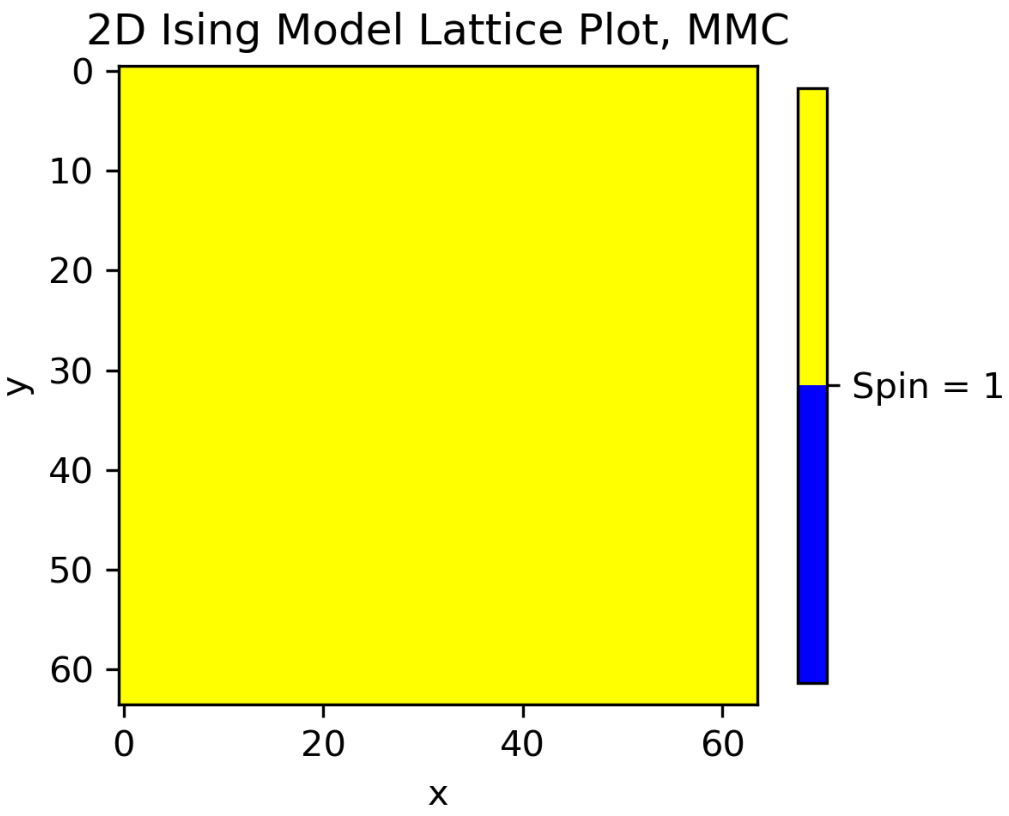


**Figure 88:** End lattice configuration ($J = 100$)

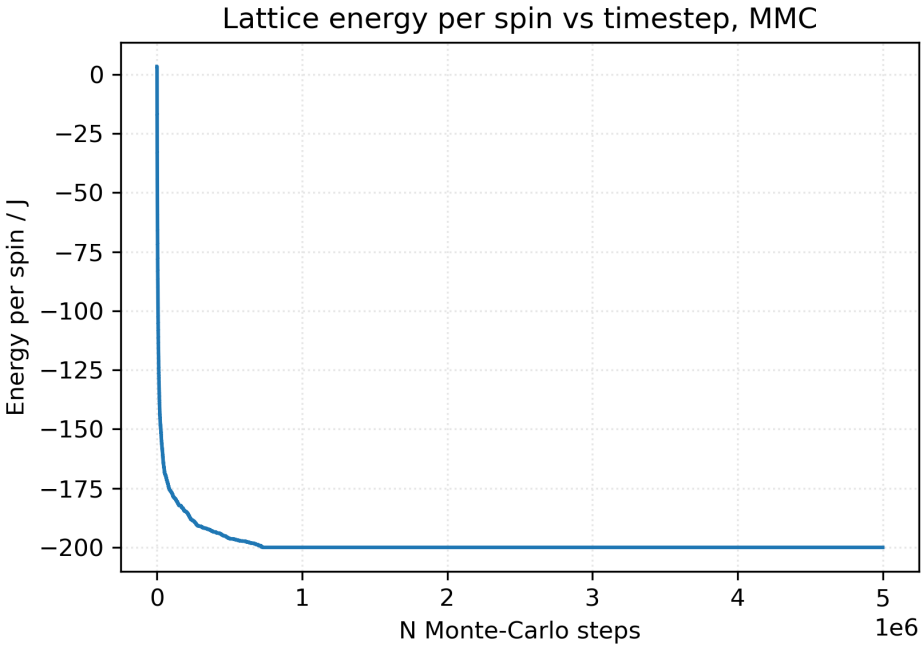


**Figure 89:** Lattice energy per spin vs timestep ($J = 100$)

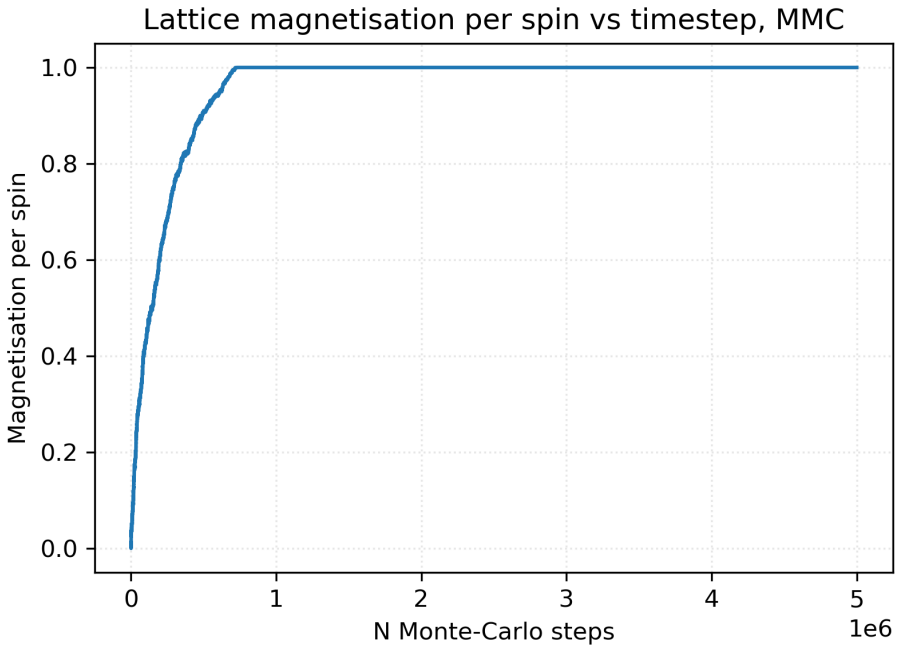


**Figure 90:** Lattice magnetisation per spin vs timestep ($J = 100$)

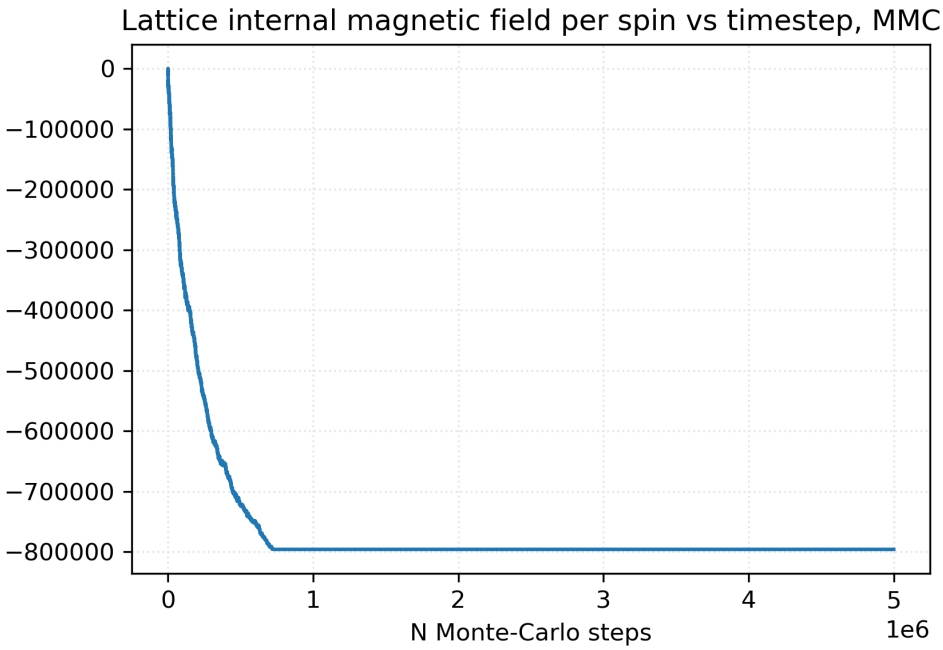


**Figure 91:** Internal magnetic field per spin vs timestep ($J = 100$)

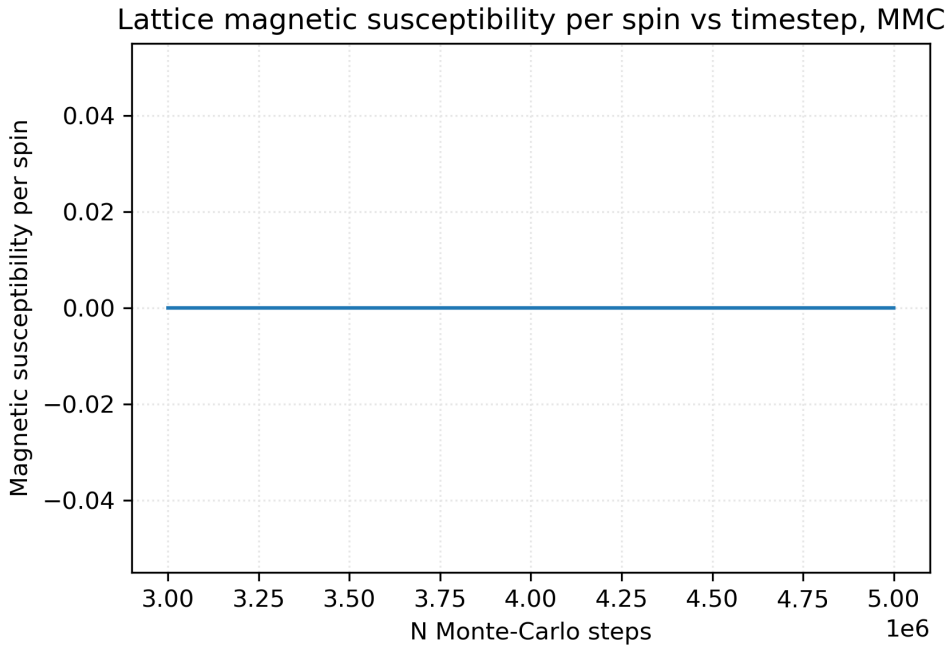


**Figure 92:** Magnetic susceptibility per spin vs timestep ($J = 100$)

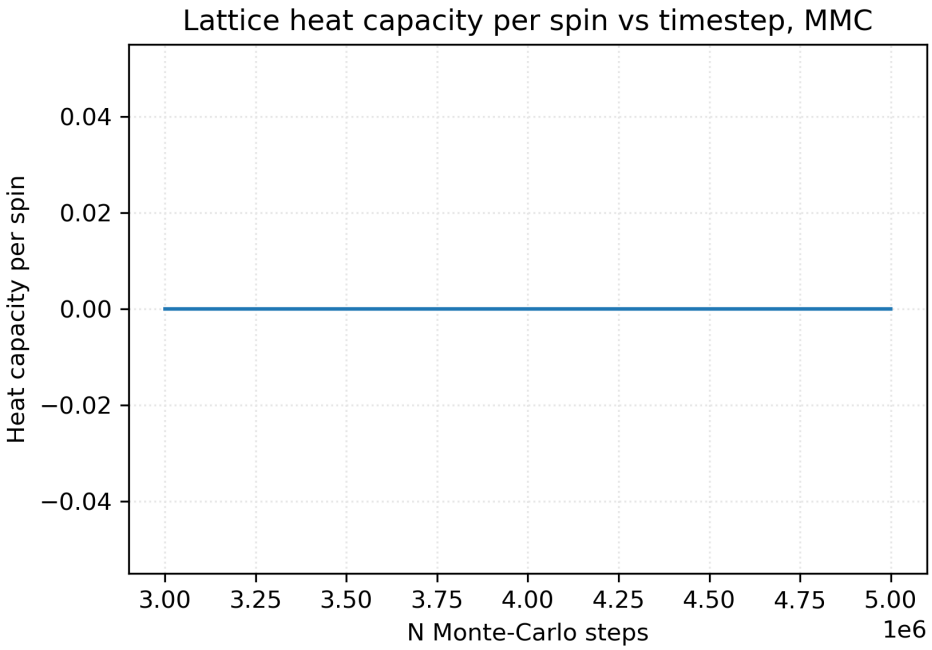


**Figure 93:** Lattice heat capacity per spin vs timestep ($J = 100$)

# 8 Metropolis Algorithm: Changing the Magnetic Field direction and magnitude $B$ at $T_{crit}$

## 8.1 $B = 1$ T

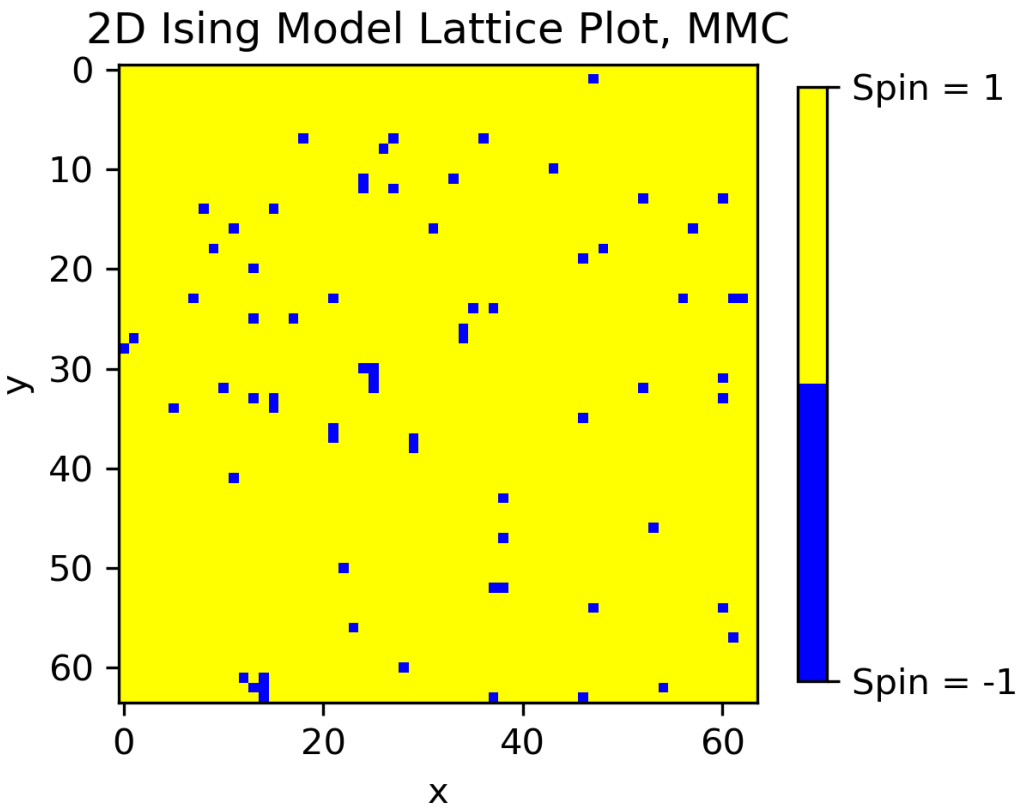


**Figure 94:** End lattice configuration ($B = 1$ T)

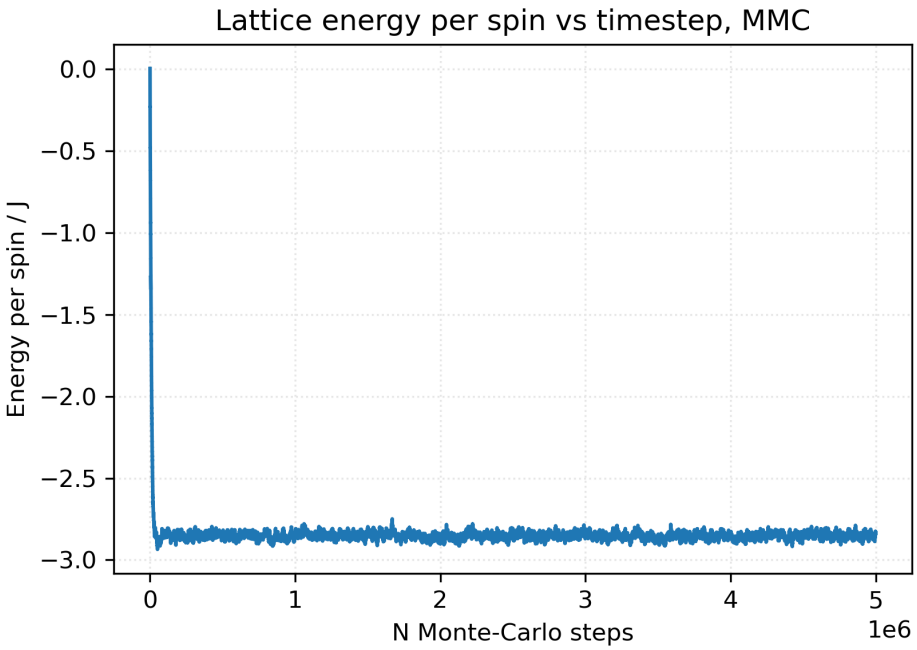


**Figure 95:** Lattice energy per spin vs timestep ($B$ = 1 T)

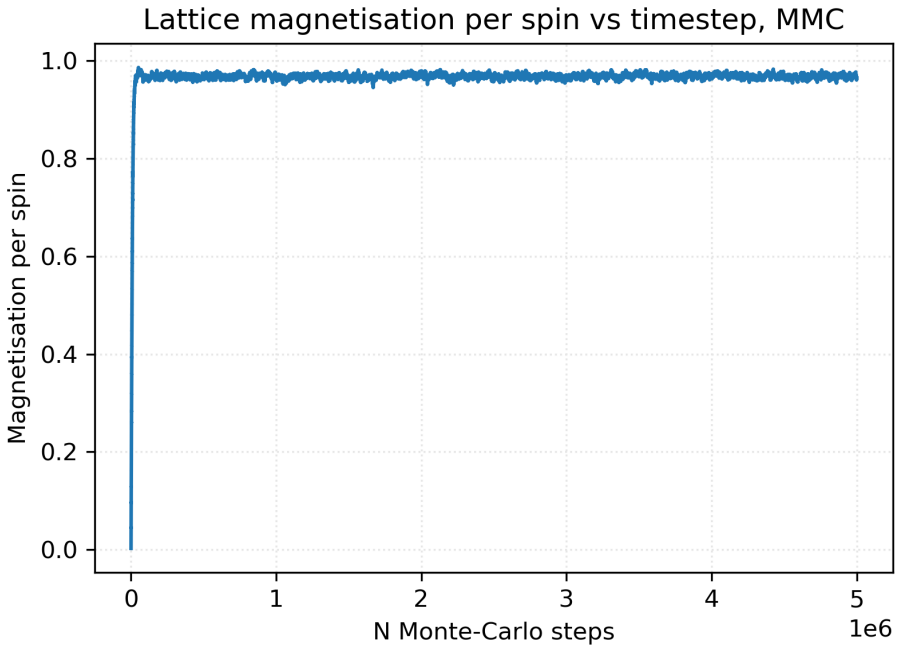


**Figure 96:** Lattice magnetisation per spin vs timestep ($B$ = 1 T)

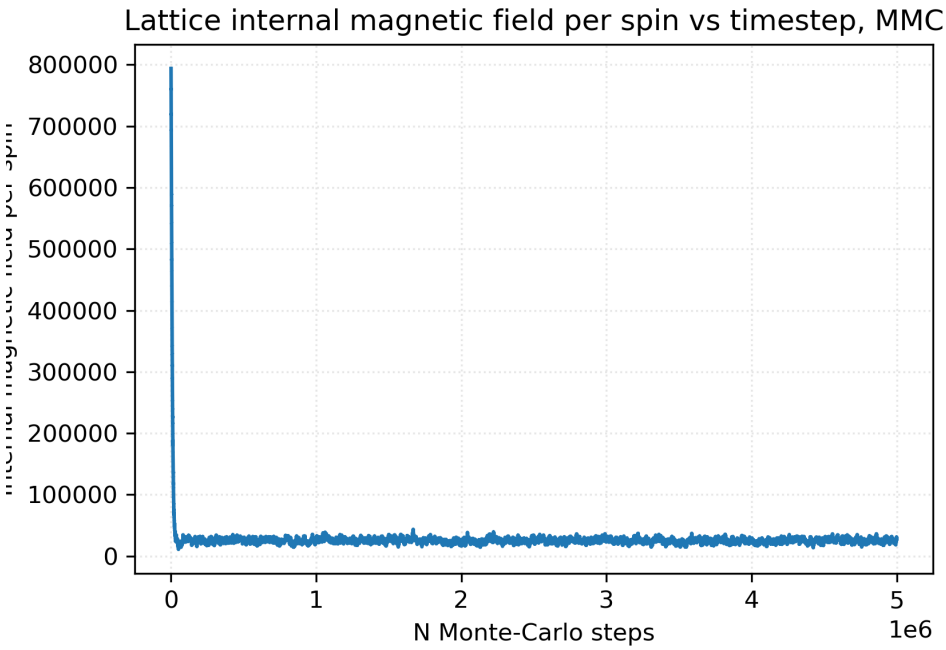


**Figure 97:** Internal magnetic field per spin vs timestep ($B$ = 1 T)

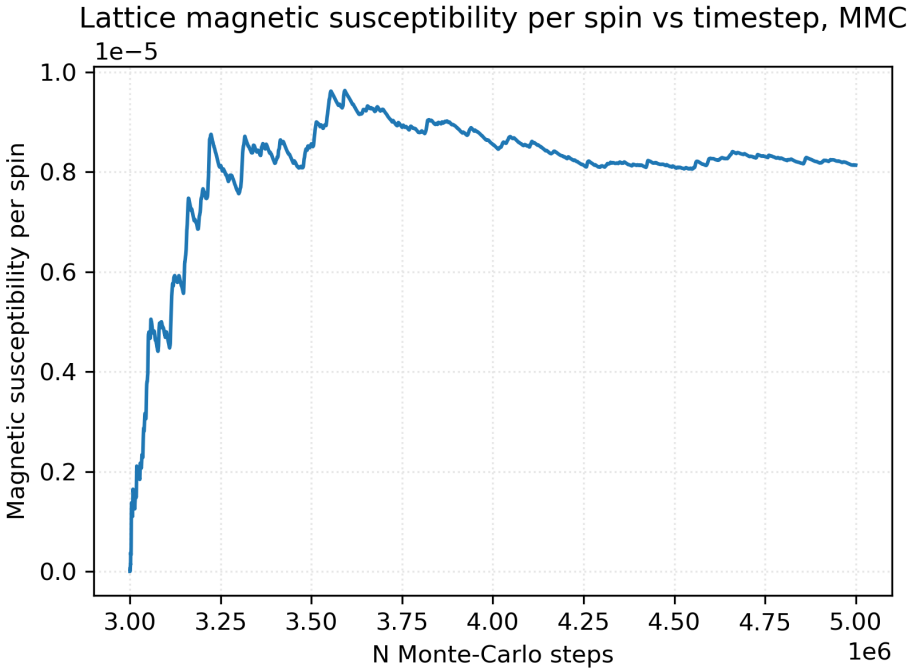


**Figure 98:** Magnetic susceptibility per spin vs timestep ($B$ = 1 T)

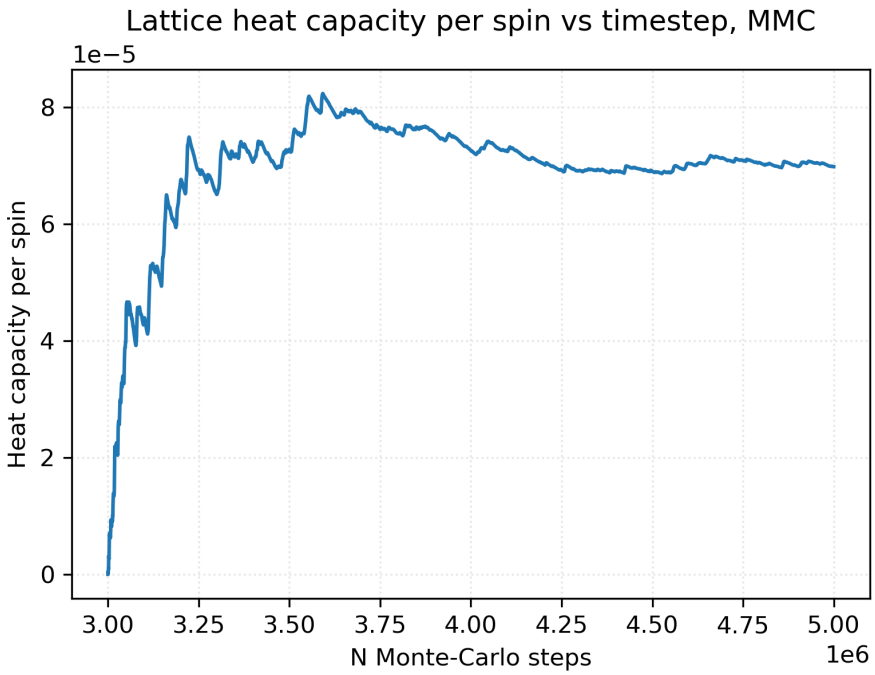


**Figure 99:** Lattice heat capacity per spin vs timestep ($B = 1$ T)

## 8.2 $B = -10$ T

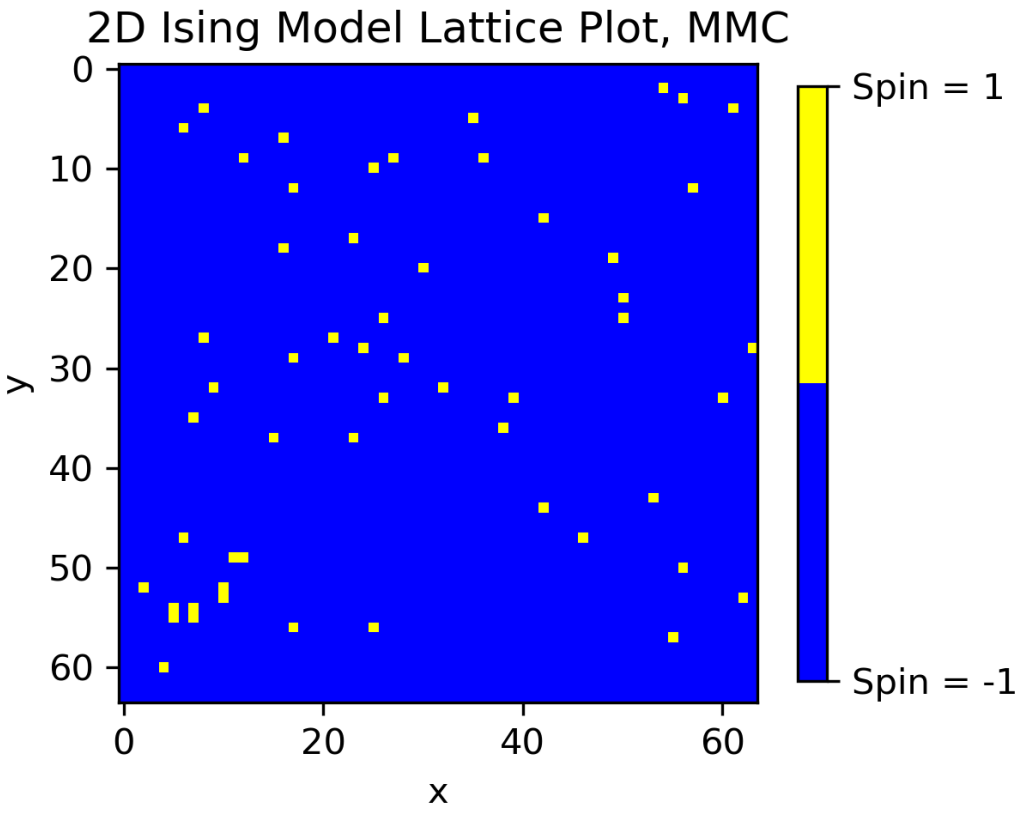


**Figure 100:** End lattice configuration ($B = -10$ T)

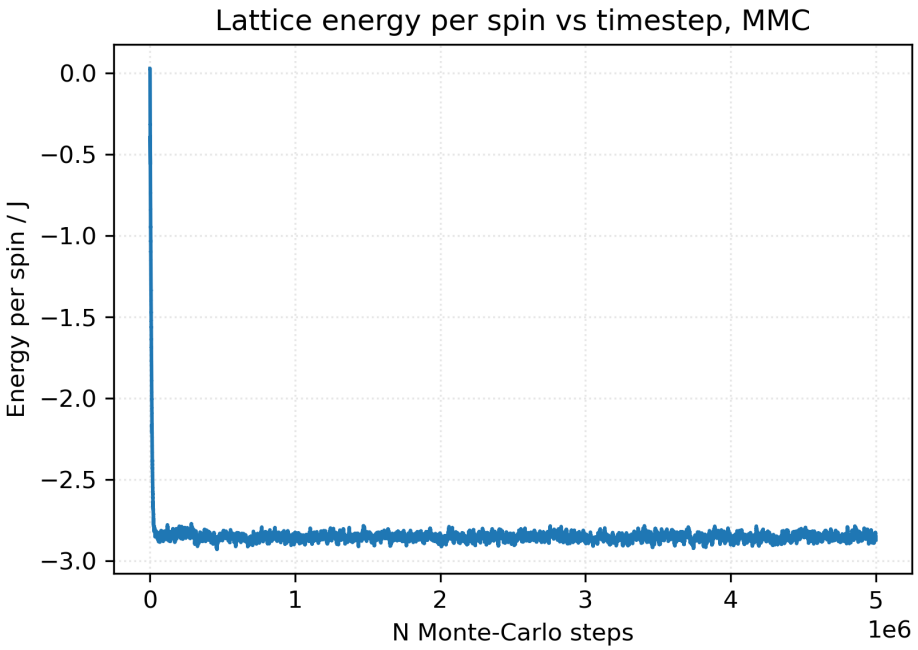


**Figure 101:** Lattice energy per spin vs timestep ($B = -10$ T)

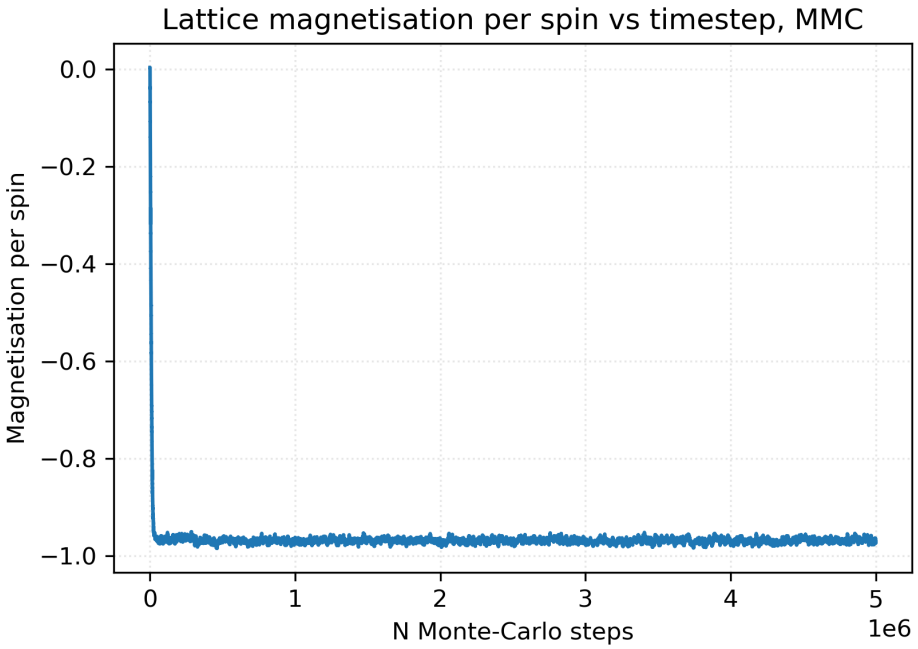


**Figure 102:** Lattice magnetisation per spin vs timestep ($B = -10$ T)

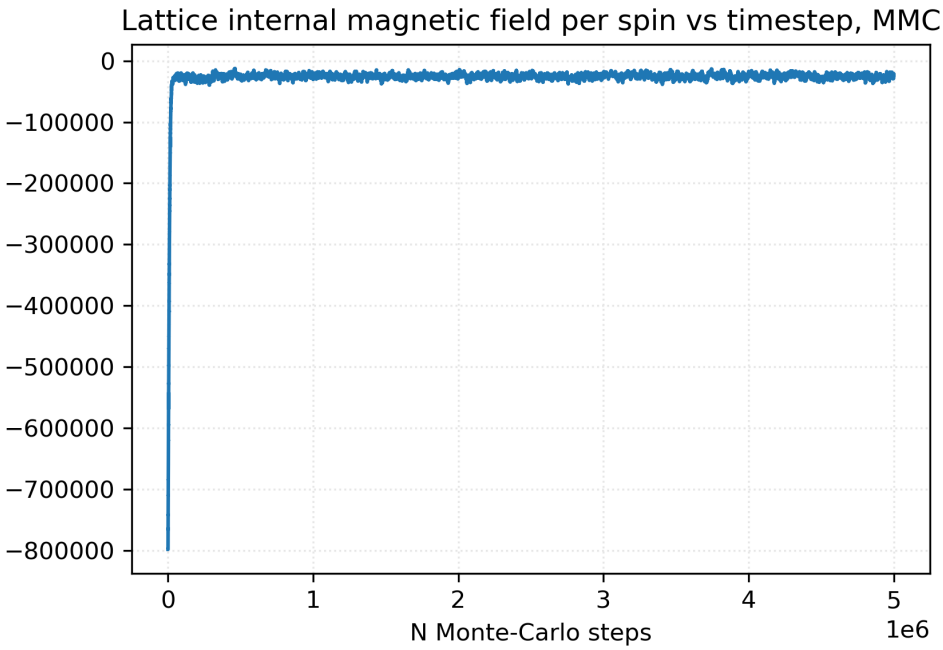


**Figure 103:** Internal magnetic field per spin vs timestep ($B = -10$ T)

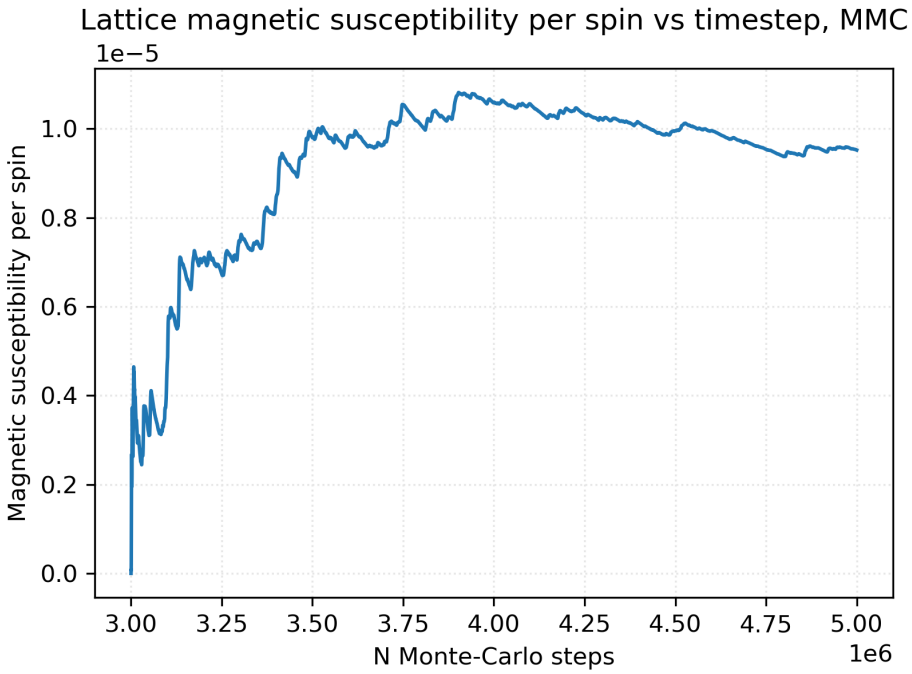


**Figure 104:** Magnetic susceptibility per spin vs timestep ($B = -10$ T)

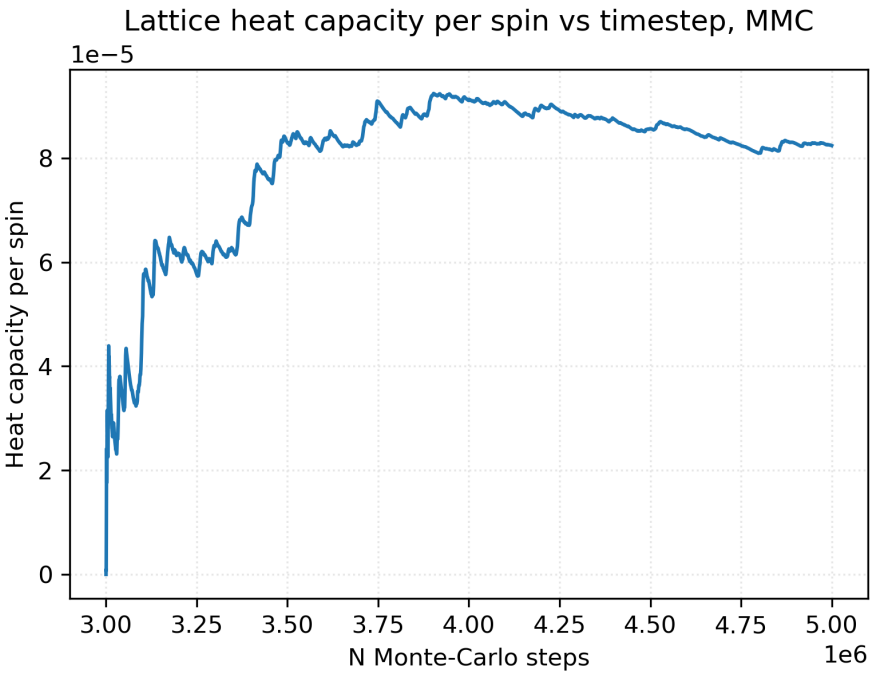


**Figure 105:** Lattice heat capacity per spin vs timestep ($B = -10$ T)

# 9 Wolff Algorithm: Increasing the Lattice Size at $T_{crit}$

## 9.1 $16 \times 16$ Lattice

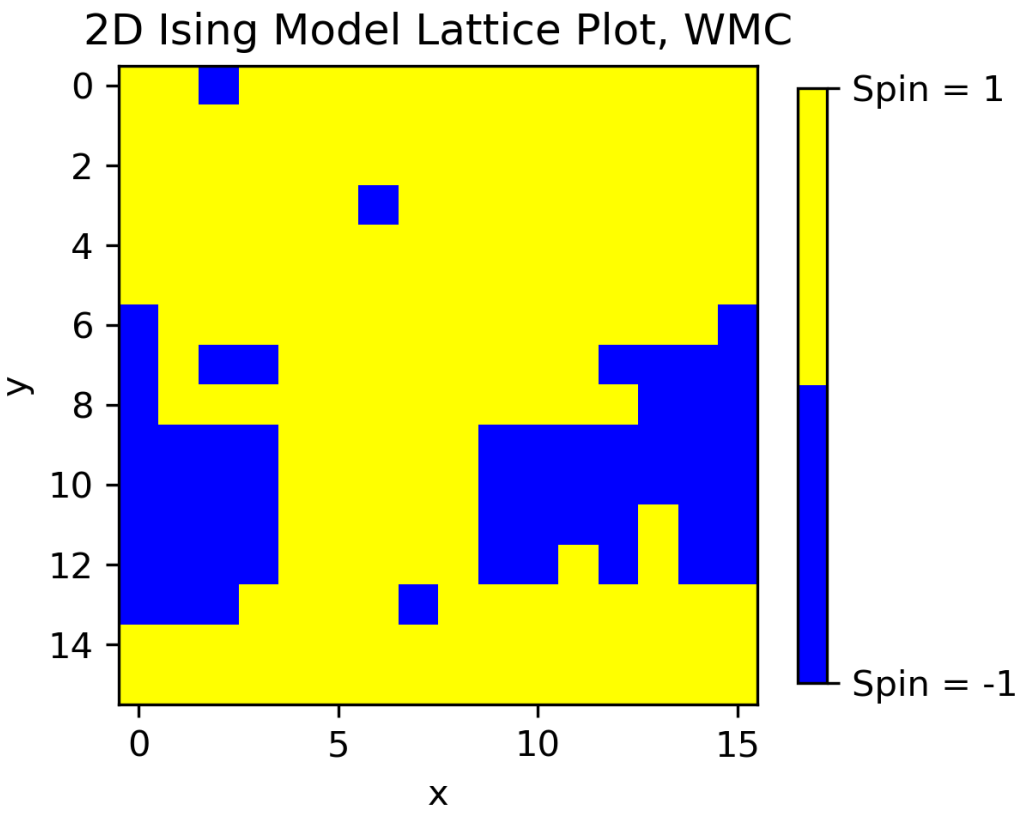


**Figure 106:** End lattice configuration ($16 \times 16$ Lattice)

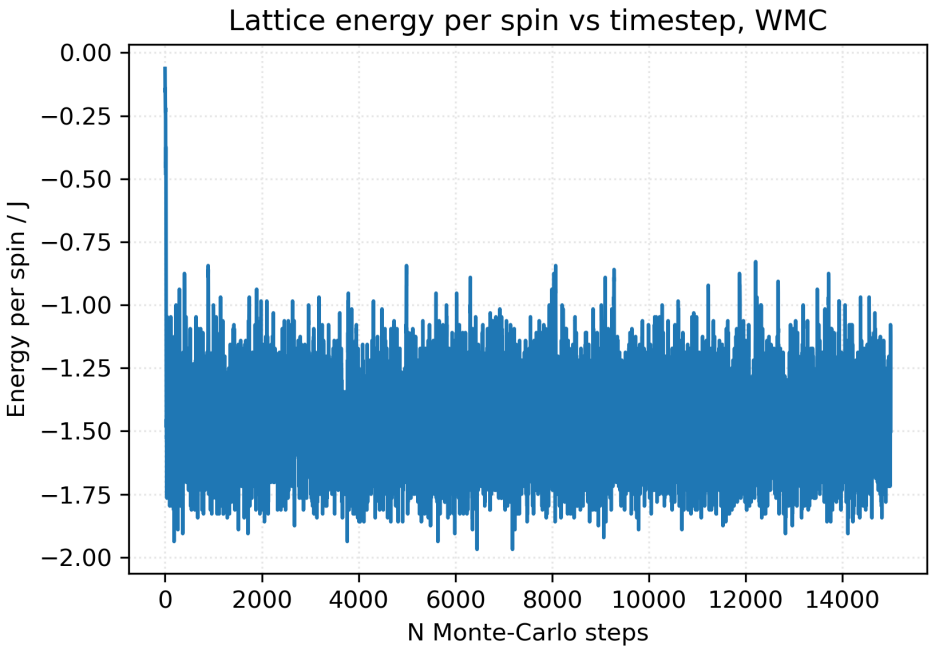


**Figure 107:** Lattice energy per spin vs timestep ($16 \times 16$ Lattice)

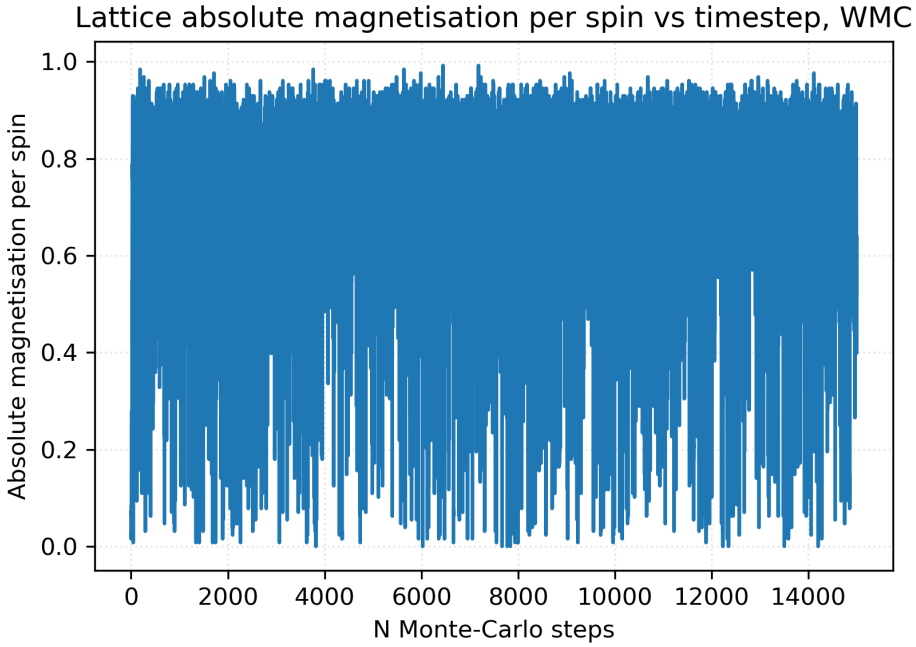


**Figure 108:** Lattice magnetisation per spin vs timestep ($16 \times 16$ Lattice)

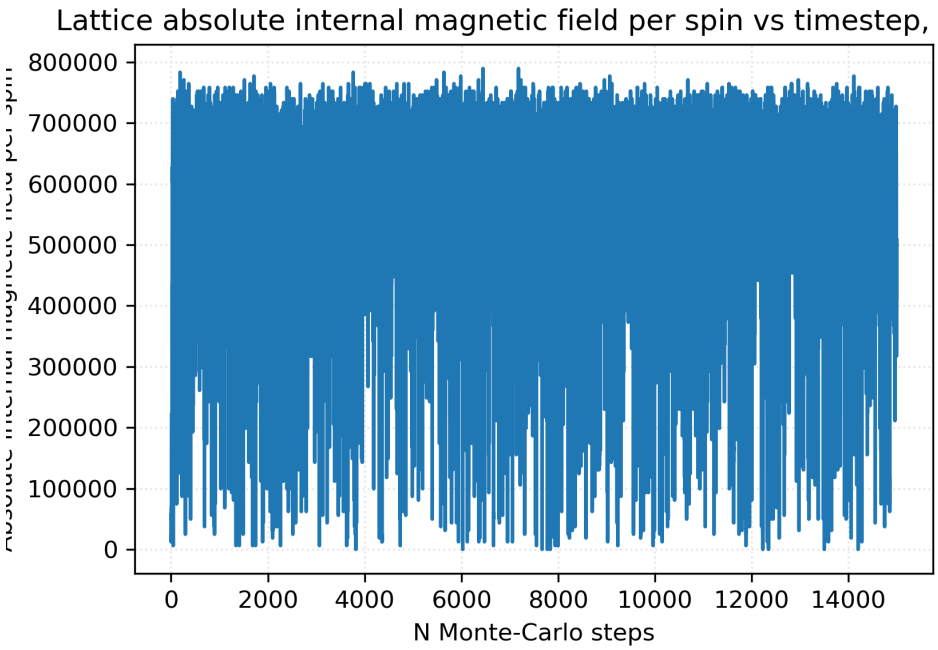


**Figure 109:** Internal magnetic field per spin vs timestep ($16 \times 16$ Lattice)

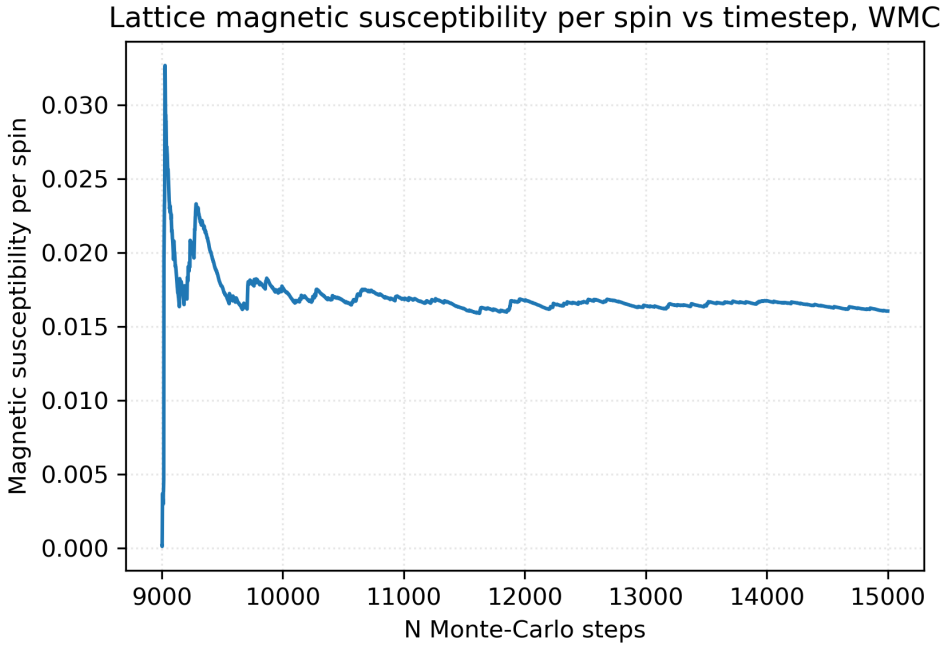


**Figure 110:** Magnetic susceptibility per spin vs timestep ($16 \times 16$ Lattice)

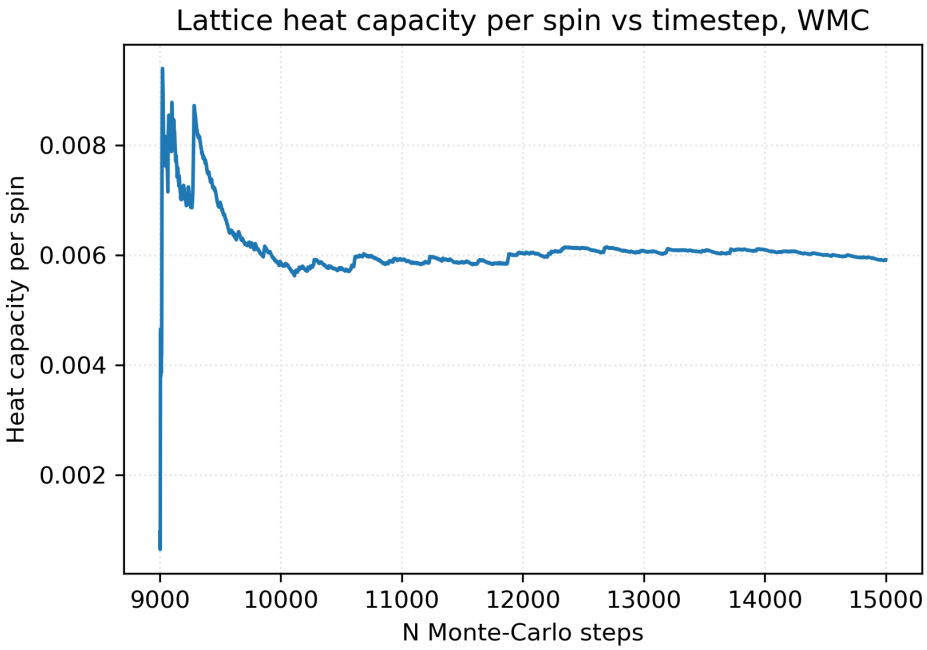


**Figure 111:** Lattice heat capacity per spin vs timestep ($16 \times 16$ Lattice)

## 9.2 $32 \times 32$ Lattice

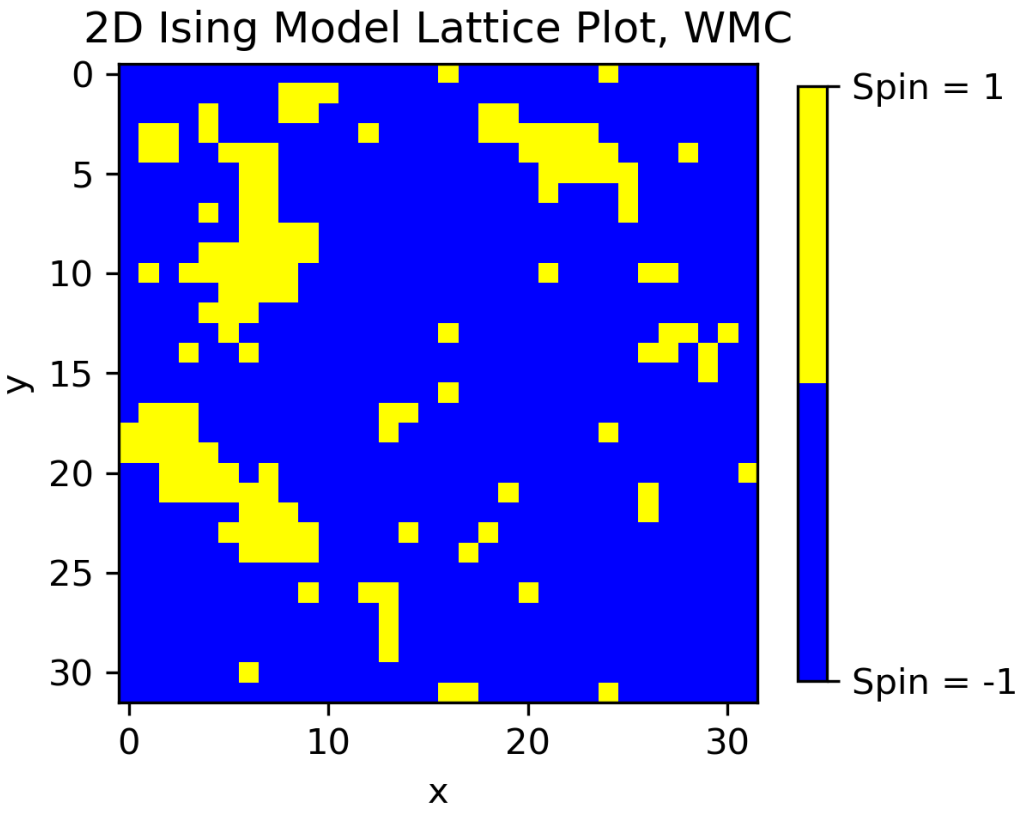


**Figure 112:** End lattice configuration ($32 \times 32$ Lattice)

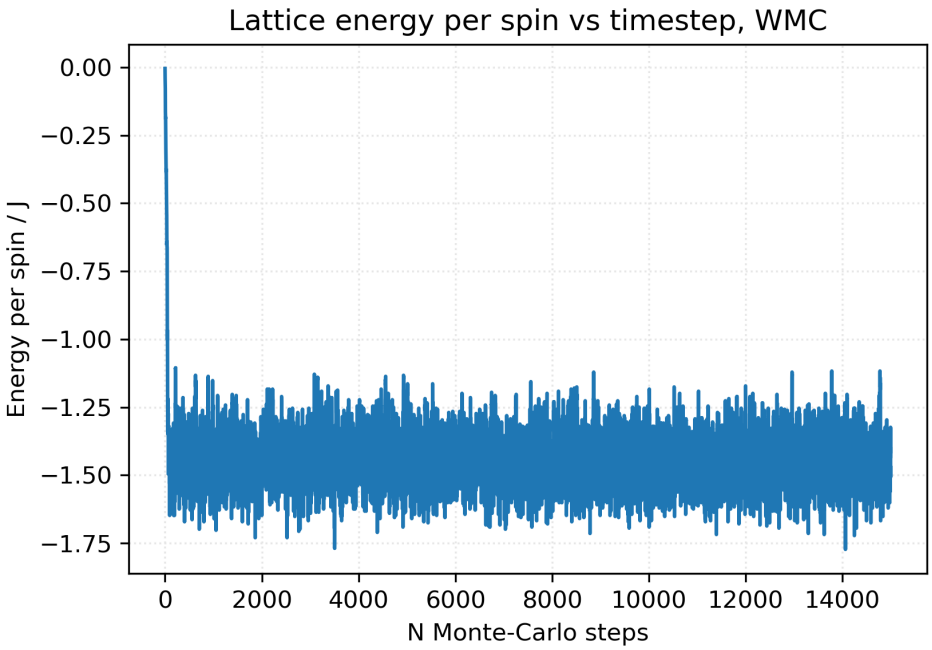


**Figure 113:** Lattice energy per spin vs timestep ($32 \times 32$ Lattice)

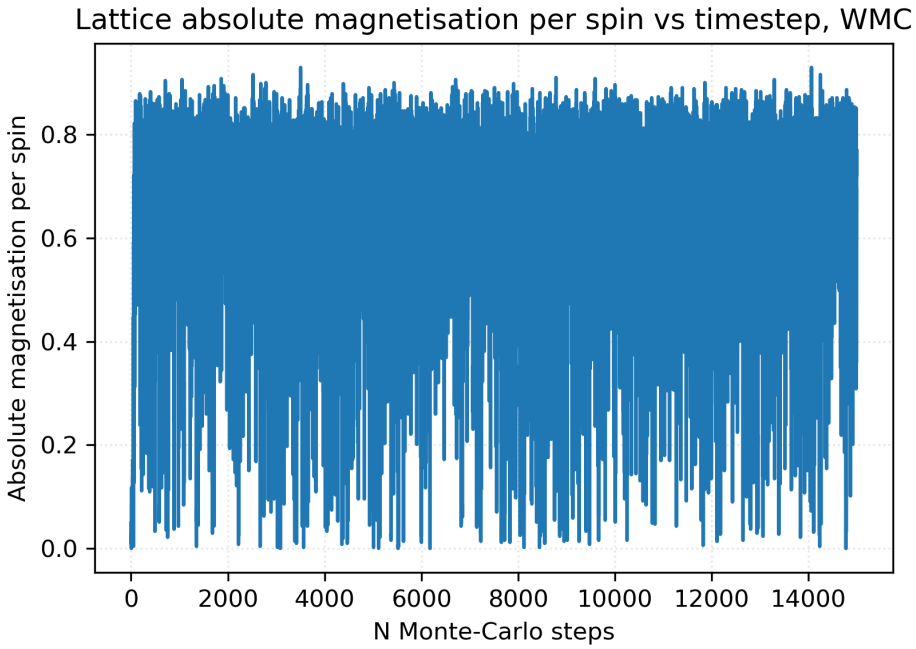


**Figure 114:** Lattice magnetisation per spin vs timestep ($32 \times 32$ Lattice)

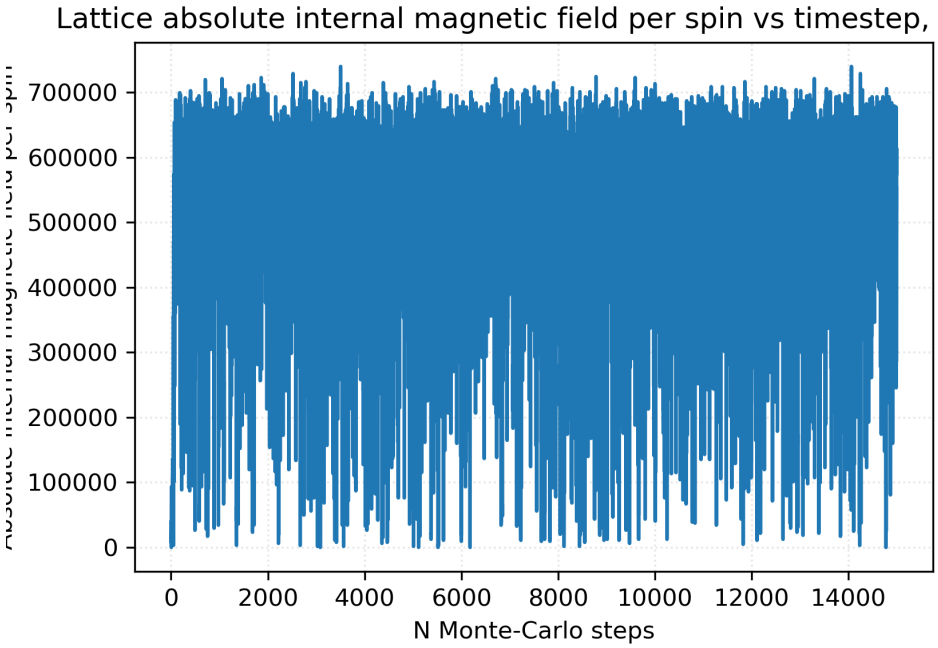


**Figure 115:** Internal magnetic field per spin vs timestep ($32 \times 32$ Lattice)

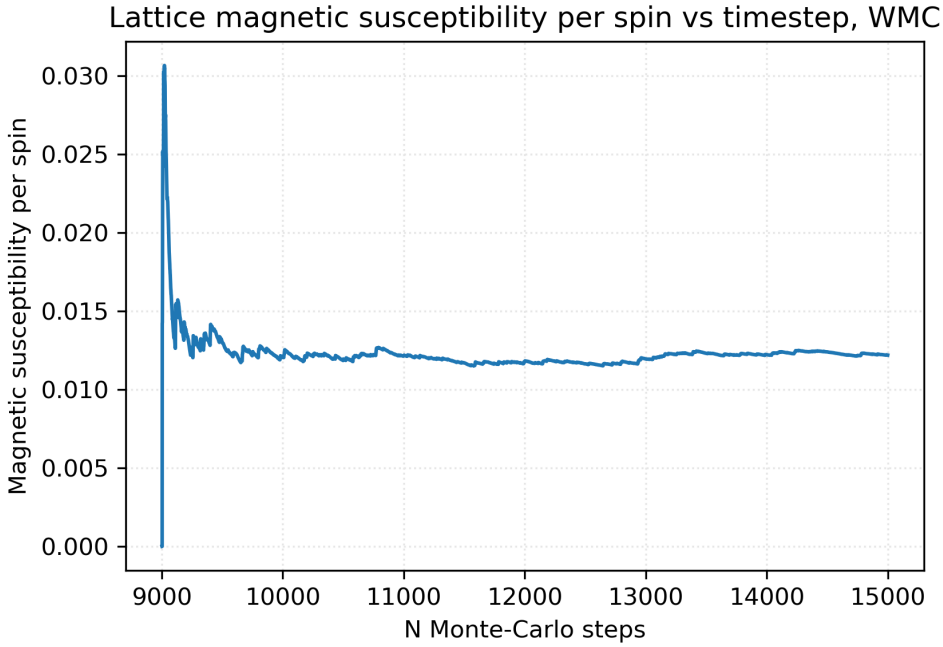


**Figure 116:** Magnetic susceptibility per spin vs timestep ($32 \times 32$ Lattice)

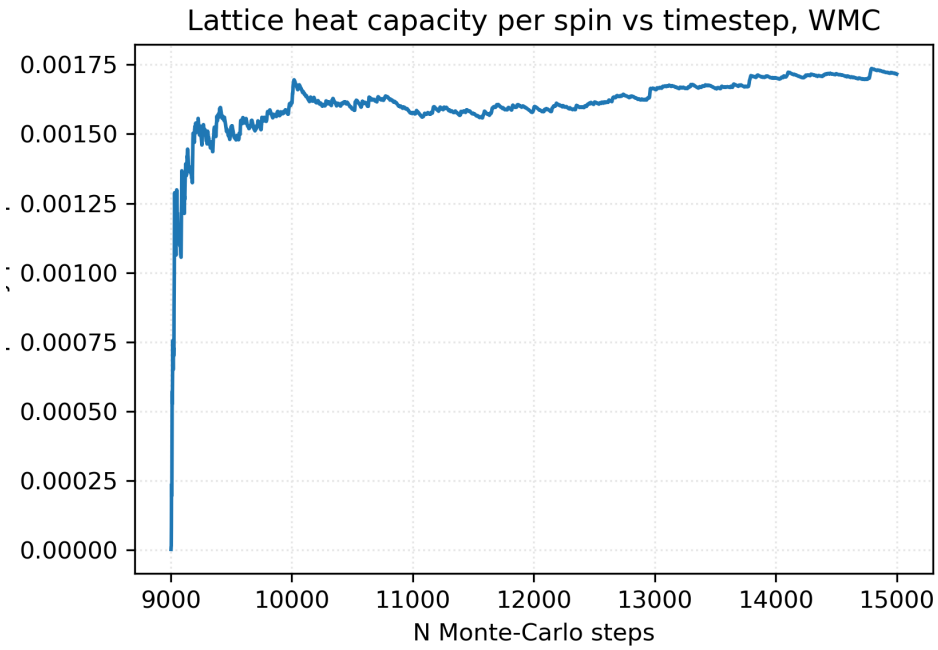


**Figure 117:** Lattice heat capacity per spin vs timestep ($32 \times 32$ Lattice)

## 9.3 $64 \times 64$ Lattice

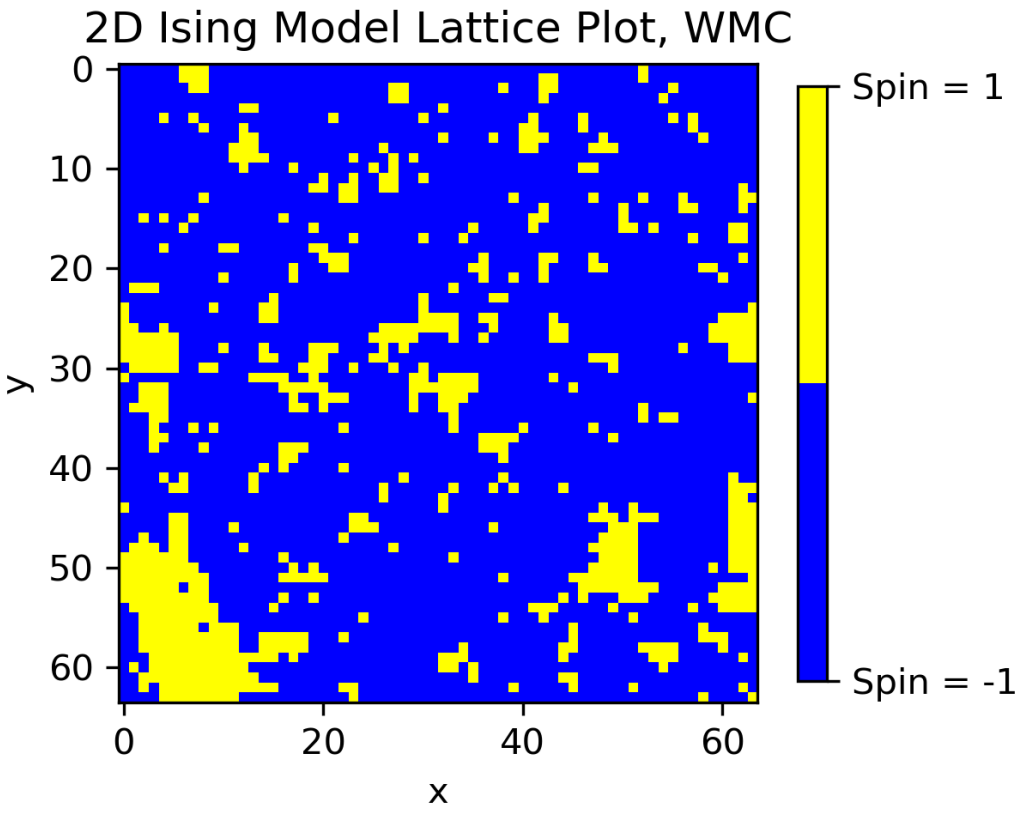


**Figure 118:** End lattice configuration ($64 \times 64$ Lattice)

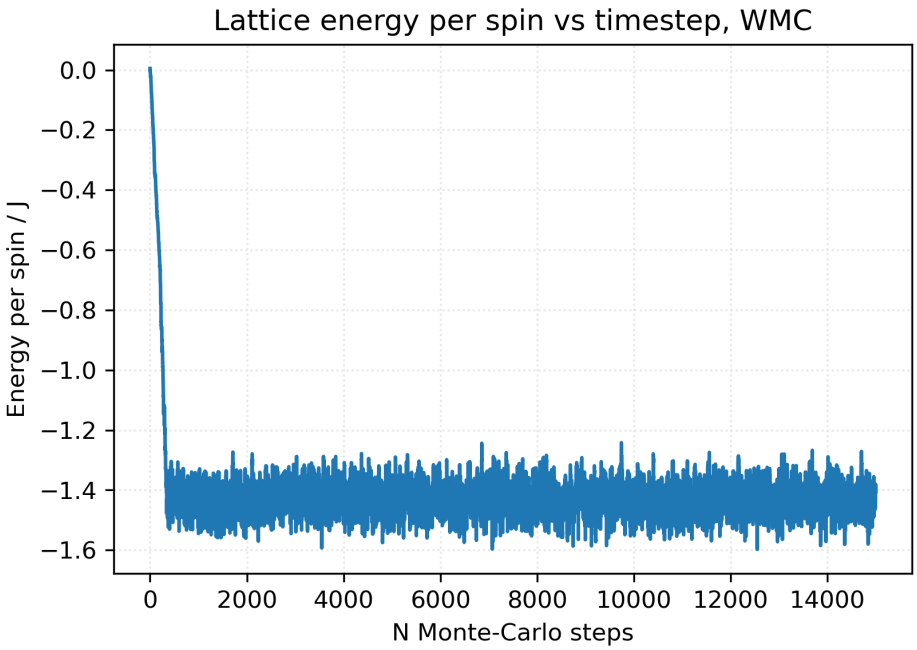


**Figure 119:** Lattice energy per spin vs timestep ($64 \times 64$ Lattice)

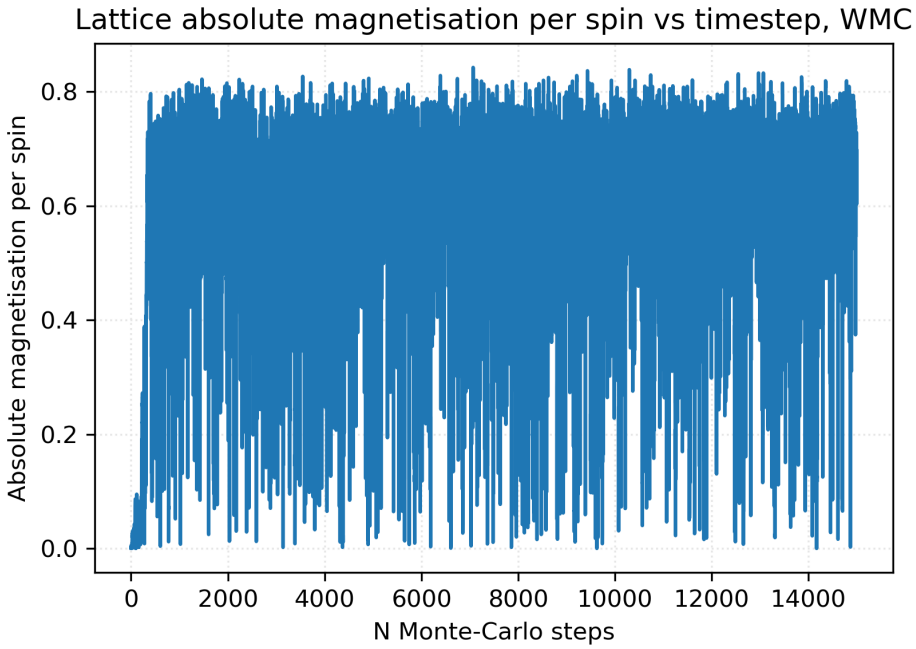


**Figure 120:** Lattice magnetisation per spin vs timestep ($64 \times 64$ Lattice)

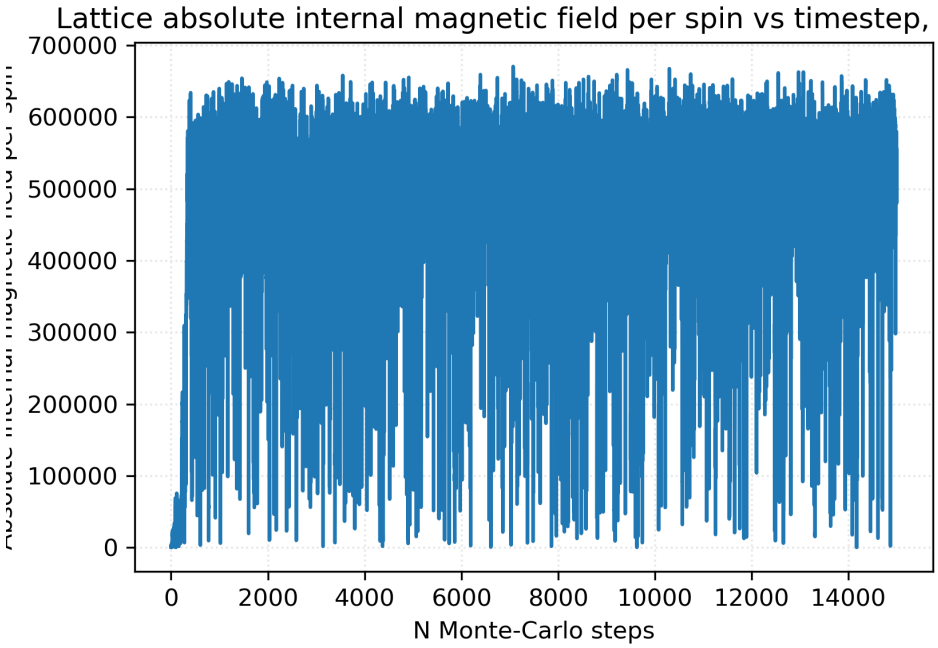


**Figure 121:** Internal magnetic field per spin vs timestep ($64 \times 64$ Lattice)

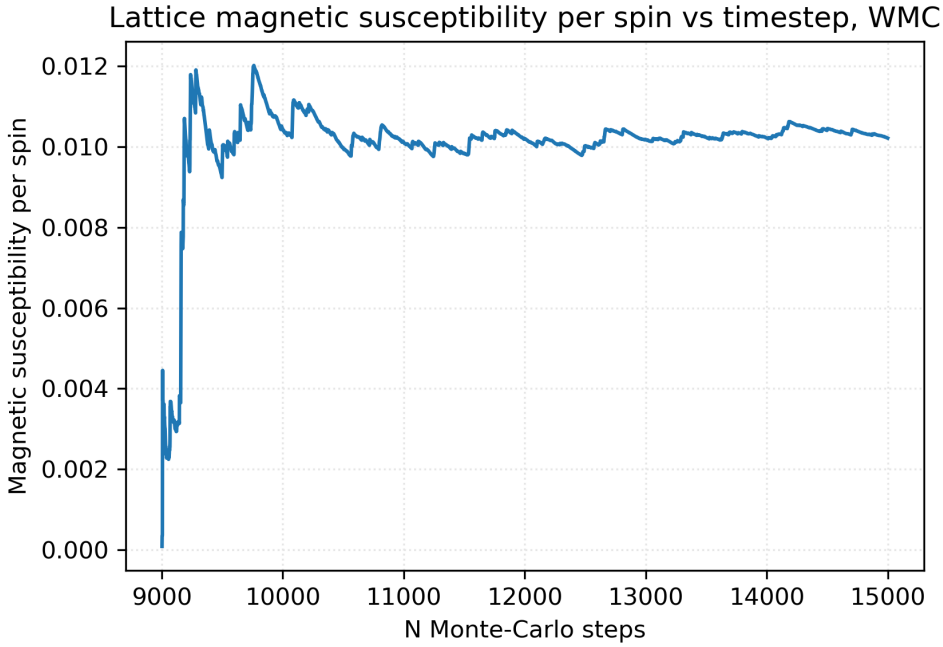


**Figure 122:** Magnetic susceptibility per spin vs timestep ($64 \times 64$ Lattice)

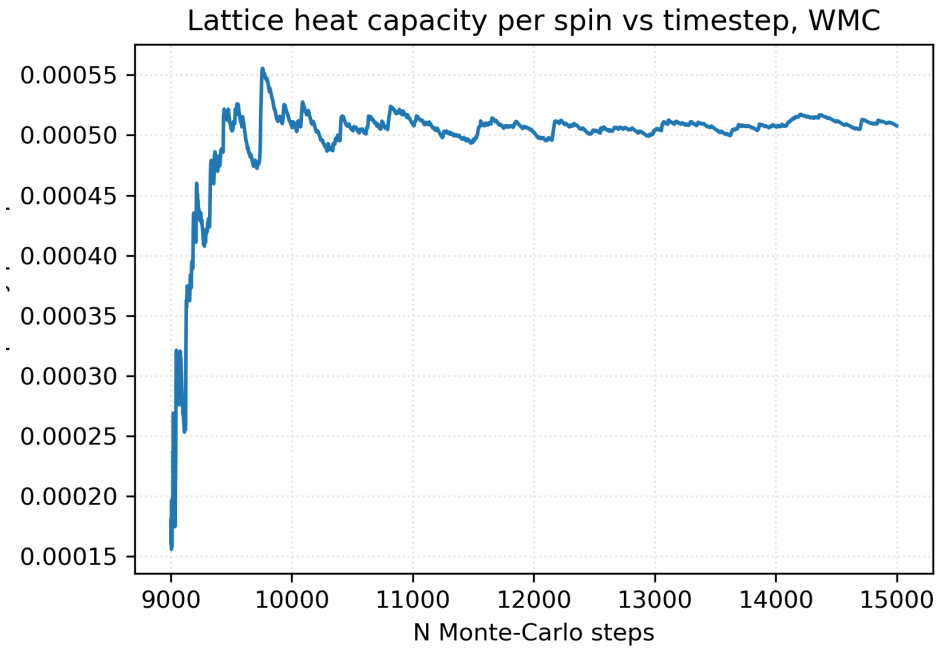


**Figure 123:** Lattice heat capacity per spin vs timestep ($64 \times 64$ Lattice)

## 9.4 $128 \times 128$ Lattice

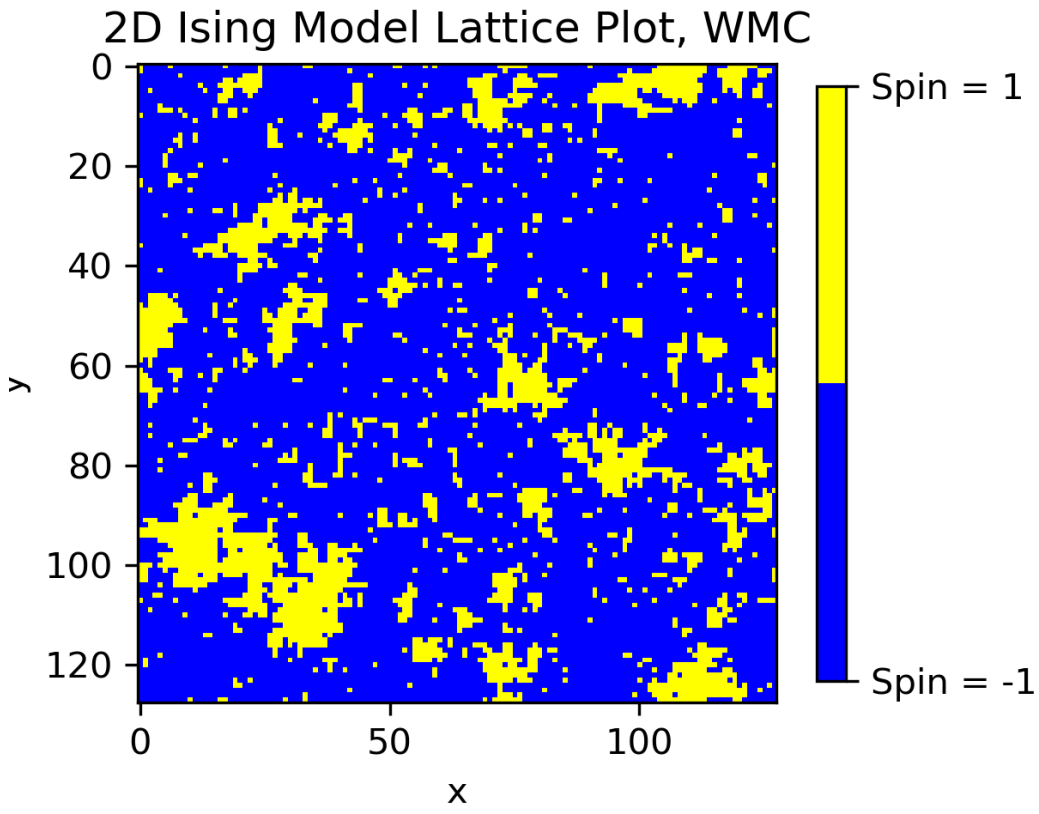


**Figure 124:** End lattice configuration ($128 \times 128$ Lattice)

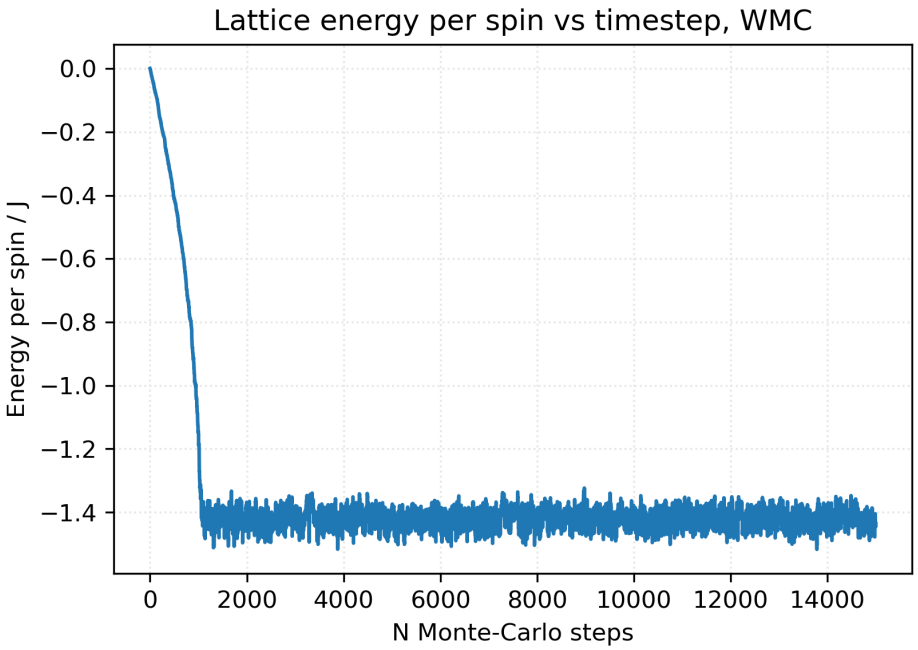


**Figure 125:** Lattice energy per spin vs timestep (128 × 128 Lattice)

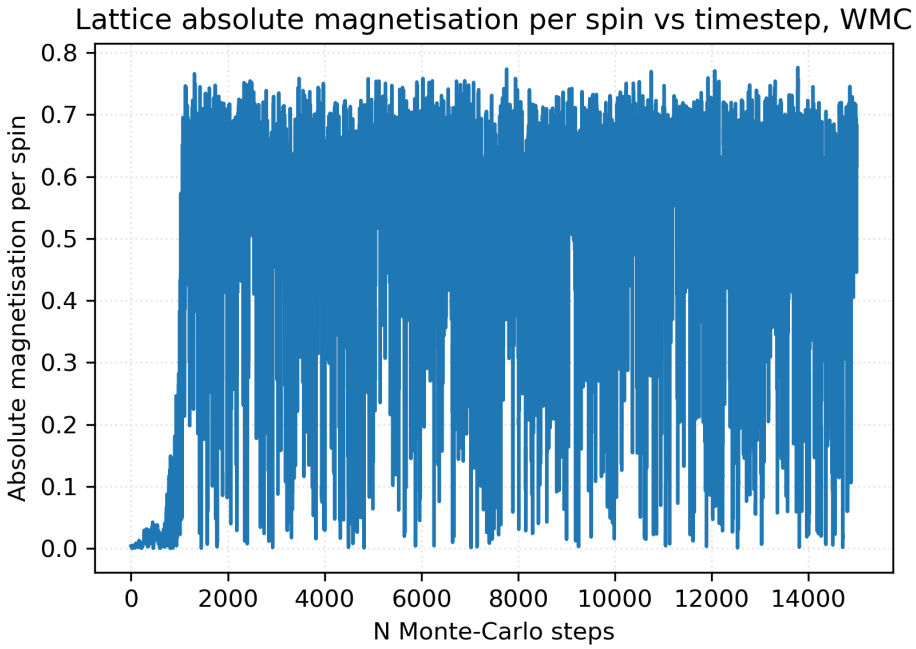


**Figure 126:** Lattice magnetisation per spin vs timestep (128 × 128 Lattice)

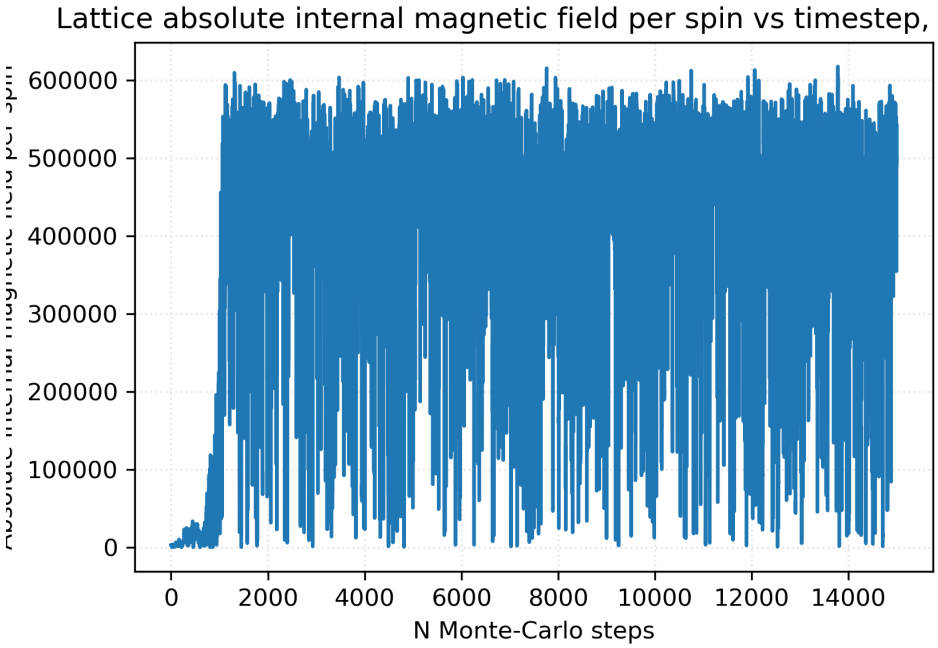


**Figure 127:** Internal magnetic field per spin vs timestep ($128 \times 128$ Lattice)

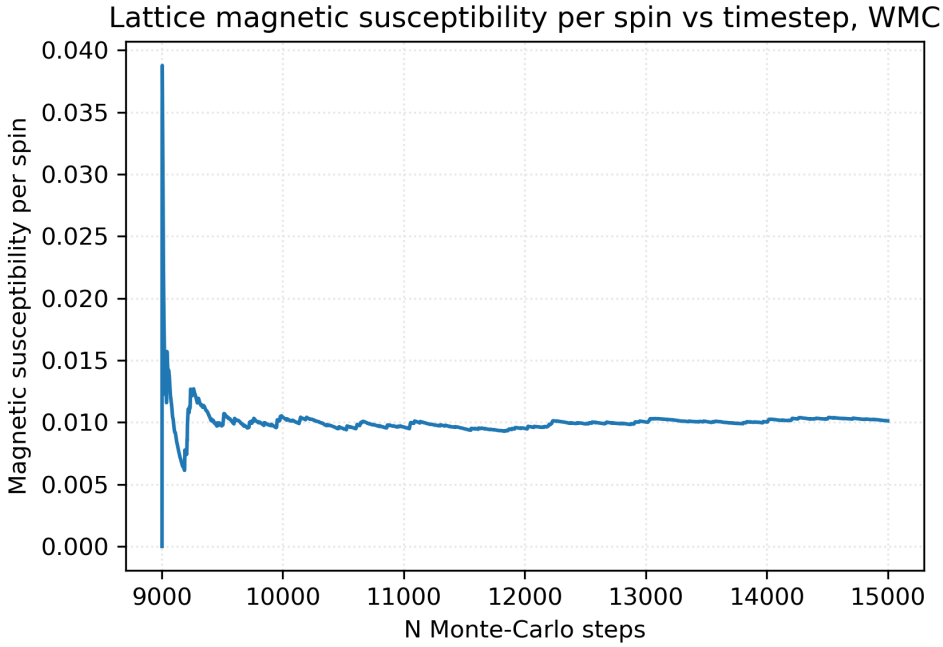


**Figure 128:** Magnetic susceptibility per spin vs timestep ($128 \times 128$ Lattice)

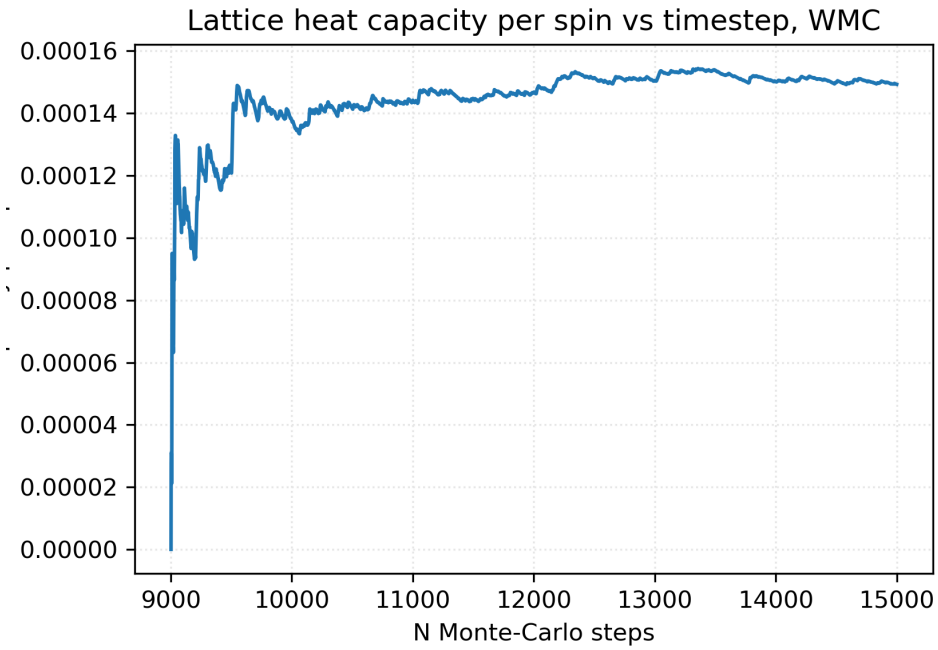


**Figure 129:** Lattice heat capacity per spin vs timestep ($128 \times 128$ Lattice)

# 10 Wolff Algorithm: Temperatures near absolute zero, below, at and above $T_{crit}$.

## 10.1 $T = 0.001$ K

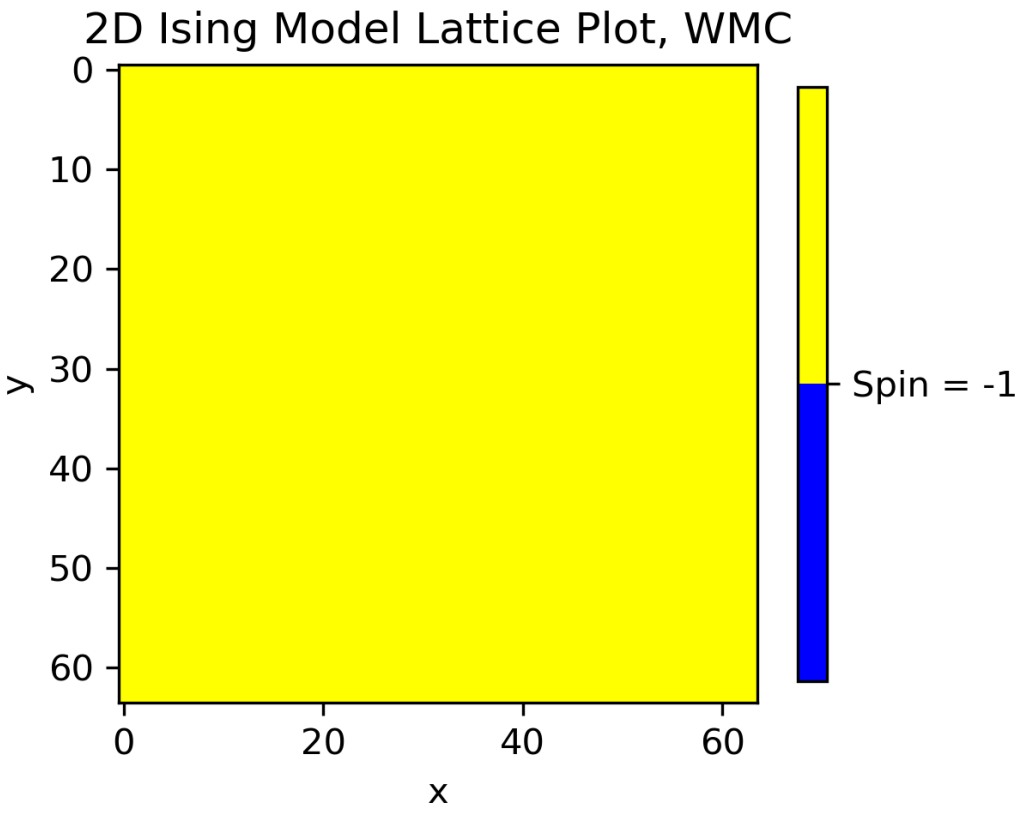


**Figure 130:** End lattice configuration ($T = 0.001$ K)

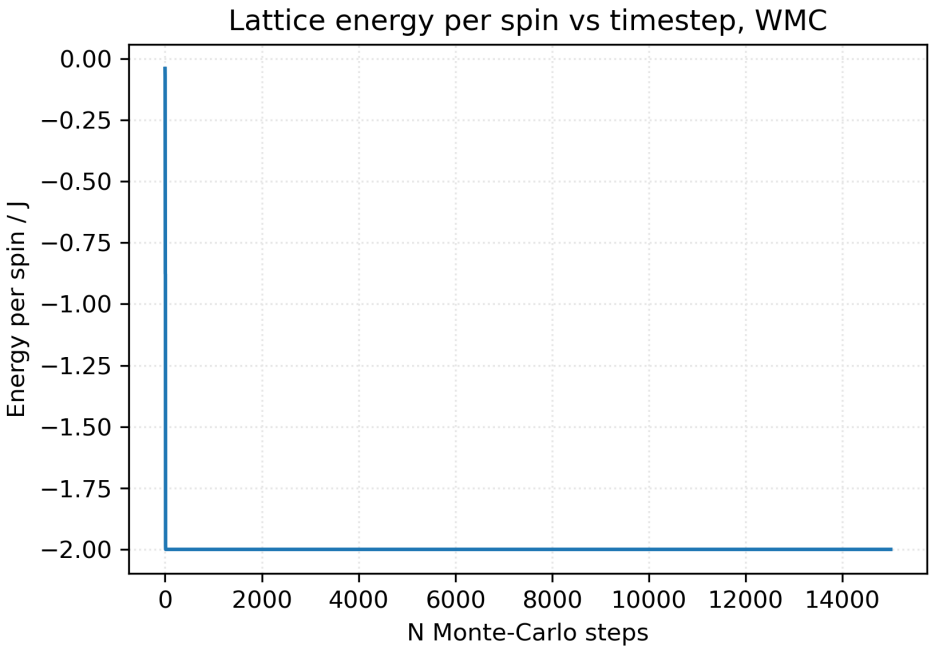


**Figure 131:** Lattice energy per spin vs timestep ($T = 0.001$ K)

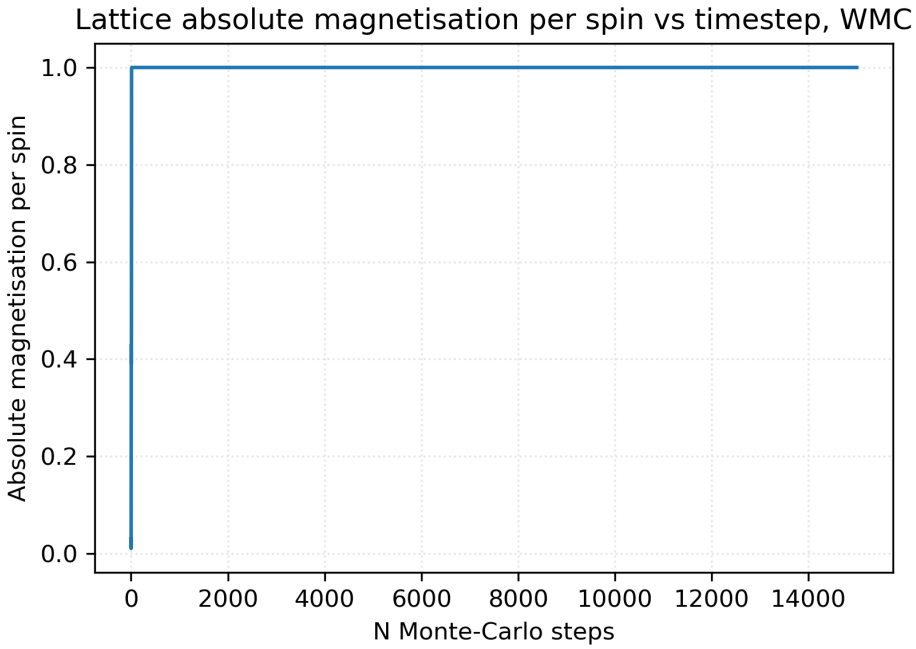


**Figure 132:** Lattice magnetisation per spin vs timestep ($T = 0.001$ K)

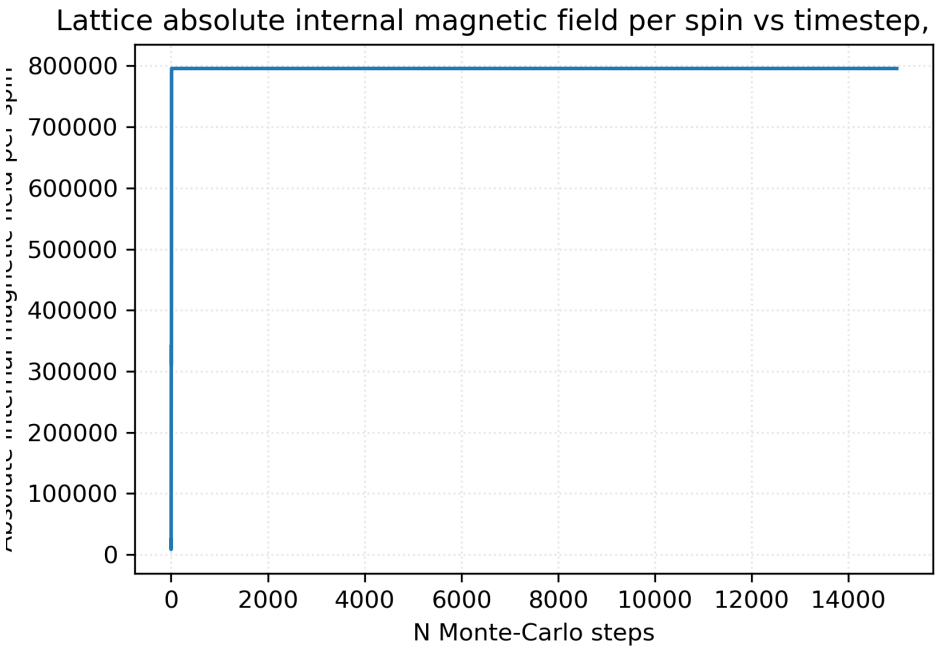


**Figure 133:** Internal magnetic field per spin vs timestep ($T$ = 0.001 K)

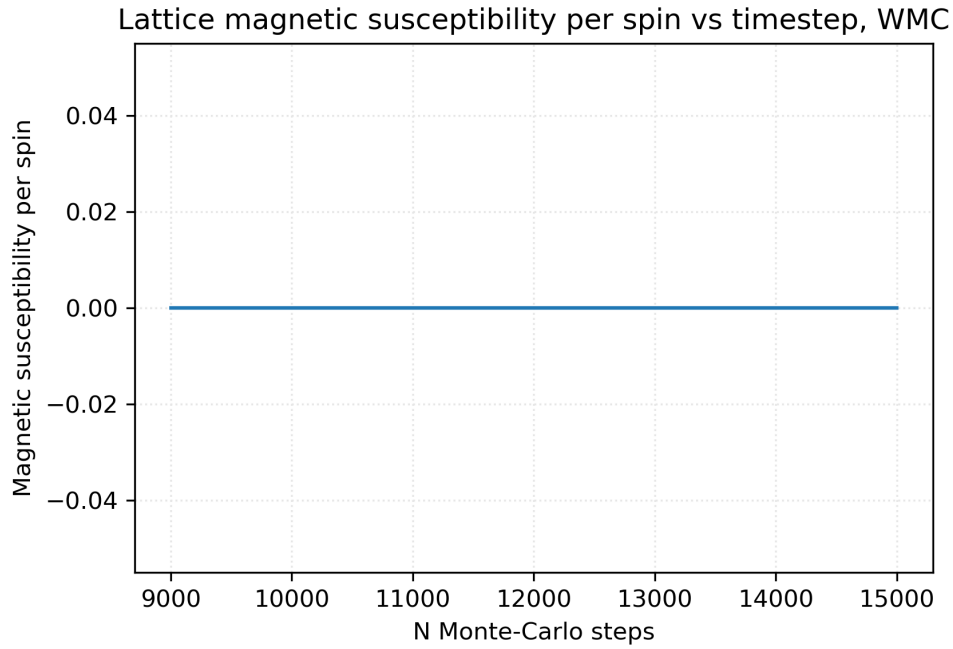


**Figure 134:** Magnetic susceptibility per spin vs timestep ($T$ = 0.001 K)

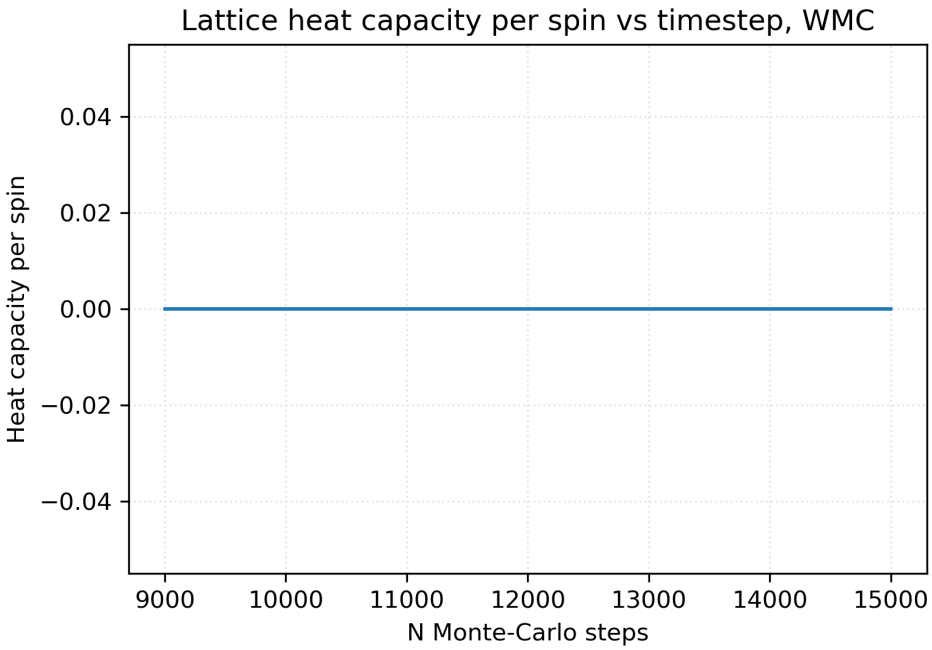


**Figure 135:** Lattice heat capacity per spin vs timestep ($T = 0.001$ K)

## 10.2 $T = 1$ K

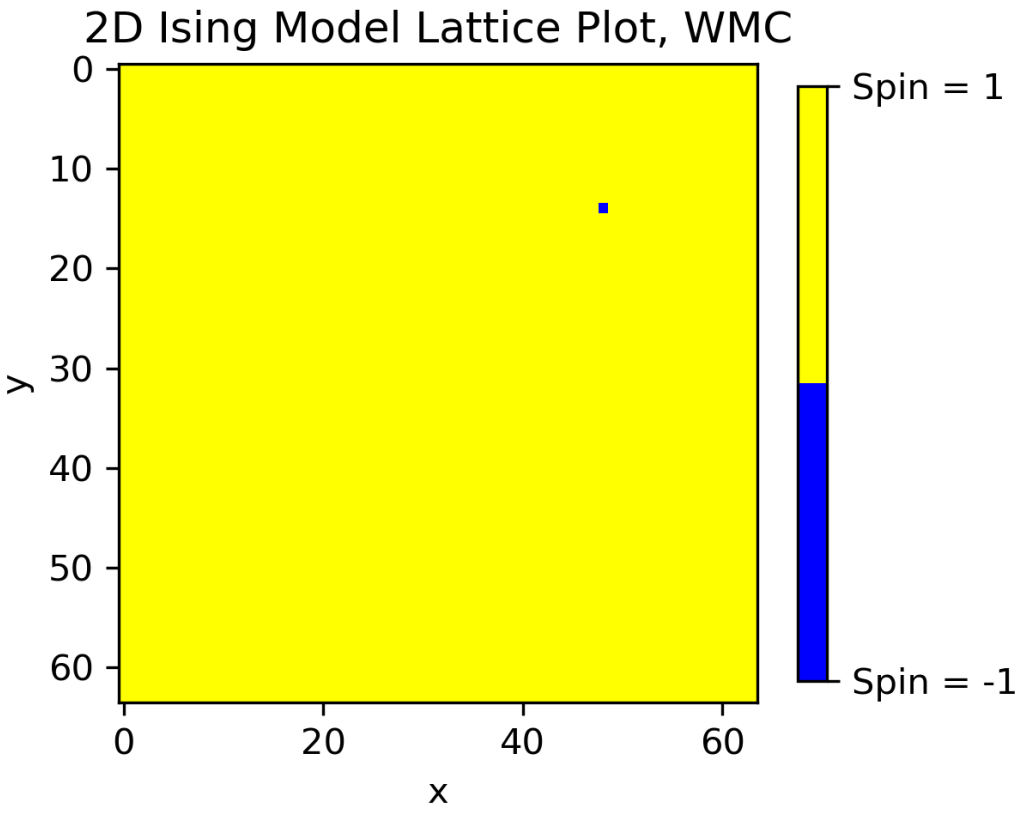


**Figure 136:** End lattice configuration ($T = 1$ K)

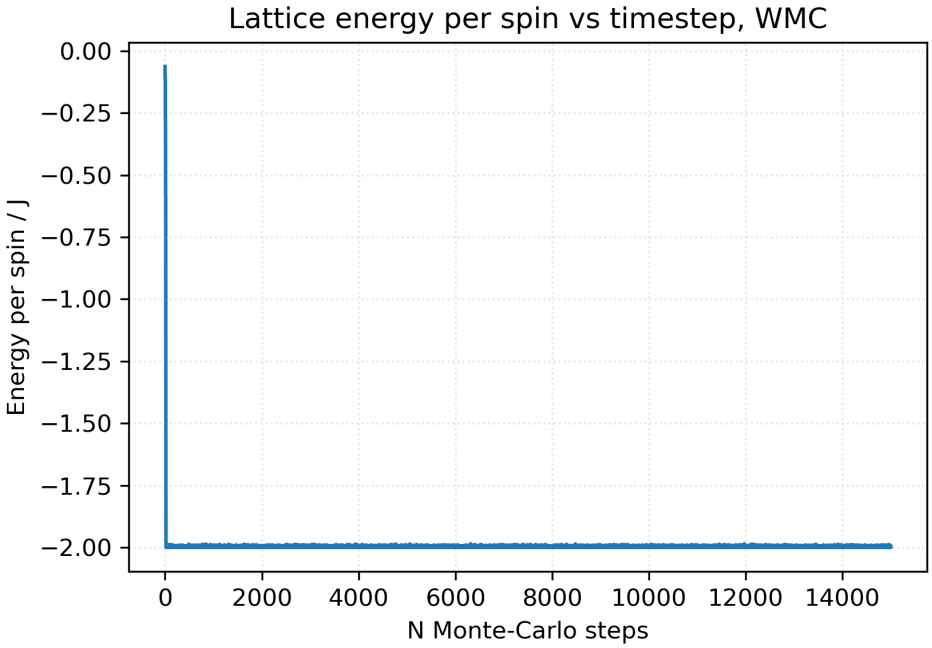


**Figure 137:** Lattice energy per spin vs timestep ($T = 1$ K)

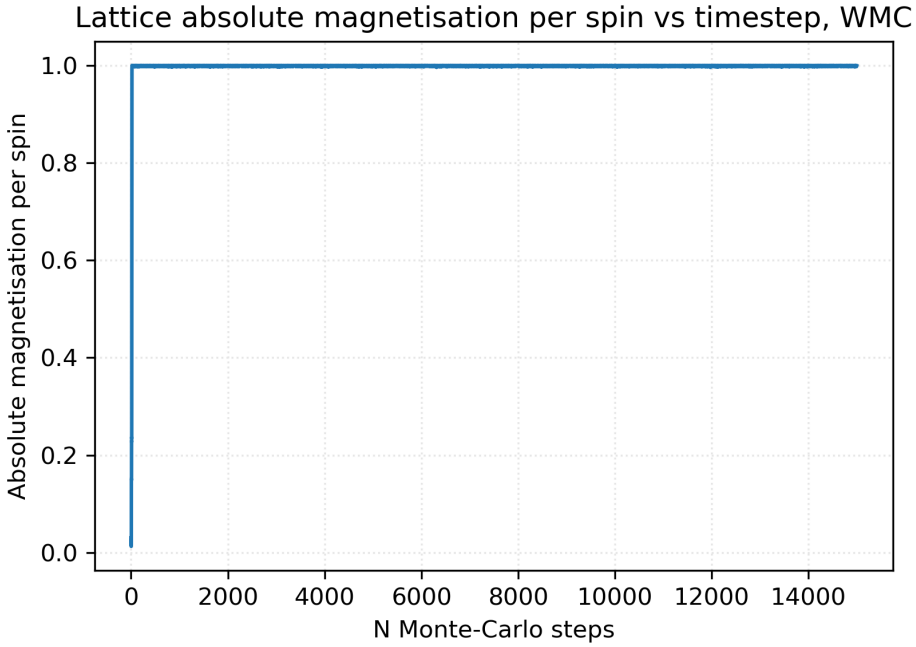


**Figure 138:** Lattice magnetisation per spin vs timestep ($T = 1$ K)

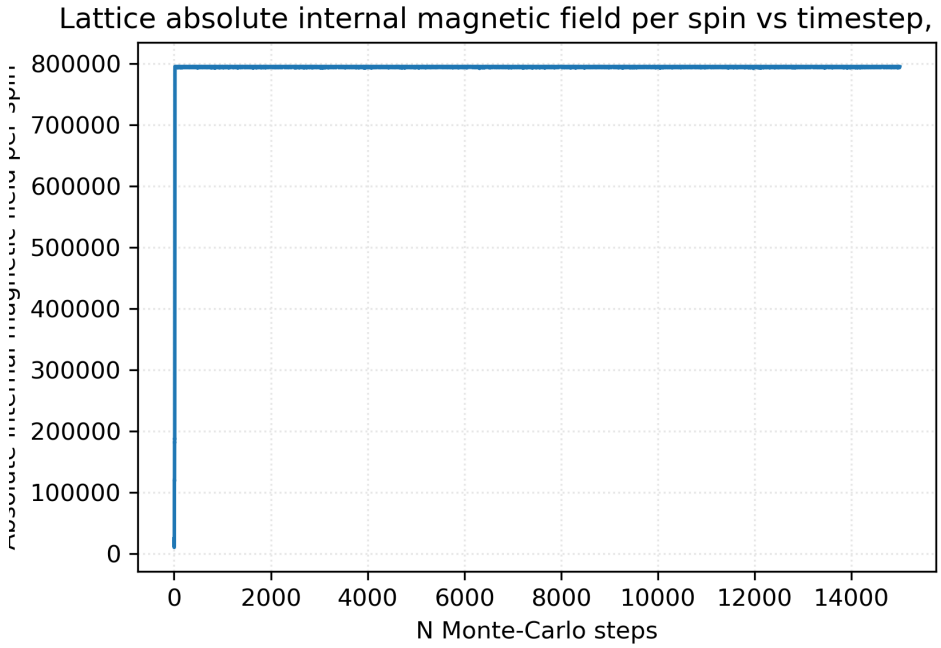


**Figure 139:** Internal magnetic field per spin vs timestep ($T$ = 1 K)

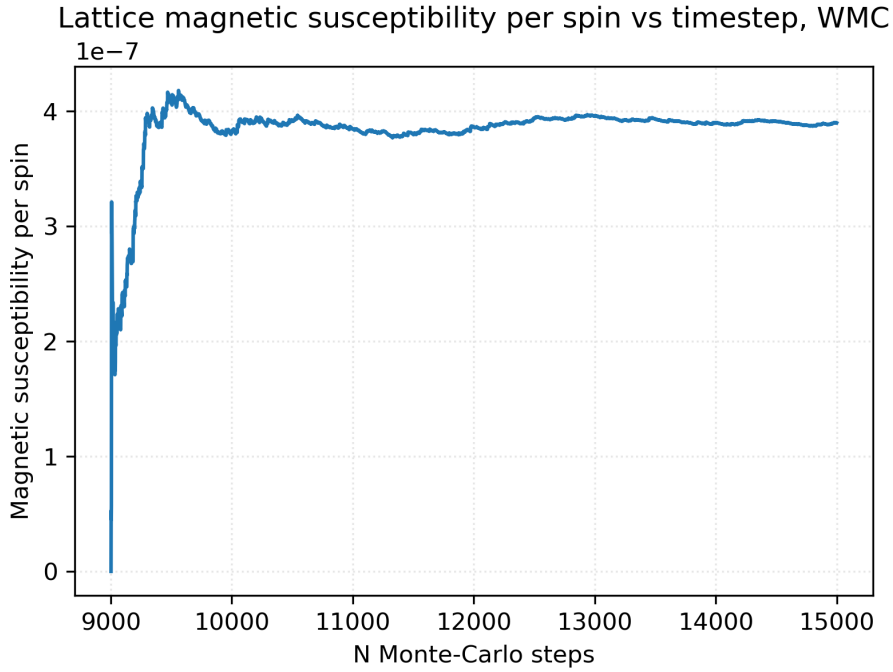


**Figure 140:** Magnetic susceptibility per spin vs timestep ($T$ = 1 K)

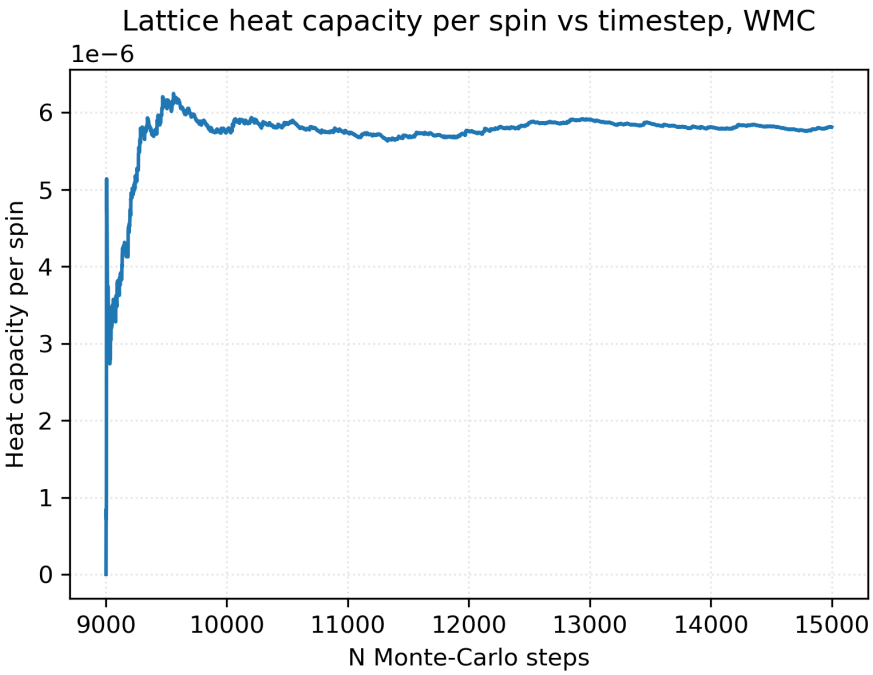


**Figure 141:** Lattice heat capacity per spin vs timestep ($T$ = 1 K)

### 10.3 $T = T_c$

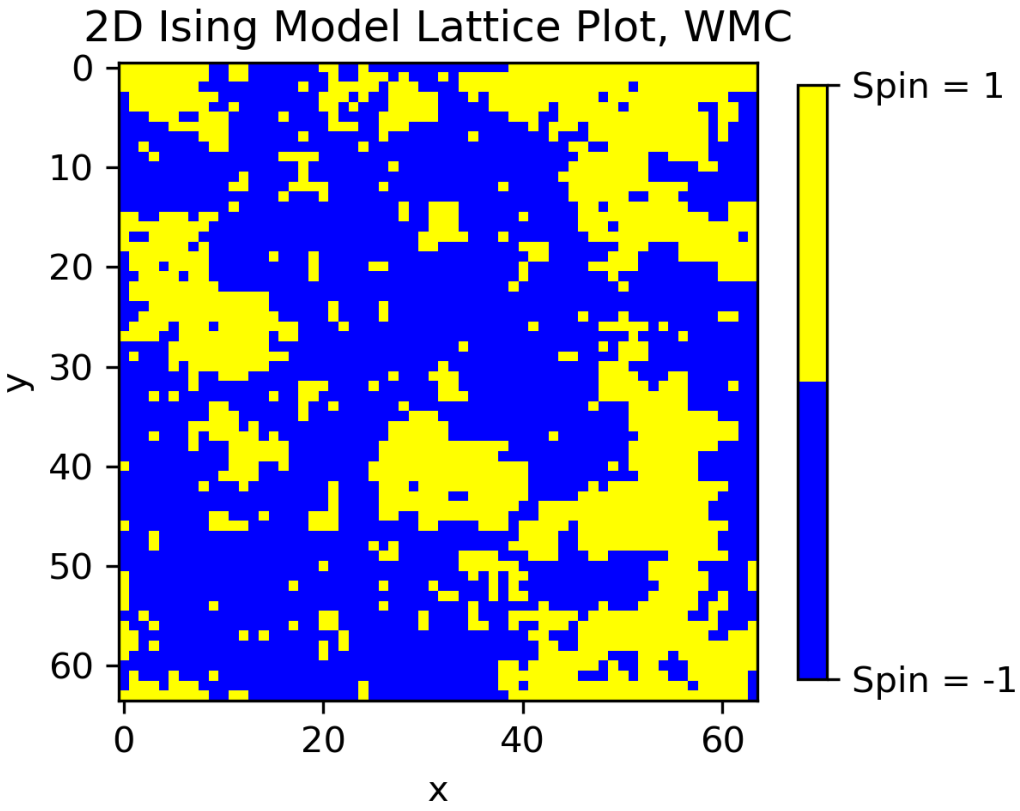


**Figure 142:** End lattice configuration ($T = T_c$)

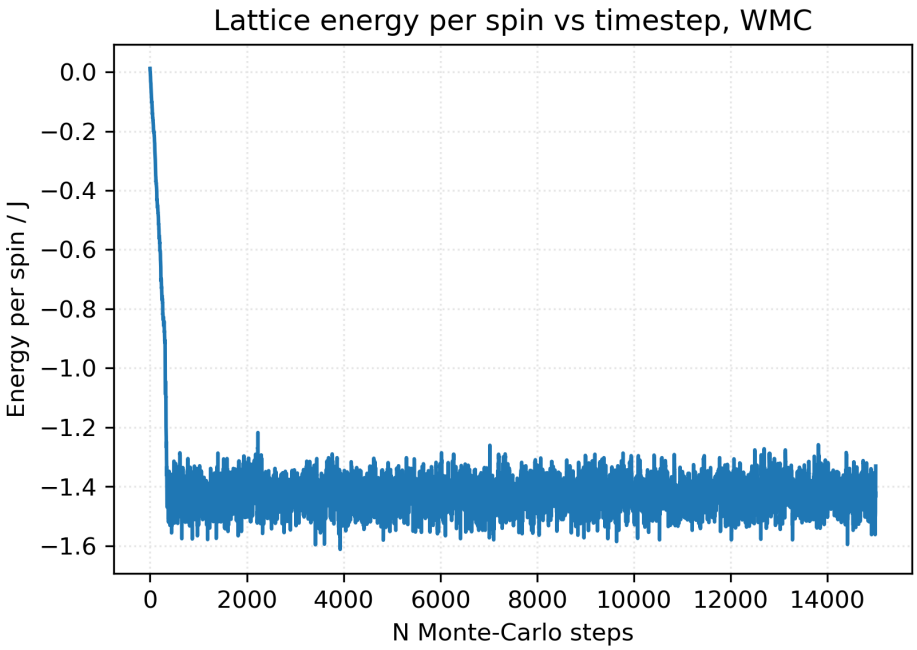


**Figure 143:** Lattice energy per spin vs timestep ($T = T_c$)

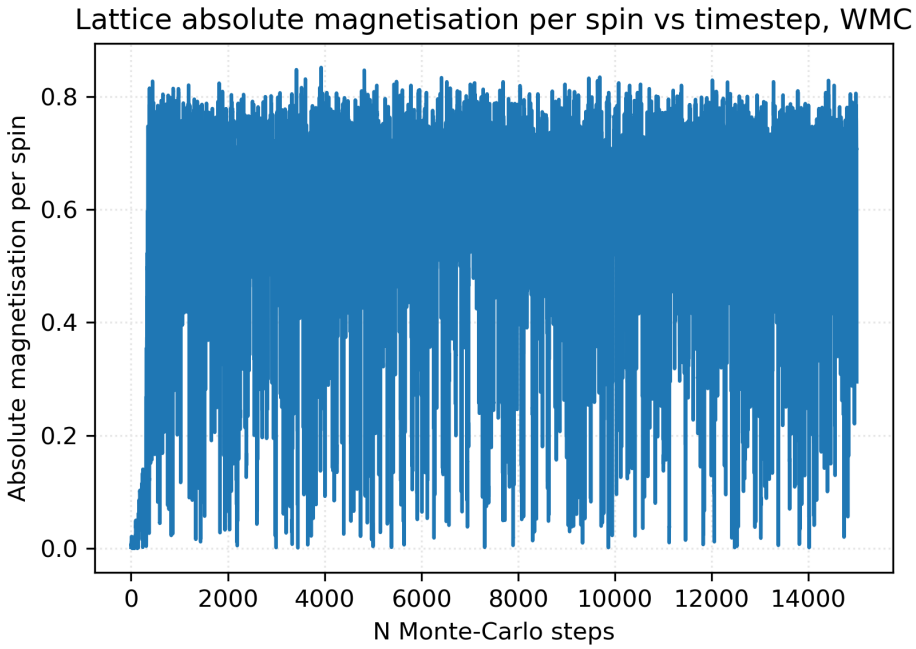


**Figure 144:** Lattice magnetisation per spin vs timestep ($T = T_c$)

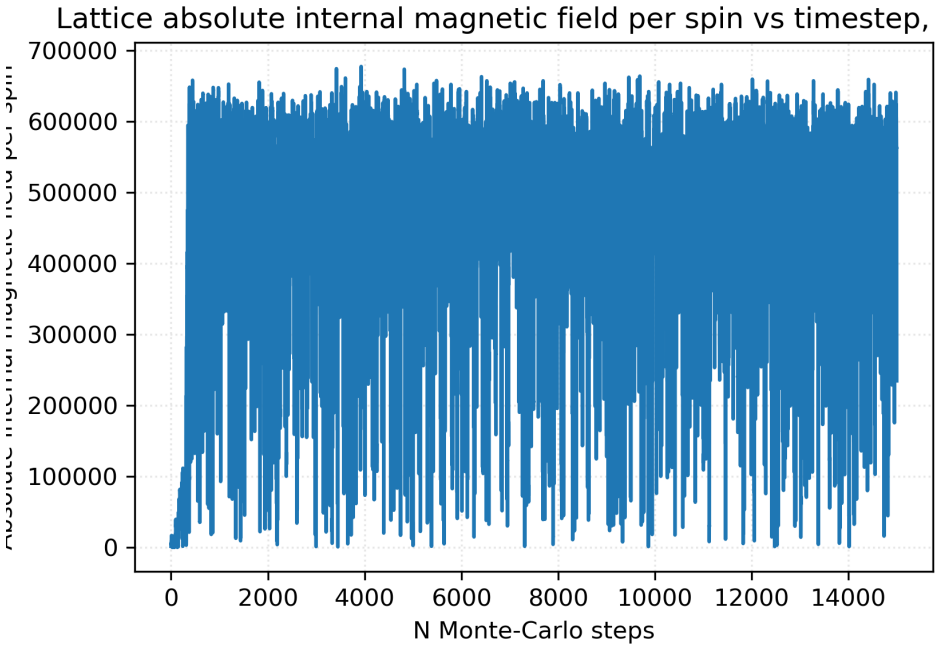


**Figure 145:** Internal magnetic field per spin vs timestep ($T = T_c$)

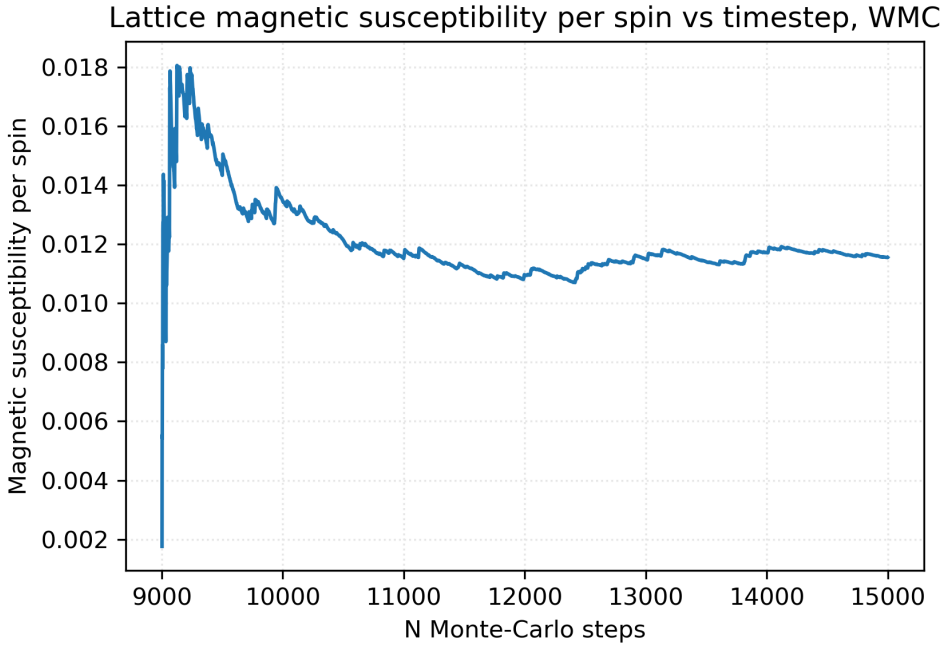


**Figure 146:** Magnetic susceptibility per spin vs timestep ($T = T_c$)

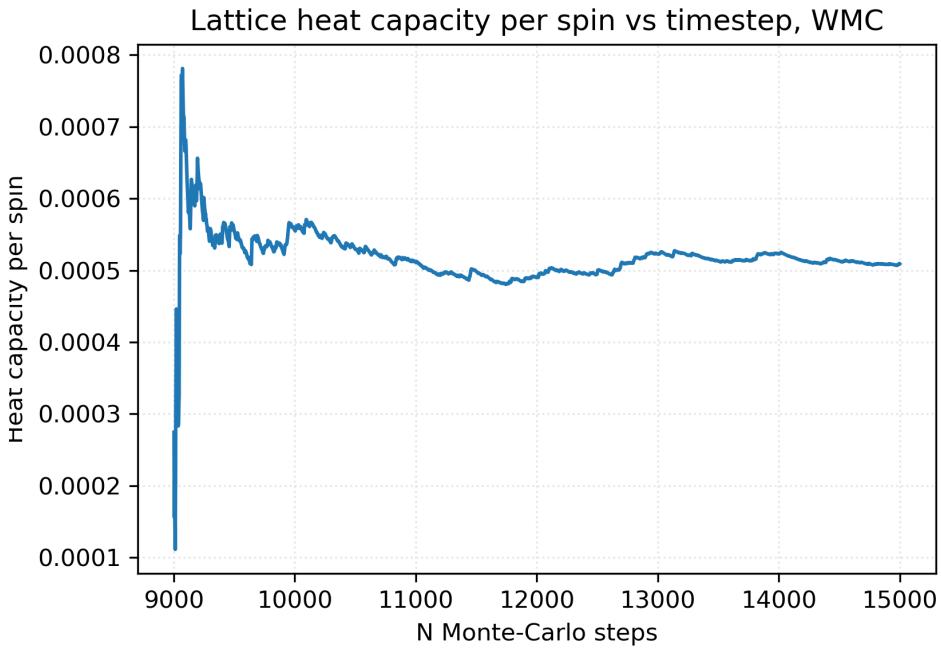


**Figure 147:** Lattice heat capacity per spin vs timestep ($T = T_c$)

### 10.4 $T = 3$ K

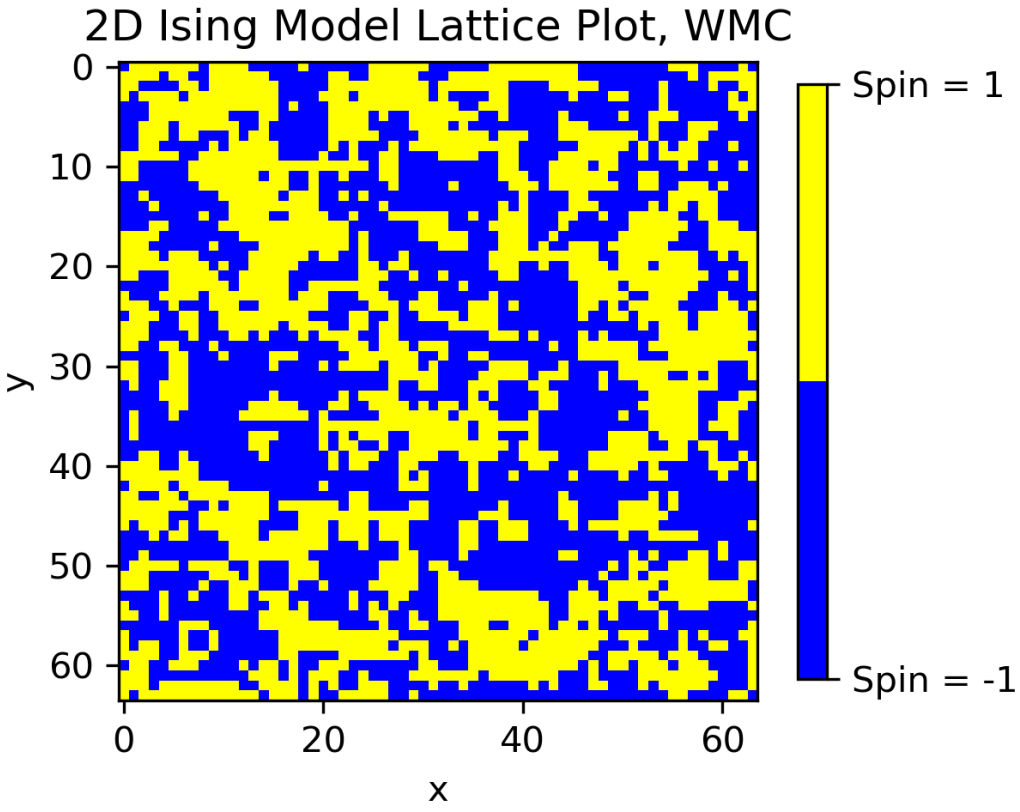


**Figure 148:** End lattice configuration ($T = 3$ K)

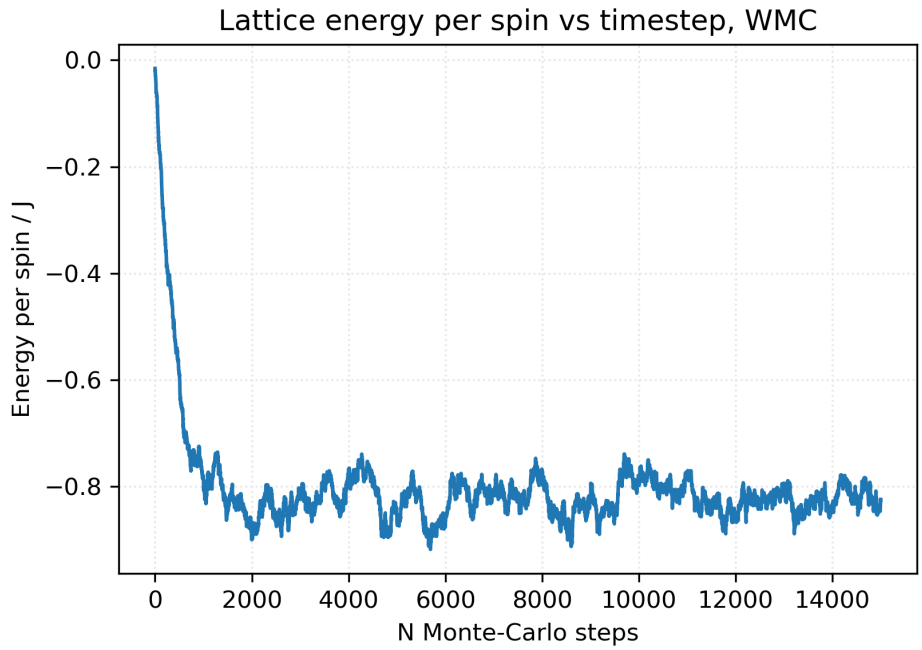


**Figure 149:** Lattice energy per spin vs timestep ($T$ = 3 K)

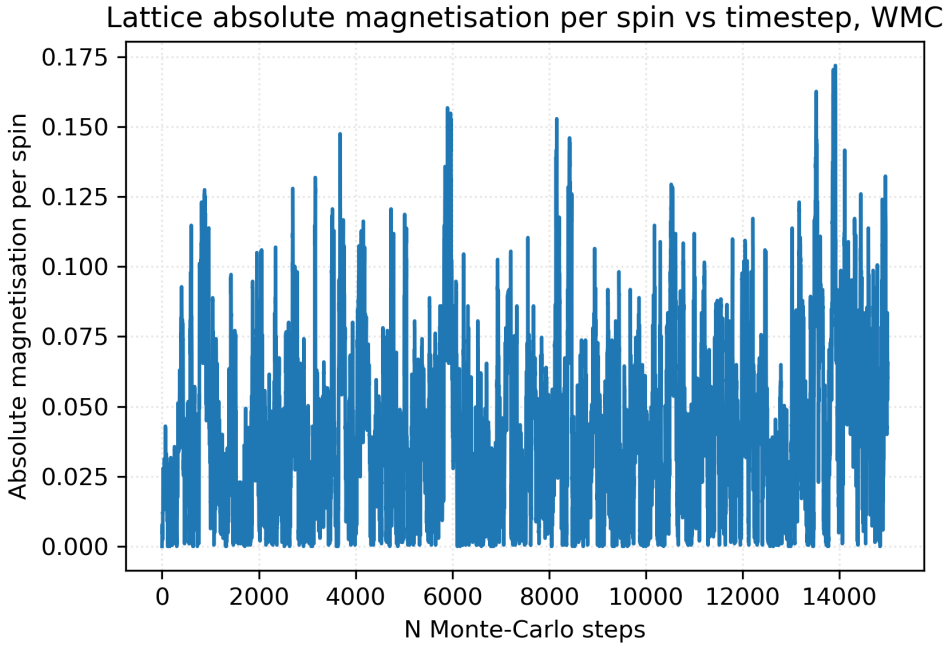


**Figure 150:** Lattice magnetisation per spin vs timestep ($T$ = 3 K)

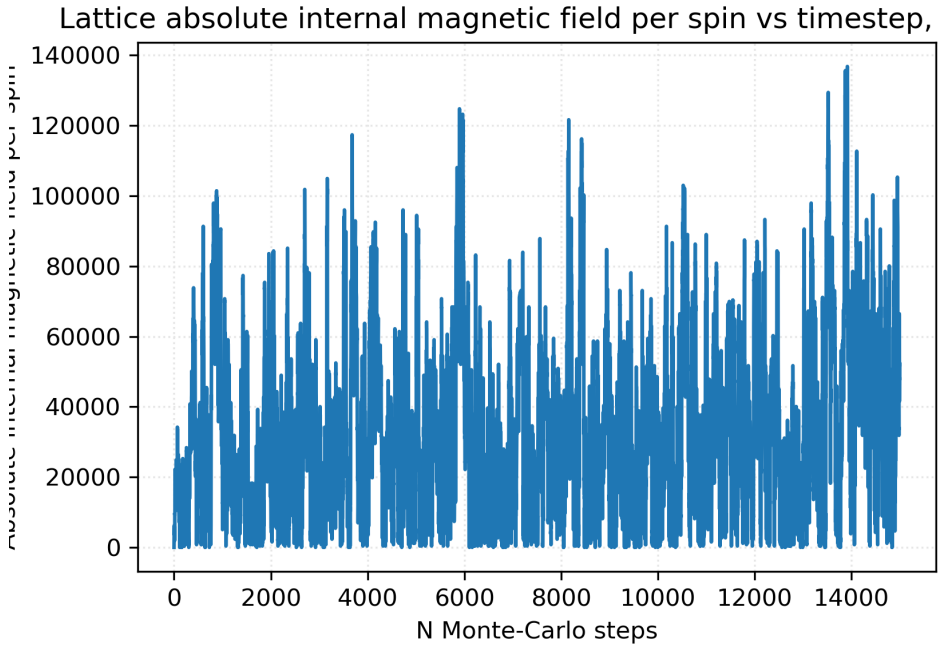


**Figure 151:** Internal magnetic field per spin vs timestep ($T = 3$ K)

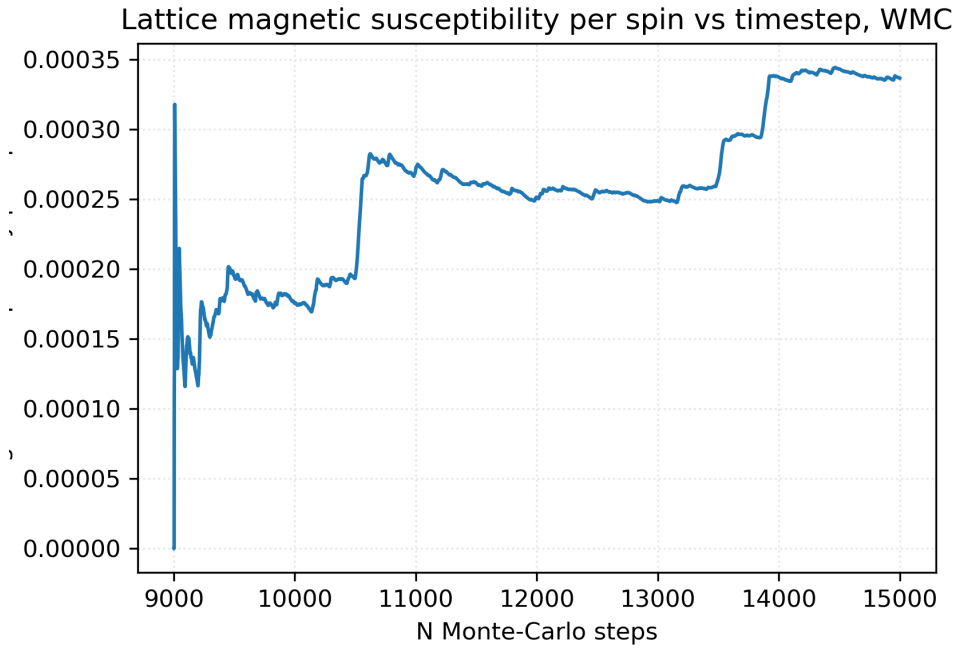


**Figure 152:** Magnetic susceptibility per spin vs timestep ($T = 3$ K)

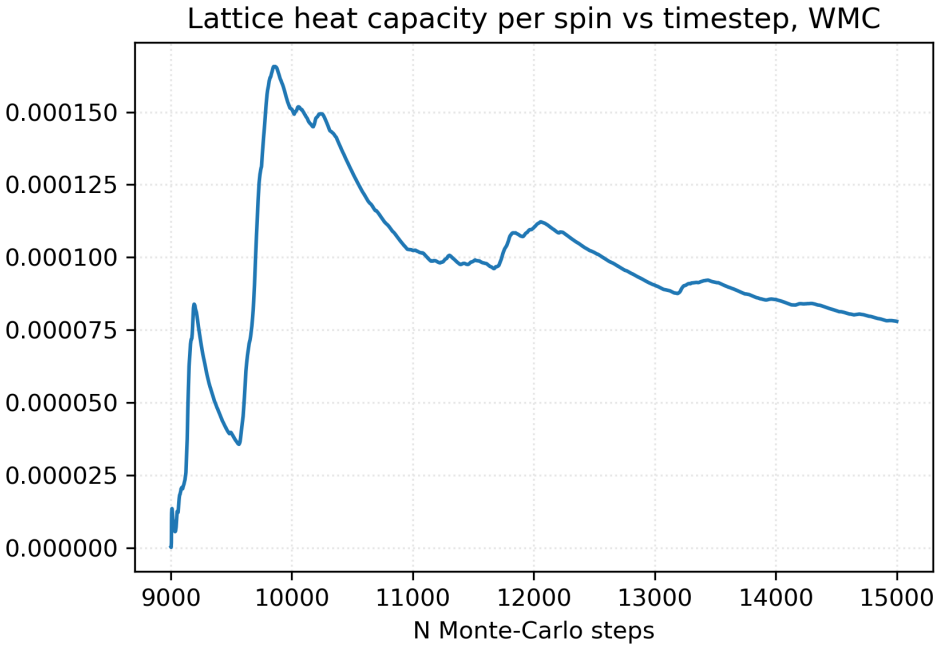


**Figure 153:** Lattice heat capacity per spin vs timestep ($T = 3$ K)

# 11 Wolff Algorithm: Changing the Exchange Constant $J$ at $T_{crit}$

## 11.1 $J = 0.01$

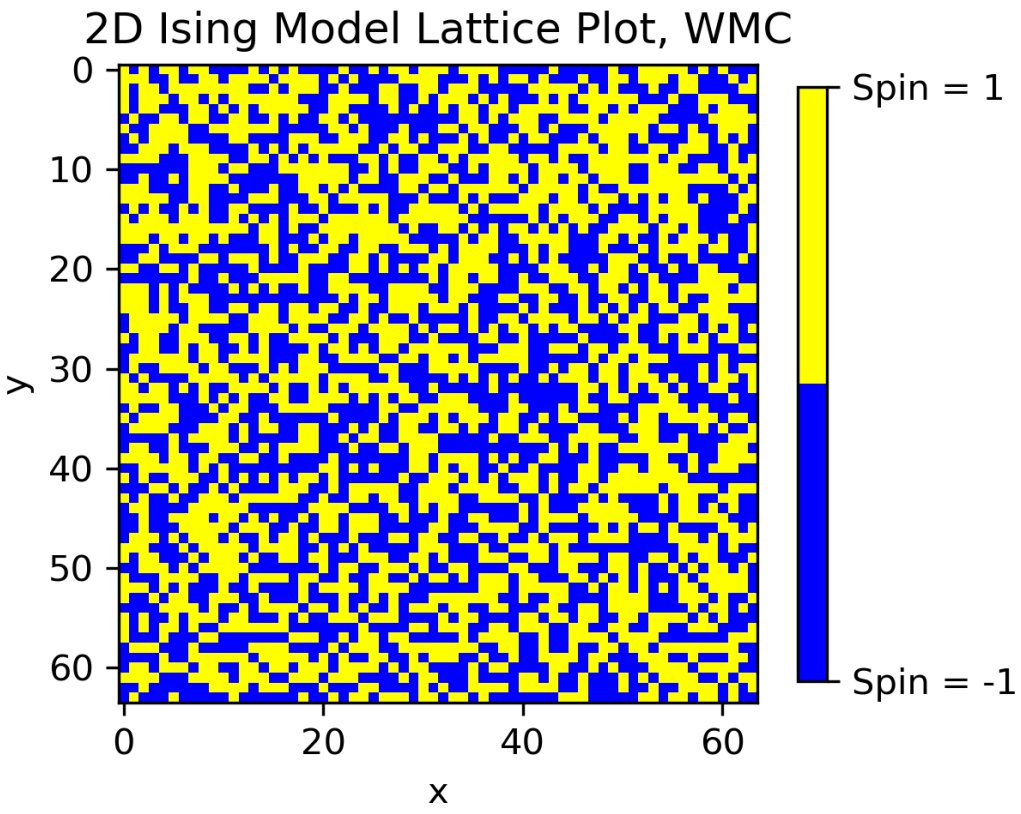


**Figure 154:** End lattice configuration ($J = 0.01$)

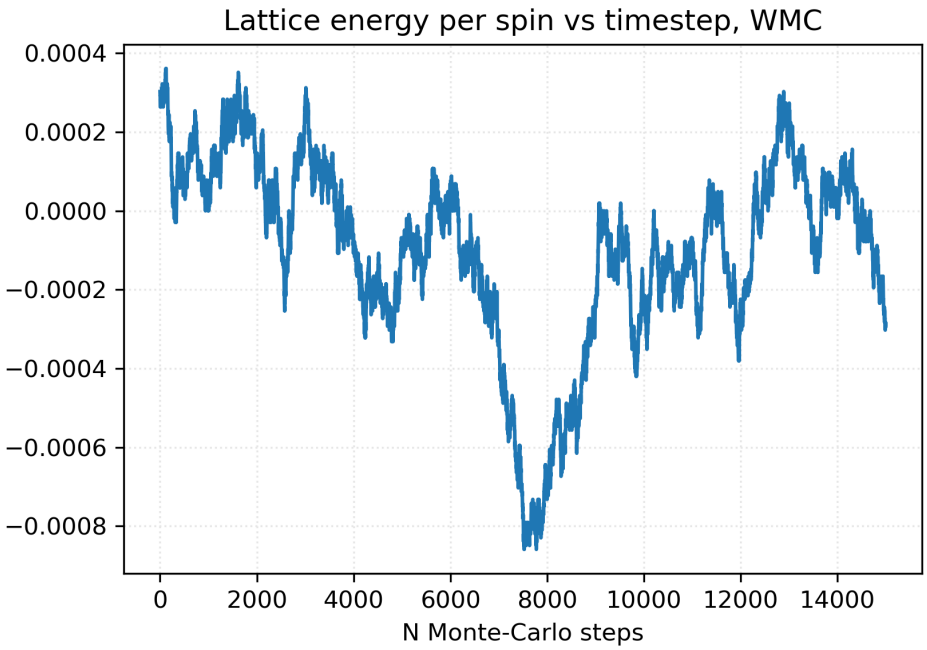


**Figure 155:** Lattice energy per spin vs timestep ($J = 0.01$)

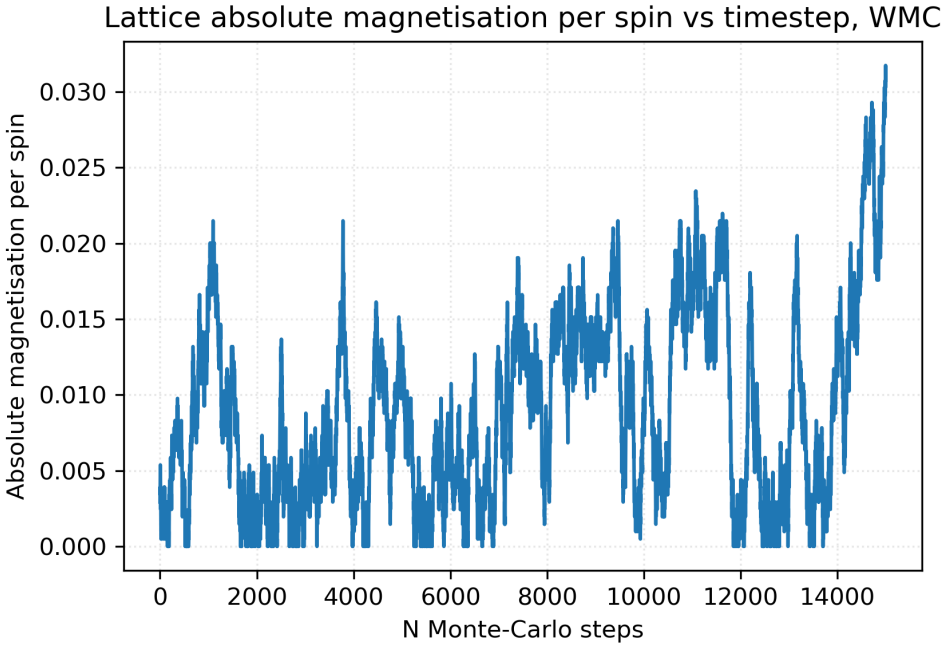


**Figure 156:** Lattice magnetisation per spin vs timestep ($J = 0.01$)

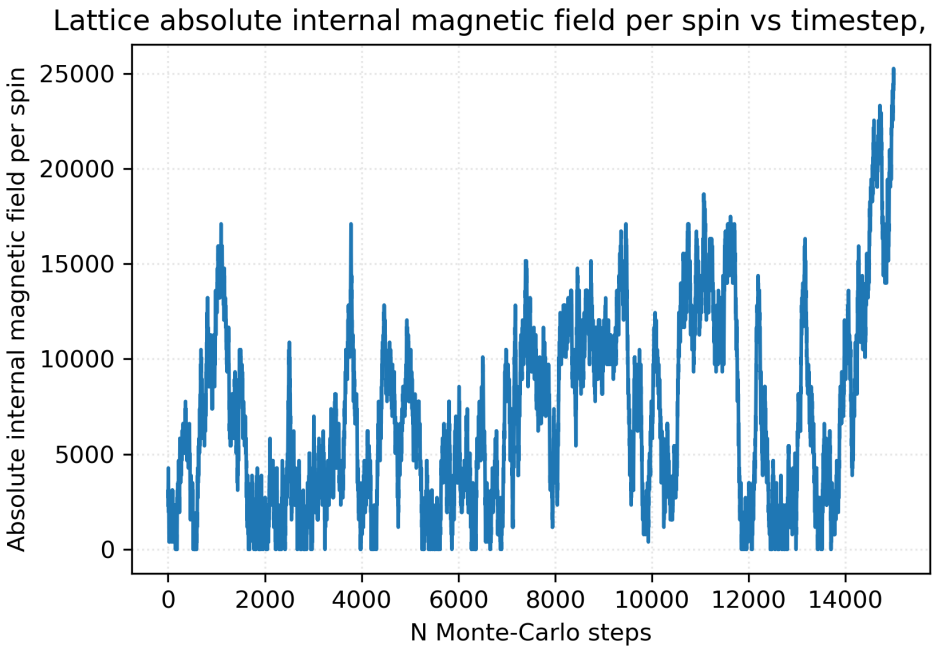


**Figure 157:** Internal magnetic field per spin vs timestep ($J = 0.01$)

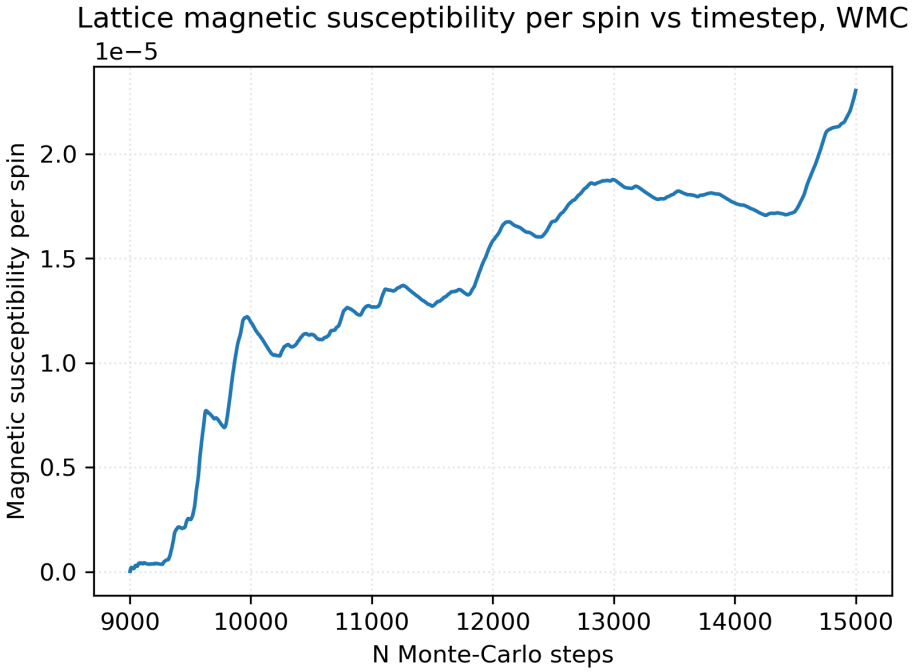


**Figure 158:** Magnetic susceptibility per spin vs timestep ($J = 0.01$)

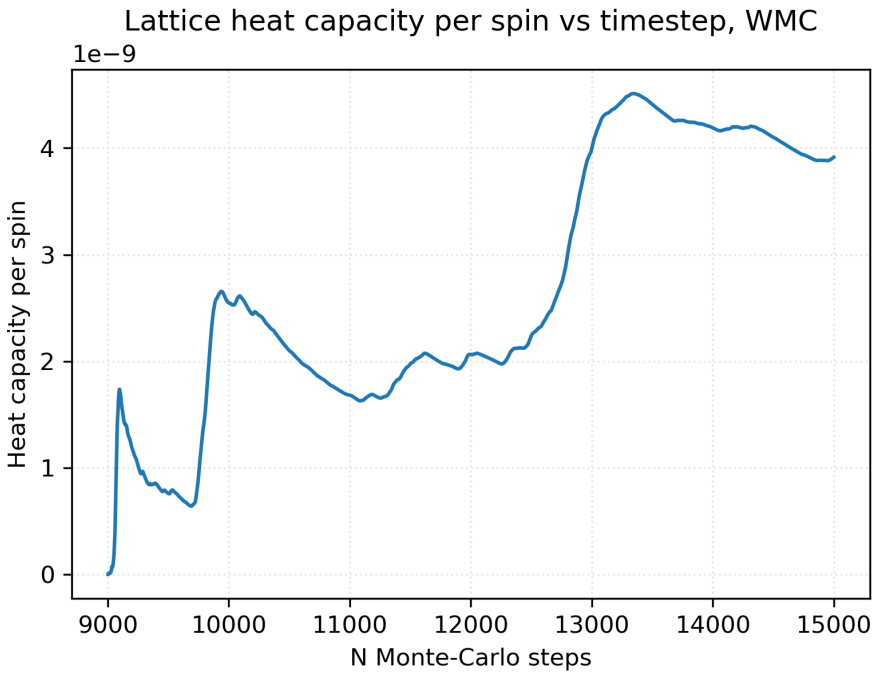


**Figure 159:** Lattice heat capacity per spin vs timestep ($J = 0.01$)

## 11.2 $J = 0.1$

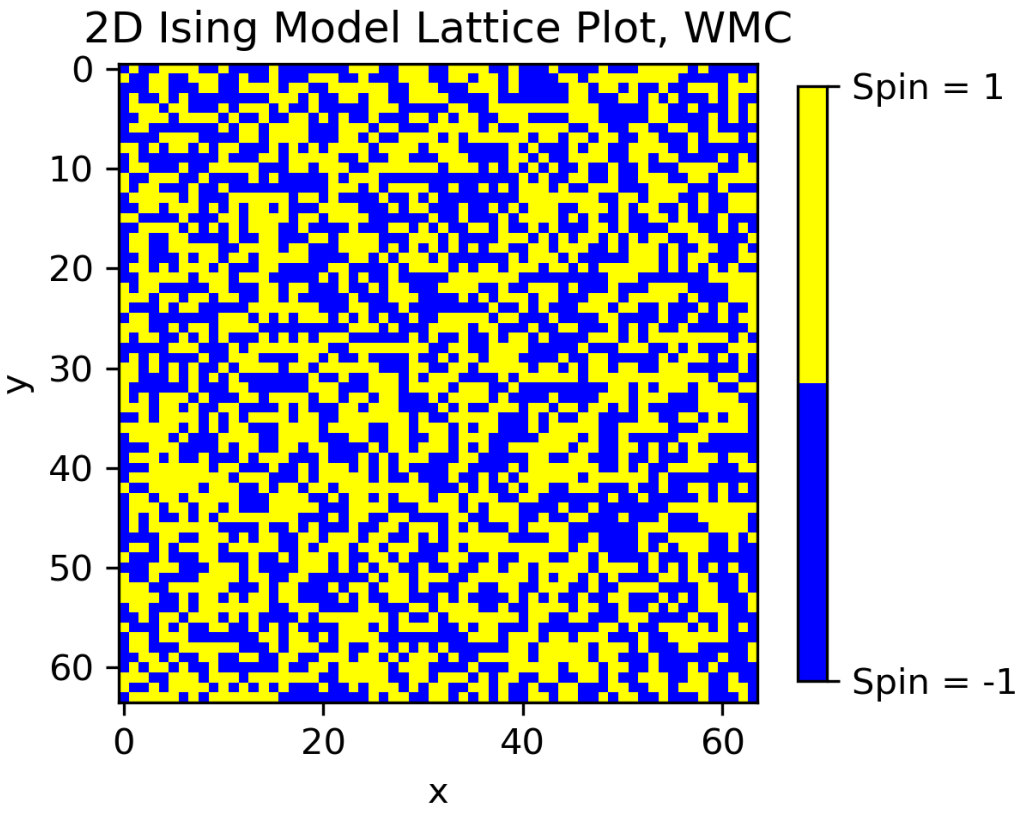


**Figure 160:** End lattice configuration ($J = 0.1$)

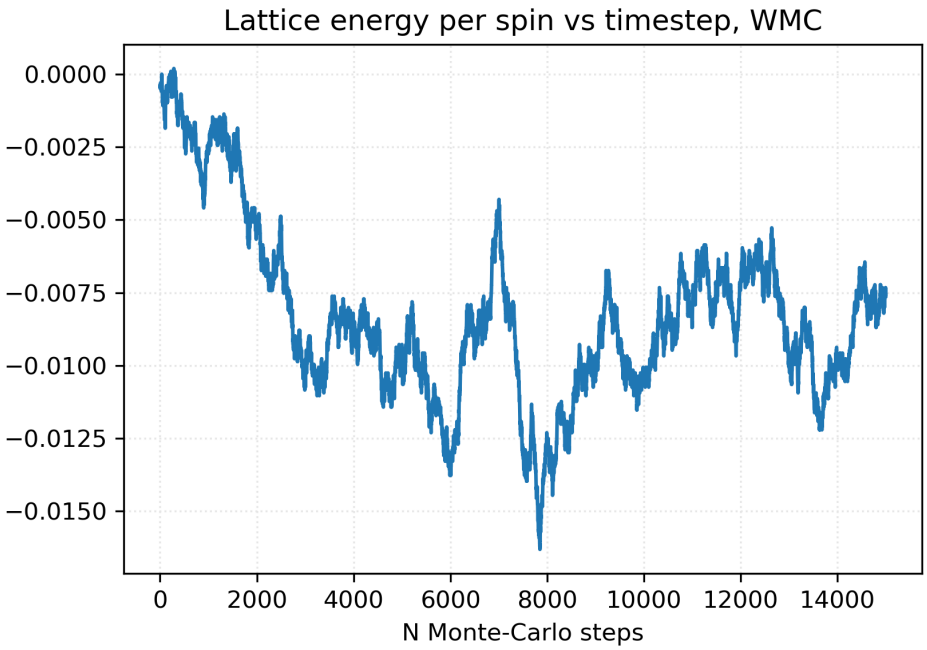


**Figure 161:** Lattice energy per spin vs timestep ($J = 0.1$)

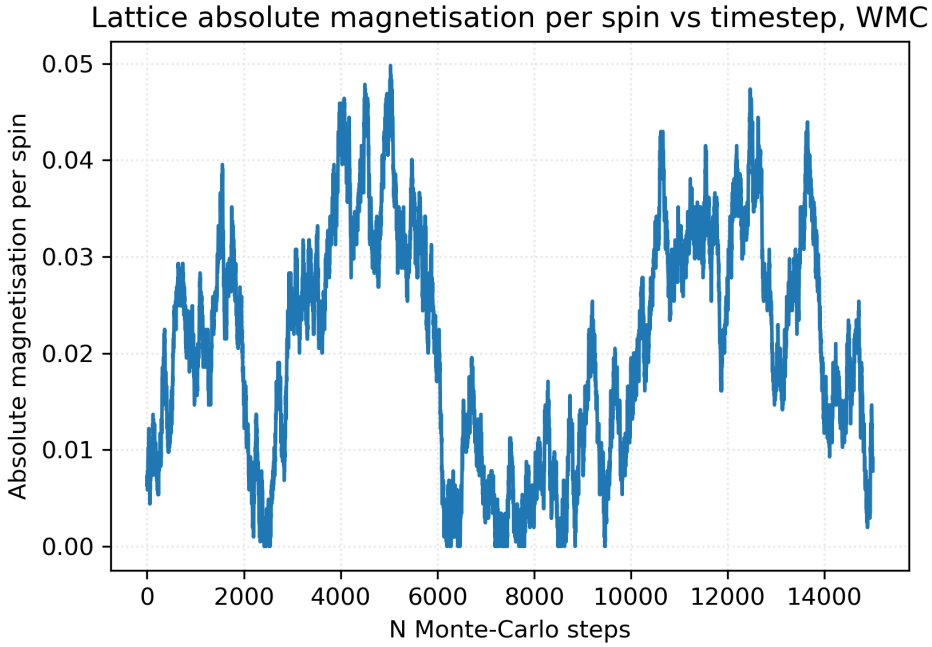


**Figure 162:** Lattice magnetisation per spin vs timestep ($J = 0.1$)

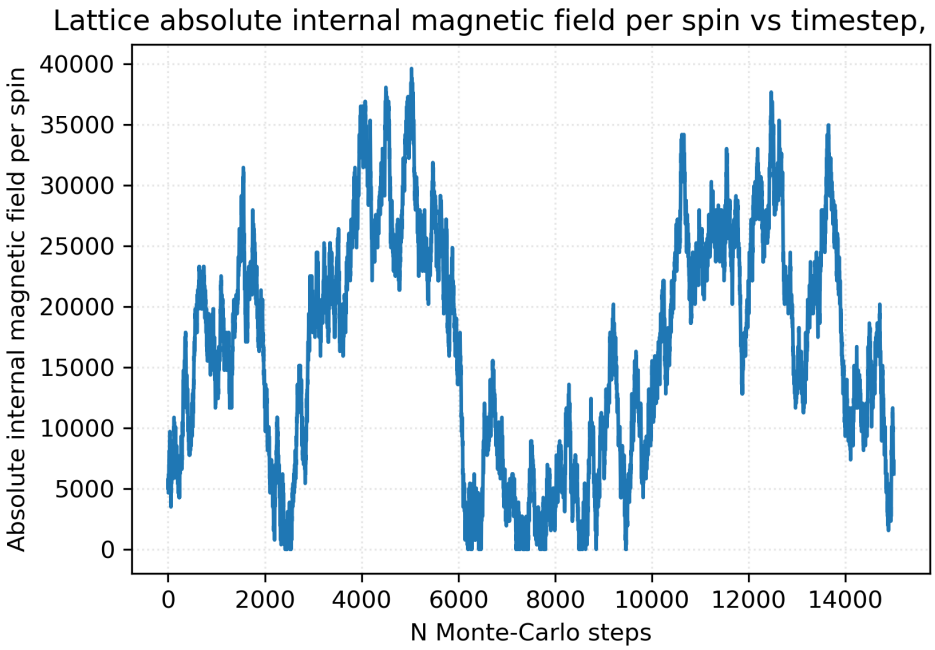


**Figure 163:** Internal magnetic field per spin vs timestep ($J = 0.1$)

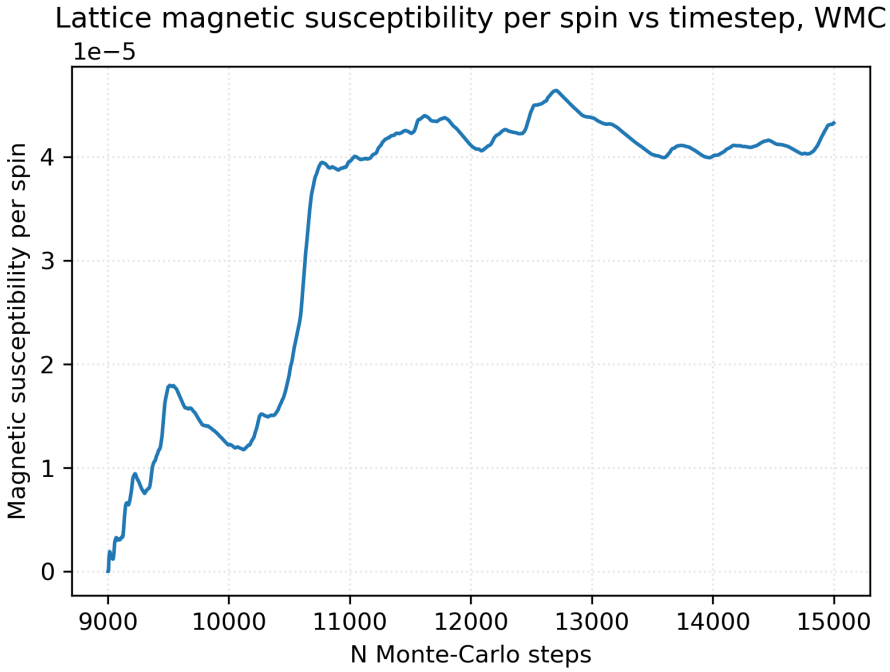


**Figure 164:** Magnetic susceptibility per spin vs timestep ($J = 0.1$)

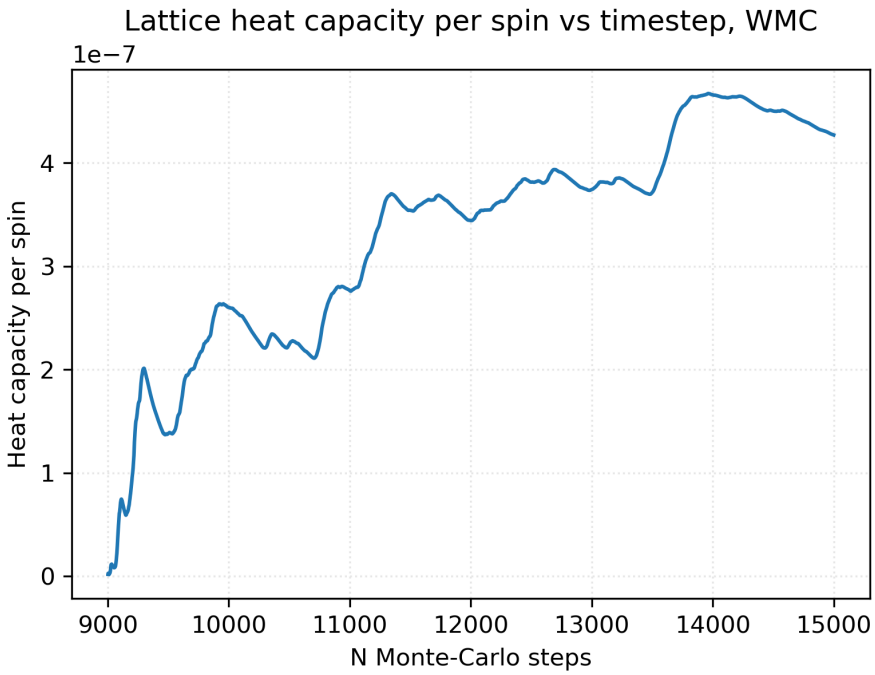


**Figure 165:** Lattice heat capacity per spin vs timestep ($J = 0.1$)

### 11.3 $J = 10$

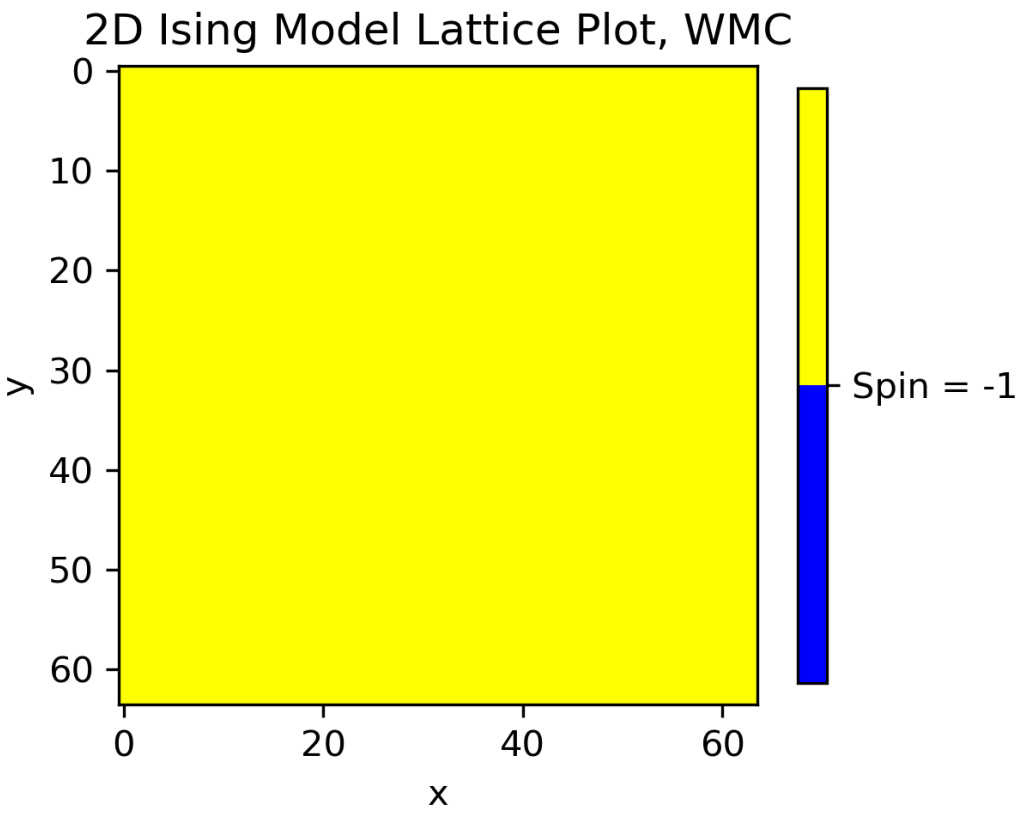


**Figure 166:** End lattice configuration ($J = 10$)

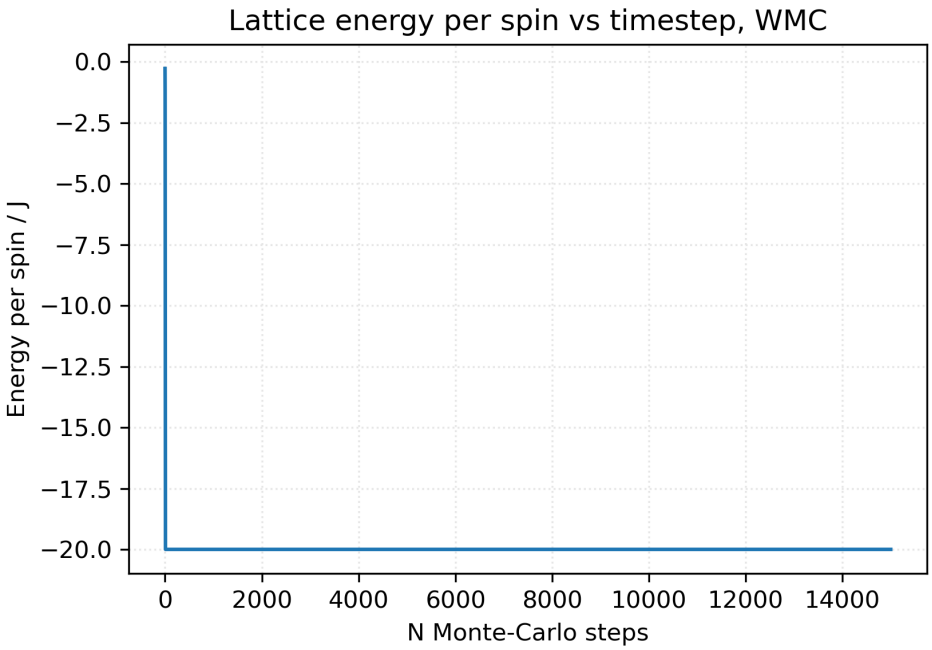


**Figure 167:** Lattice energy per spin vs timestep ($J = 10$)

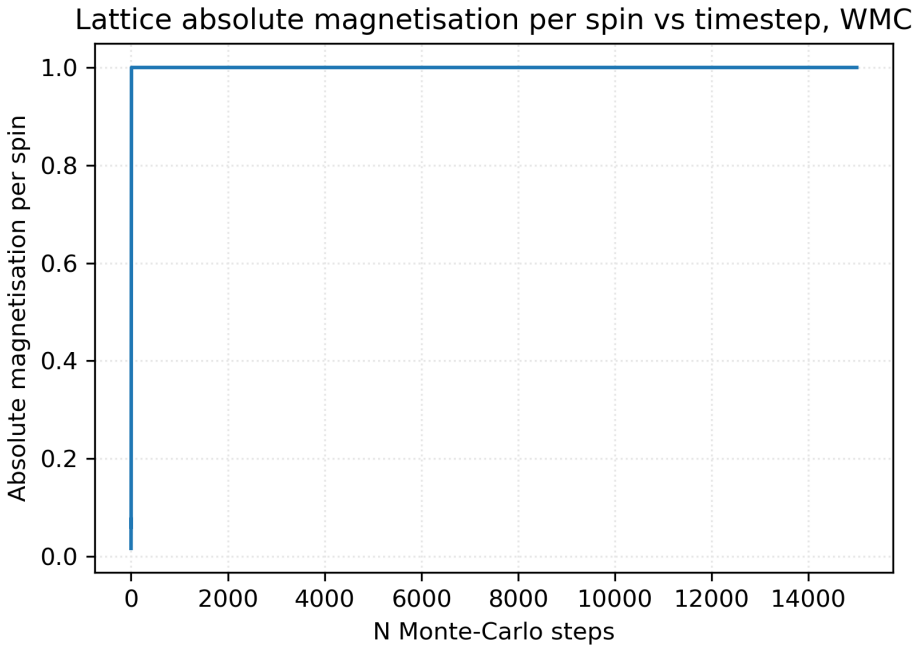


**Figure 168:** Lattice magnetisation per spin vs timestep ($J = 10$)

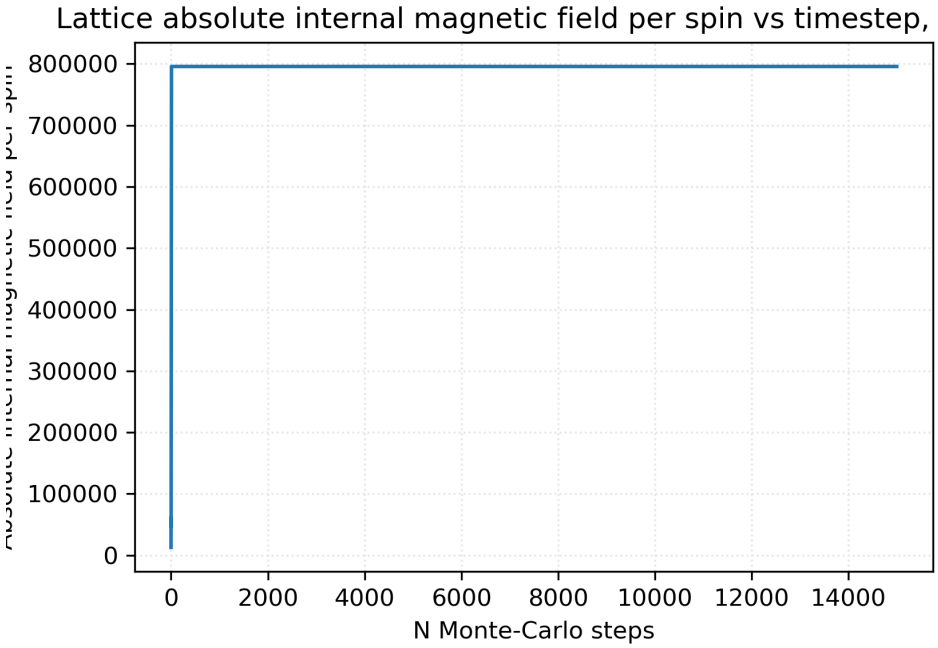


**Figure 169:** Internal magnetic field per spin vs timestep ($J = 10$)

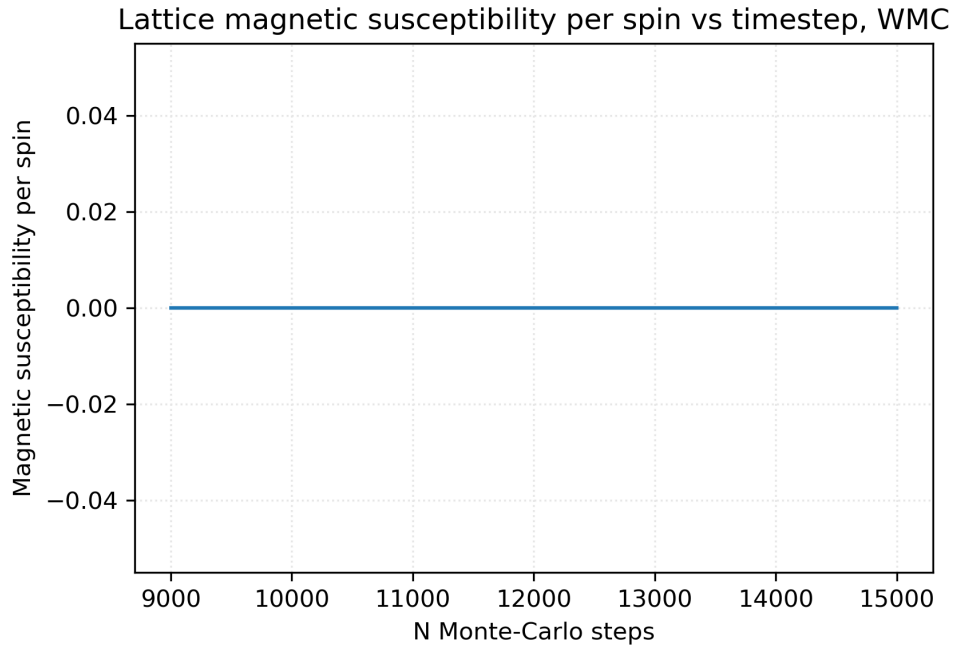


**Figure 170:** Magnetic susceptibility per spin vs timestep ($J = 10$)

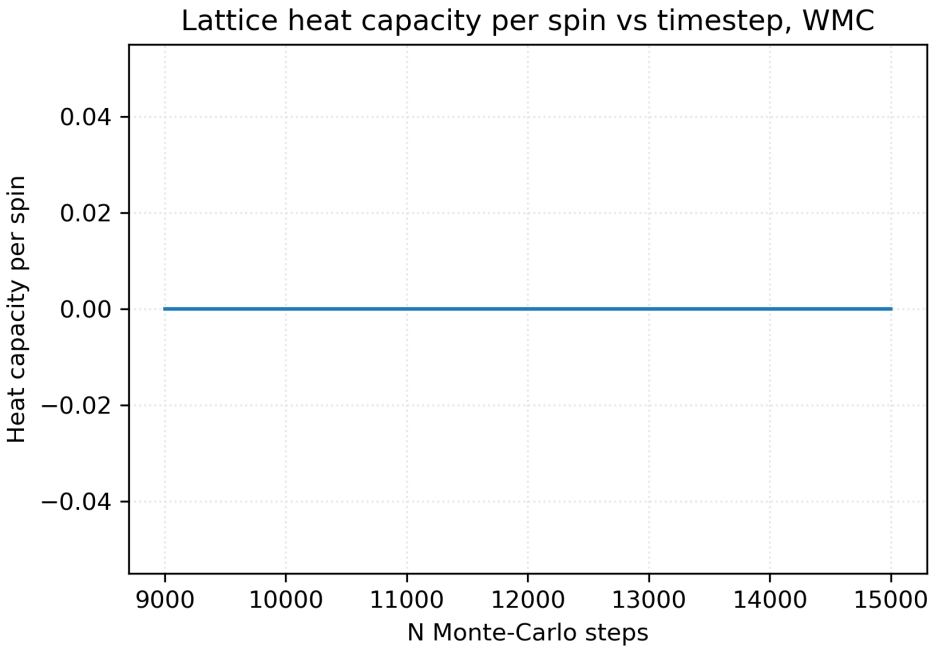


**Figure 171:** Lattice heat capacity per spin vs timestep ($J = 10$)

## 11.4 $J = 100$

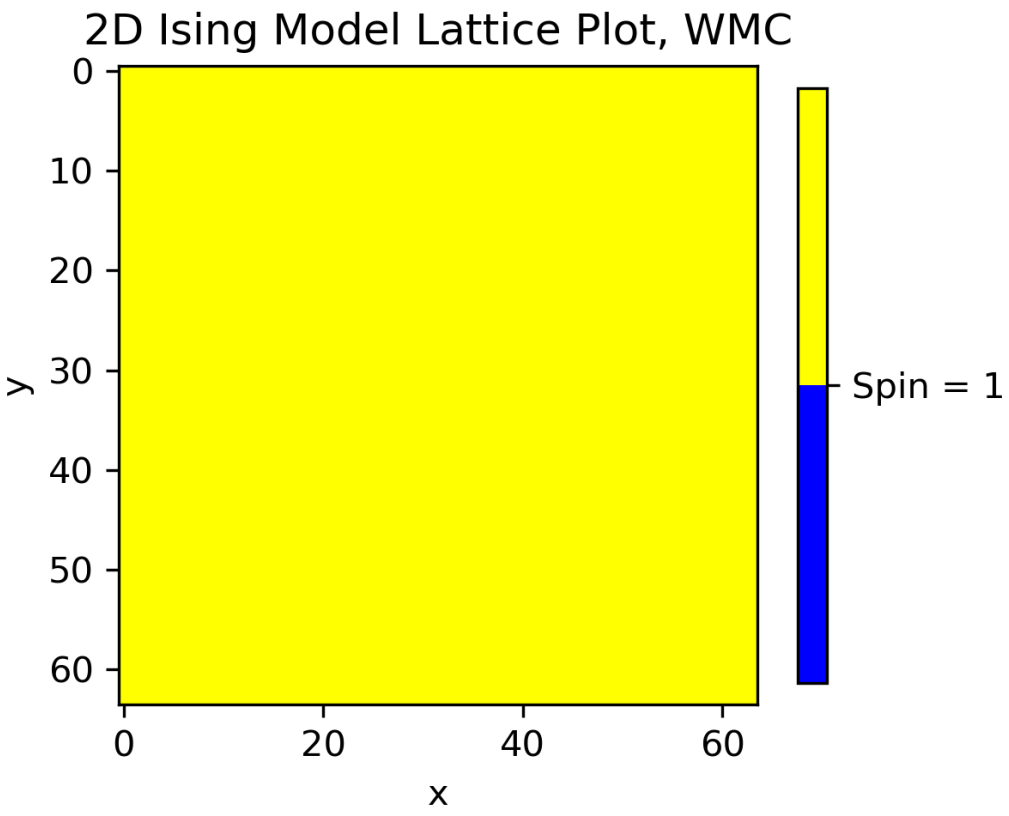


**Figure 172:** End lattice configuration ($J = 100$)

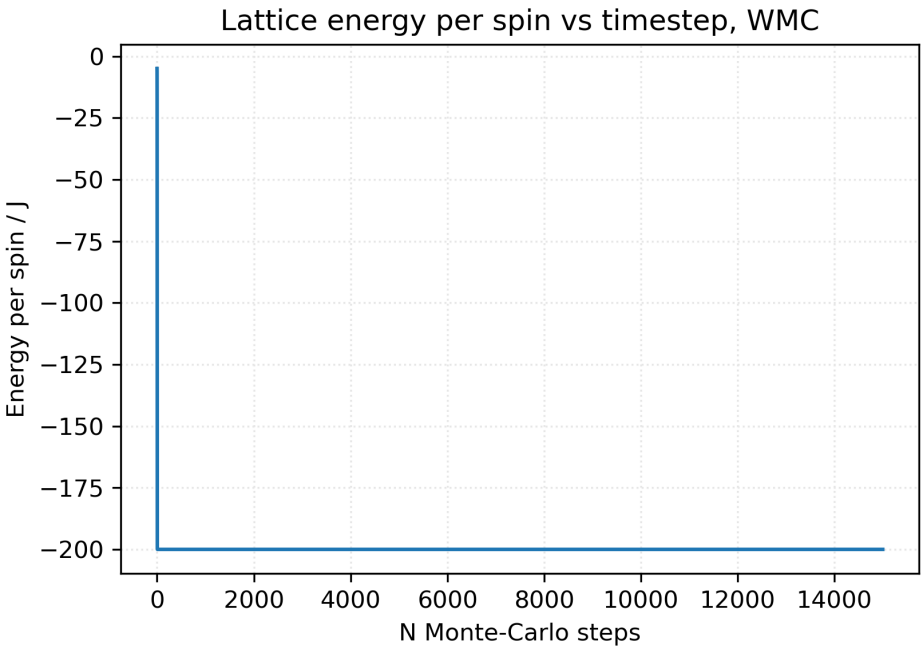


**Figure 173:** Lattice energy per spin vs timestep ($J = 100$)

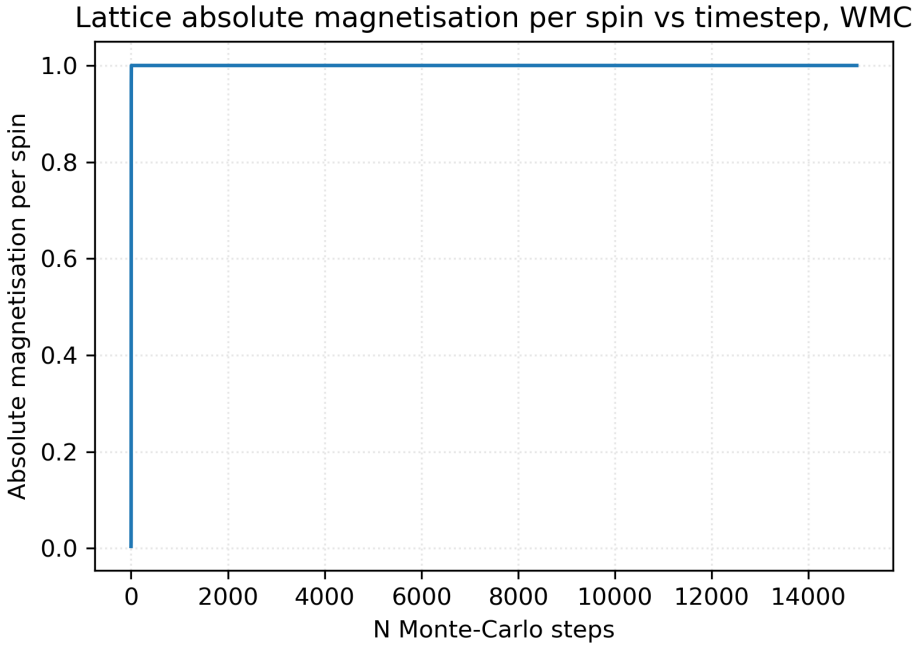


**Figure 174:** Lattice magnetisation per spin vs timestep ($J = 100$)

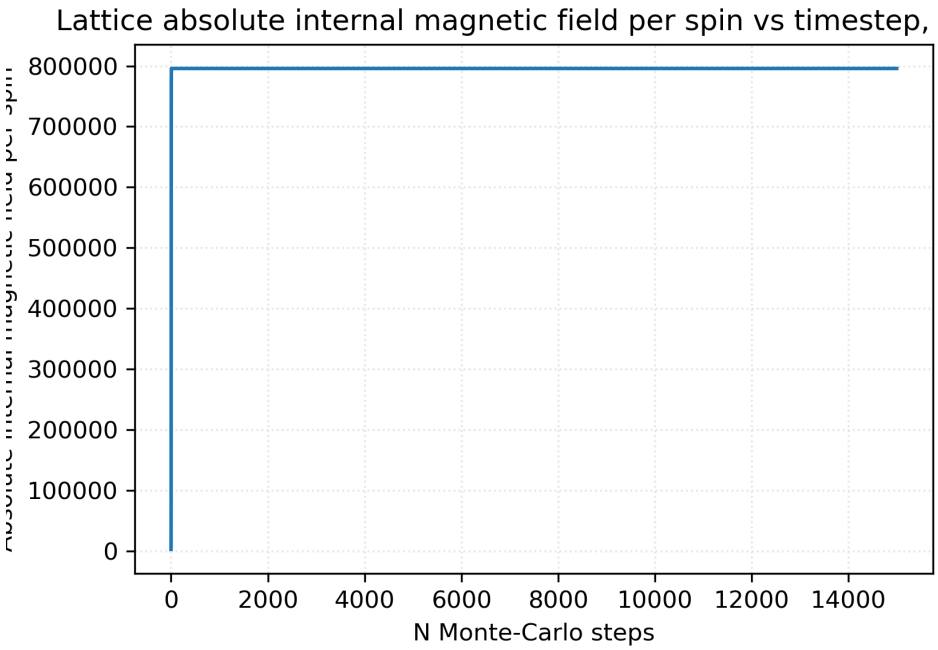


**Figure 175:** Internal magnetic field per spin vs timestep ($J = 100$)

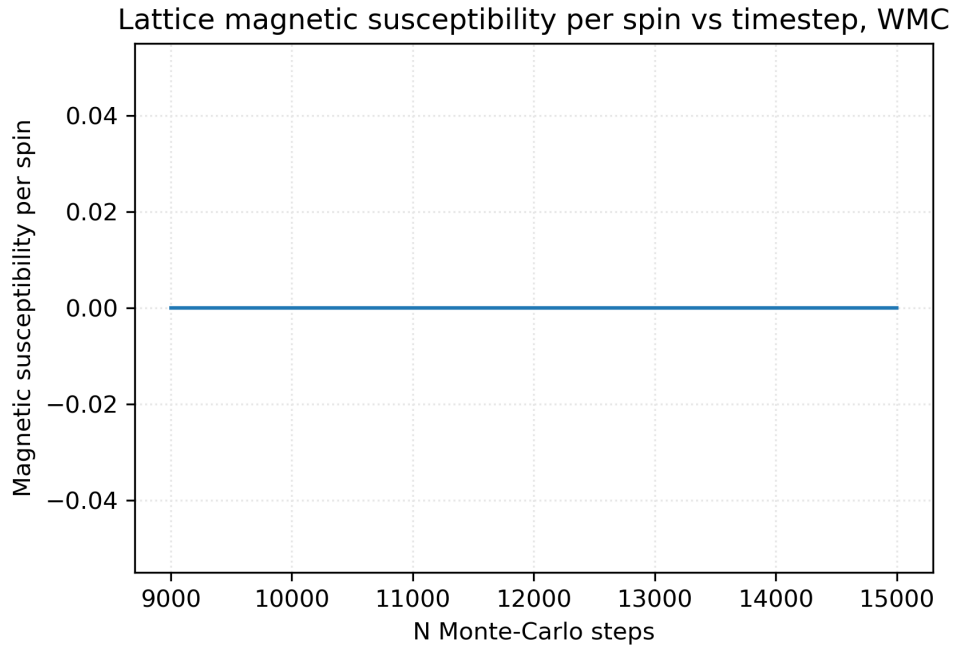


**Figure 176:** Magnetic susceptibility per spin vs timestep ($J = 100$)

# 12 Results for Critical Temperature and Exponent Investigations using the Metropolis Algorithm for a $32 \times 32$ lattice

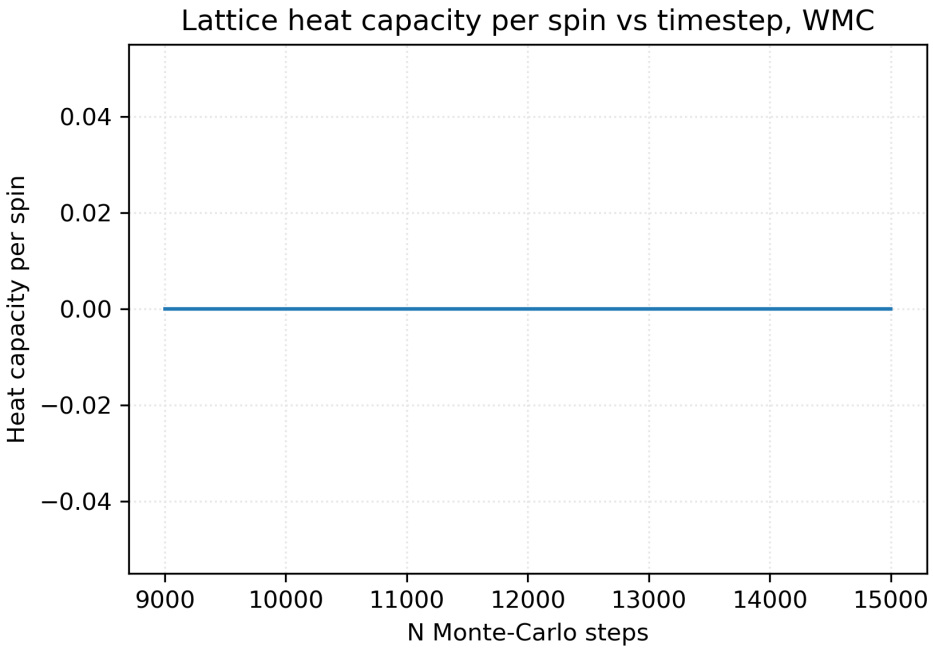


**Figure 177:** Lattice heat capacity per spin vs timestep ($J = 100$)

**Table 2:** Critical temperature results from the Metropolis algorithm on a $32 \times 32$ lattice compared with the Onsager (1944) analytical solution.

| **Method** | $T_c$ **(K)** | **Error (K)** |
|---|---|---|
| Onsager (1944) analytical | 2.2692 | — |
| Magnetisation drop | 2.2575 | 0.0117 |
| Magnetic susceptibility peak | 2.2870 | 0.0178 |
| Heat capacity peak | 2.2480 | 0.0212 |
| Average of three methods | 2.2642 | 0.0050 |

**Table 3:** Onsager (1944) analytical values for the critical exponents of the 2D Ising model.

| **Quantity** | **Onsager Value** |
|---|---|
| Critical amplitude $A$ | 0.4945 |
| $\alpha$ | 0 |
| $\beta$ | 0.125 |
| $\gamma$ | −1.75 |

**Table 4:** Calculated critical exponents from the Metropolis algorithm on a $32 \times 32$ lattice compared with the Onsager (1944) analytical values.

| Exponent | Regime | Onsager | Calculated | Error |
|---|---|---|---|---|
| $-A$ | Above $T_c$ | $-0.4945$ | $-0.3386$ | 0.1559 |
| $-A$ | Below $T_c$ | $-0.4945$ | $-0.4086$ | 0.0859 |
| $-A$ | Average | $-0.4945$ | $-0.3736$ | 0.1209 |
| $\beta$ | Below $T_c$ | 0.125 | 0.0998 | 0.0252 |
| $\gamma$ | Above $T_c$ | $-1.75$ | $-0.1287$ | 1.6213 |
| $\gamma$ | Below $T_c$ | $-1.75$ | $-1.2933$ | 0.4567 |
| $\gamma$ | Average | $-1.75$ | $-0.7110$ | 1.0390 |

# 13 Results for Critical Temperature and Exponent Investigations using the Wolff Algorithm for a $64 \times 64$ lattice

**Table 5:** Critical temperature results from the Wolff algorithm on a $64 \times 64$ lattice compared with the Onsager (1944) analytical solution.

| Method | $T_c$ (K) | Error (K) |
|---|---|---|
| Onsager (1944) analytical | 2.2692 | — |
| Magnetisation drop | 2.3065 | 0.0373 |
| Magnetic susceptibility peak | 2.3020 | 0.0328 |
| Heat capacity peak | 2.2820 | 0.0128 |
| Average of three methods | 2.2968 | 0.0276 |

**Table 6:** Onsager (1944) analytical values for the critical exponents of the 2D Ising model.

| Quantity | Onsager Value |
|---|---|
| Critical amplitude $A$ | 0.4945 |
| $\alpha$ | 0 |
| $\beta$ | 0.125 |
| $\gamma$ | $-1.75$ |

**Table 7:** Calculated critical exponents from the Wolff algorithm on a $64 \times 64$ lattice compared with the Onsager (1944) analytical values.

| Exponent | Regime | Onsager | Calculated | Error |
|---|---|---|---|---|
| $-A$ | Above $T_c$ | $-0.4945$ | $-0.3944$ | 0.1001 |
| $-A$ | Below $T_c$ | $-0.4945$ | $-0.6394$ | 0.1449 |
| $-A$ | Average | $-0.4945$ | $-0.5169$ | 0.0224 |
| $\beta$ | Below $T_c$ | 0.125 | 0.1658 | 0.0408 |
| $\gamma$ | Above $T_c$ | $-1.75$ | $-0.6951$ | 1.0549 |
| $\gamma$ | Below $T_c$ | $-1.75$ | $-2.1785$ | 0.4285 |
| $\gamma$ | Average | $-1.75$ | $-1.4368$ | 0.3132 |

# 14 $C_0$ results using the Metropolis Algorithm

**Table 8:** Calculated value of $C_0$ from the Metropolis algorithm, obtained by extrapolating the heat capacity per spin to infinite lattice size over a range of $D = 16$ to $D = 1024$ with $1.5 \times 10^7$ steps per run.

| Quantity | Value |
|---|---|
| Onsager $C_0$ | 0.2473 |
| Calculated $C_0$ | 0.2411 |
| Error | 0.0061 |

# 15 $C_0$ results using the Wolff Algorithm

**Table 9:** Calculated value of $C_0$ from the Wolff algorithm, obtained by extrapolating the heat capacity per spin to infinite lattice size over a range of $D = 16$ to $D = 16384$ with $1.5 \times 10^4$ steps per run.

| Quantity | Value |
|---|---|
| Onsager $C_0$ | 0.2473 |
| Calculated $C_0$ | 0.2463 |
| Error | 0.0010 |

# 16 Discussion and Conclusion

## 16.1 Algorithm Comparison

Overall, the Metropolis and Wolff algorithms are two very different, but effective algorithms for simulating the 2D, 2-spin Ising Model and modelling phase transitions. The strength of the Wolff algorithm lies in its ability to flip multiple spins at once by growing and then flipping clusters of spins, reaching equilibrium very quickly, far faster than the Metropolis Algorithm. The Wolff algorithm is also resistant to becoming stuck in what is known as a local optimum, where the metropolis algorithm lattice finds itself in a state where around half of its spins are pointing up, and the other half of the spins are pointing down. It is unable to escape from this state in any reasonable timeframe because it is energetically unfavourable for the metropolis algorithm to flip any of the spins, because it is only able to flip one at a time and the system is in a metastable (unstable only over astronomically long periods), even though in a real life system, at temperatures below the critical temperature complete and spontaneous magnetisation occurs. [23] also reports this exact phenomena, and it is also visible even in the figures from Faris 2024 in the theory section. It is important to note that this phenomena occurred in Figure 46:

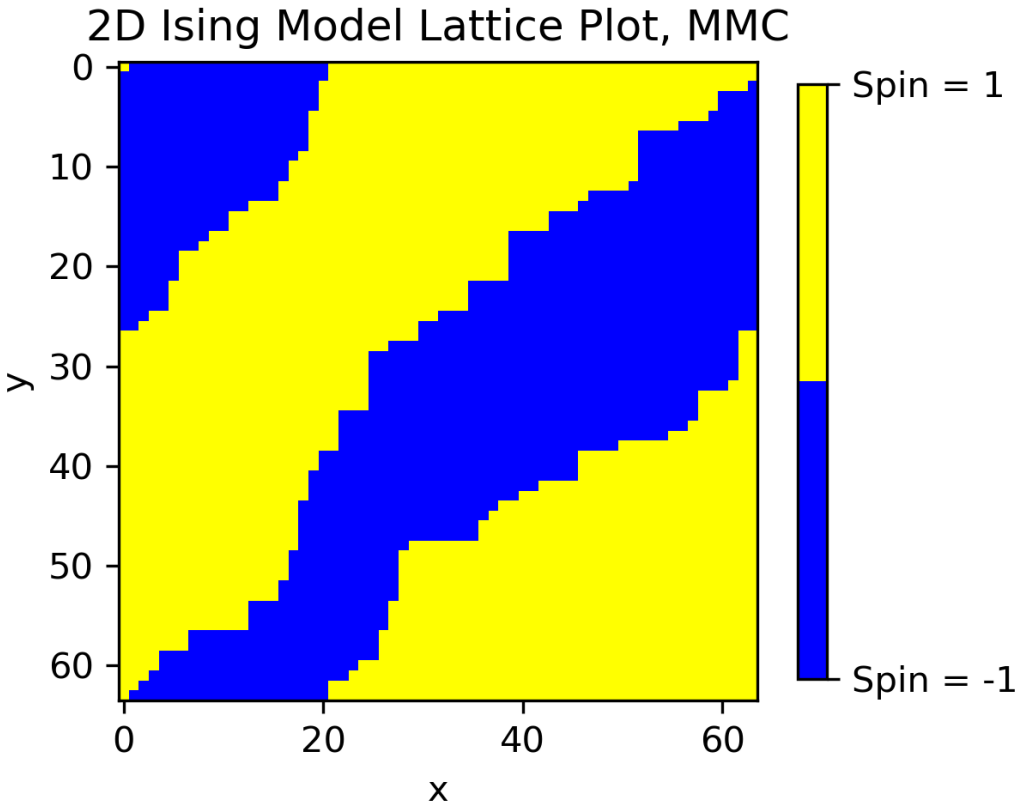


**Figure 178:** Figure 46, showing the end lattice configuration of a Metropolis Algorithm Simulation near absolute zero that became stuck in a metastable state

However, this was not seen in the equivalent Wolff algorithm simulation, Figure 130, which reached a perfect equilibrium state with all of its spins aligned as expected near absolute zero:

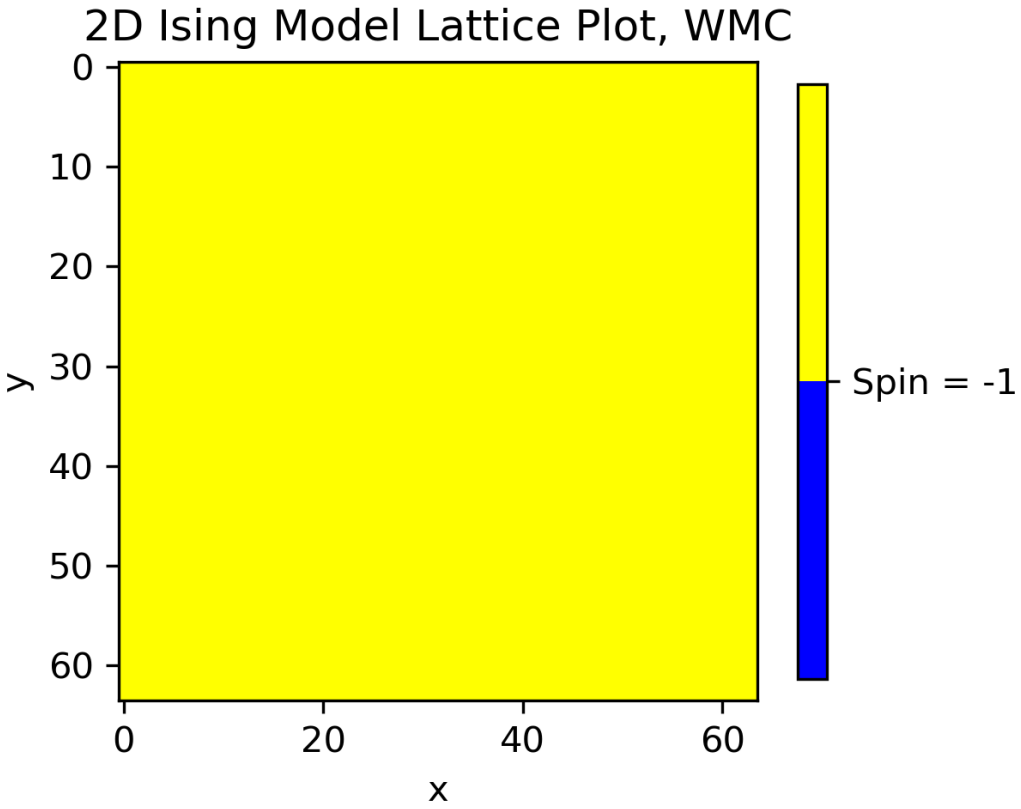


**Figure 179:** Figure 130, showing the end lattice configuration of a Wolff Algorithm Simulation near absolute zero that successfully escaped from a metastable optima and reached equilibrium successfully as expected at such a low temperature.

However, as good as the Wolff algorithm is for escaping from metastable states, it is unable to account for magnetic induction fields, for which the metropolis algorithm is actually far more viable as the probability of an individual spin flip is much more favourable and the Metropolis algorithm can easily account for a magnetic field.

## 16.2 Temperature and exponents comparison

Surprisingly for the critical temperature investigation, despite the Wolff algorithm being more advanced the the metropolis algorithm in terms of being able to reach equilibrium more efficiently, the metropolis algorithm produced more accurate results for the critical temperature, despite from the graphs it can be clearly seen between 2.1K ¡= T ¡= 2.4K the metropolis algorithm clearly ran into metastable states that the Wolff algorithm did not. This was despite the metropolis algorithm also running on a smaller, $32 \times 32$ lattice, which should have produced

more errors due to finite size effects that affect small systems. However, for calculating the critical exponents, the Wolff algorithm produces far better results than the metropolis algorithm, with smoother plots due to the algorithm's ability to escape from unrealistic metastable states.

## 16.3 $C_0$ investigation comparison

As expected, the Wolff algorithm produced a more accurate result for $C_0$ than the metropolis algorithm due to the faster time to reach equilibrium and the Wolff algorithm's ability to escape from metastable states, which the metropolis algorithm lacks.

## 16.4 Comparing individual lattice runs for both algorithms

### 16.4.1 Changing lattice size

For both algorithms, as lattice size increases, the spins get fewer chances to flip for a set number of steps, which means as the lattice size increases it becomes harder and harder for the system to equilibrate. The simulations also take much longer to run as lattice size increases due to a doubling of side length meaning the amount of spins quadruples for a finite system. However, as spin size increases, the plots for energy, magnetisation, magnetic susceptibility and heat capacity have reduced noise. This is because as lattice size increases, the strength of anomalous edge effects is reduced, since a bigger lattice is a much better approximation of Onsager's analytical infinite lattice.

### 16.4.2 Changing the temperature

As temperature increases, for both algorithms this serves to increase the amount of randomness in the system. At temperatures below $T_{crit} = 2.269185K$ the system remains almost perfectly magnetised, at $T = T_{crit}$, the system is exactly in the middle of a phase transition and the spin configuration is 'undecided' on being ordered or disordered, and as such has enormous fluctuations that contribute to peaks in energy and magnetisation variance - Explaining the peaks in heat capacity and magnetic susceptibility.

### 16.4.3 Changing the exchange interaction constant J

At low values of the exchange interaction constant J, the lattice is incredibly disordered, and almost appears as if nothing has been done to it in the entire simulation from its randomly configured initial state. The plots of the energy per spin, magnetisation per spin and internal magnetic field against timestep are incredibly noisy, however, the plots of heat capacity and magnetic susceptibility appear to have converged. At high values of the exchange constant J, the lattices behave in a vastly different way; all quantities of interest converge incredibly quickly as a high value of J makes the spins align very quickly for the Metropolis Algorithm and almost

immediately for the Wolff Algorithm. At high values of J, magnetic susceptibility and Heat Capacity remain at 0.

### 16.4.4 Changing the Magnetic Induction Field B (Metropolis algorithm only)

As the external magnetic induction field is changed, depending on the strength of the magnetic field the lattice will tend to reach an equilibrium state where the vast majority of the spins point in the same direction of the magnetic field, since the implementation of the Metropolis Algorithm in this code is for a ferromagnetic rather than an anti-ferromagnetic material (spins align with, rather than against a magnetic induction field). The stronger the magnetic field is, the faster the spins will align.

# References


[1] L. Onsager, "Crystal statistics. i. a two-dimensional model with an order-disorder transition," *Physical review*, vol. 65, no. 3-4, p. 117, 1944.

[2] C. N. Yang, "The spontaneous magnetization of a two-dimensional ising model," *Physical Review*, vol. 85, no. 5, p. 808, 1952.

[3] E. Barouch, B. M. McCoy, and T. T. Wu, "Zero-field susceptibility of the two-dimensional ising model near t c," *Physical Review Letters*, vol. 31, no. 23, p. 1409, 1973.

[4] S. J. Blundell and K. M. Blundell, *Concepts in Thermal Physics*. Oxford: Oxford University Press, 2nd ed., 2010.

[5] G. Cossu, L. Del Debbio, T. Giani, A. Khamseh, and M. Wilson, "Machine learning determination of dynamical parameters: The ising model case," *Physical Review B*, vol. 100, no. 6, p. 064304, 2019.

[6] A. Repp and I. Szapudi, "An ising model for galaxy bias," *arXiv preprint arXiv:1909.09171*, 2019.

[7] T. S. Farias, V. V. Schultz, J. C. Mombach, and J. Maziero, "A differentiable programming framework for spin models," *Computer Physics Communications*, vol. 302, p. 109234, 2024.

[8] I. O. Morales, E. Landa, C. C. Angeles, J. C. Toledo, A. L. Rivera, J. M. Temis, and A. Frank, "Behavior of early warnings near the critical temperature in the two-dimensional ising model," *PloS one*, vol. 10, no. 6, p. e0130751, 2015.

[9] Z. H. Min, "Monte carlo study on ferromagnetic phase transition of two-dimensional ising model," in *2016 Joint International Information Technology, Mechanical and Electronic Engineering Conference*, pp. 506–509, Atlantis Press, 2016.

[10] D. J. Griffiths, *Introduction to Electrodynamics*. Cambridge University Press, 5th ed., 2023.

[11] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, "Equation of state calculations by fast computing machines," *The journal of chemical physics*, vol. 21, no. 6, pp. 1087–1092, 1953.

[12] M. E. O'Neill, "PCG, a family of better random number generators," 2014.

[13] M. E. O'Neill, "Pcg: A family of simple fast space-efficient statistically good algorithms for random number generation," Tech. Rep. HMC-CS-2014-0905, Harvey Mudd College, Claremont, CA, Sept. 2014.

[14] U. Wolff, "Collective monte carlo updating for spin systems," *Physical Review Letters*, vol. 62, no. 4, p. 361, 1989.

[15] G. Barkema and M. Newman, "New monte carlo algorithms for classical spin systems," *arXiv preprint cond-mat/9703179*, 1997.

[16] B. V. Kryzhanovsky, M. Y. Malsagov, and I. M. Karandashev, "Dependence of critical parameters of 2d ising model on lattice size," *Optical Memory and Neural Networks*, vol. 27, no. 1, pp. 10–22, 2018.

[17] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, Sept. 2020.

[18] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[19] J. D. Hunter, "Matplotlib: A 2d graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[20] Python Software Foundation, *Python Language Reference, version 3.x*, 2024.

[21] C. O. da Costa-Luis, "tqdm: A fast, extensible progress meter for python and cli," *Journal of Open Source Software*, vol. 4, no. 37, p. 1277, 2019.

[22] B. P. Welford, "Note on a method for calculating corrected sums of squares and products," *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.

[23] T. Fricke, "Monte carlo investigation of the ising model," *preprint*, 2006.